

MesoConnect Atlas Tutorial

Supplementary Methods

Corridor-constrained tractography and along-tract microstructure for mesolimbic pathways

September 2026

<https://diffusientensorimaging-repos.github.io/MesoConnect-Tutorial/>

Introduction

The MesoConnect Atlas is a probabilistic atlas of mesolimbic white-matter pathways derived from 7 T diffusion magnetic resonance imaging data of the Human Connectome Project (HCP; Vu et al., 2015). It covers connections among the ventral tegmental area (VTA), hippocampus, nucleus accumbens, ventral pallidum and amygdala. This tutorial describes how to reconstruct those pathways in new participants using the atlas as an anatomical constraint on participant-level tractography, and how to quantify microstructure along the reconstructed bundles.

Scope

The procedure comprises nine steps. The atlas map for a tract is registered to each participant and dilated into a corridor; probabilistic tractography is run through the corridor from the pathway's seed region to its target; the resulting bundle is cleaned of outlying streamlines; and scalar maps, fractional anisotropy (FA) and the indices of neurite orientation dispersion and density imaging (NODDI; Zhang et al., 2012), are sampled at 100 equidistant points along the bundle. Group-level inference may be performed at three resolutions, the whole tract, quartiles of the tract, or individual nodes, and the tutorial describes the model and correction appropriate to each. Node-wise results may be inspected in the Node-wise Tract Explorer, a browser-based viewer distributed with this site that renders *t*-value profiles, clusters and hemispheric comparisons from a results file.

Two alternative uses of the atlas, averaging a scalar map within the warped atlas mask and sampling along an atlas-derived centerline, are described in the appendix. Corridor-constrained tractography is presented as the primary method because the streamlines are estimated from each participant's own data and support along-tract analysis.

Prerequisites

The tutorial assumes preprocessed diffusion data (denoised, corrected for susceptibility and eddy-current distortion, and brain-masked), a T1-weighted anatomical image, and a Unix environment with FSL (Jenkinson et al., 2012), ANTs (Avants et al., 2008), MRtrix3 (Tournier et al., 2019) and Python with DIPY (Garyfallidis et al., 2014) and pyAFQ (Kruper et al., 2021). Preprocessing is documented in the TUBRIC DTI tutorial; QSIPrep produces equivalent inputs. Estimation of fibre orientation distributions, which that pipeline does not include, is described on the software page.

Organization

Table S1 summarizes the sections of the site.

Table S1

Sections of the Tutorial

Section	Contents
Atlas	File types, tract families, sources of the seed and target regions, downloads
Workflow	The nine steps, each with parameters, verification criteria, the full script, and results from an example dataset
Explorer	The results viewer and its input format
Alternative approaches	Whole-tract extraction; atlas-guided synthetic streamlines
Reference	Consolidated parameters, troubleshooting, software, scripts, suggested methods text, abbreviations, citation, references

Example dataset

Numerical results and figures on the workflow pages are drawn from a single dataset of 57 adults scanned at 3 T with a multi-shell protocol ($b = 1000, 2000, 3250$ and 5000 s/mm²) and processed for the posterior and anterior VTA → hippocampus pathways. The atlas itself was constructed at 7 T. The example dataset is included to illustrate expected output at a field strength typical of most studies.

Atlas overview

The MesoConnect Atlas comprises group probabilistic maps of mesolimbic tracts connecting the ventral tegmental area (VTA), hippocampus, nucleus accumbens, ventral pallidum and amygdala. The maps were generated from Human Connectome Project (HCP) 7 T diffusion data (Vu et al., 2015) using probabilistic tractography (Tournier et al., 2010) with tract-specific exclusion masks, streamline cleaning and group-level probabilistic mapping, and are distributed in Montreal Neurological Institute (MNI) space on the FSL MNI152 1 mm grid ($182 \times 218 \times 182$ voxels).

The atlas represents plausible streamline trajectories. It does not establish monosynaptic connectivity, fibre directionality or neurotransmitter identity, and several components of the hippocampal–VTA circuit are polysynaptic.

File types

Each tract is distributed as three files (Table S2).

Table S2

Atlas File Types

Suffix	Contents	Typical use
*_GroupMean_OverlapProp.nii.gz	Probabilistic map. Each voxel holds the proportion of contributing participants whose binarized tract included it (range 0 to 1).	Visualization, threshold selection, probability-weighted extraction
*_GroupMean_thr50.nii.gz	Binary map of voxels present in at least 50% of participants.	Corridor construction, whole-tract extraction
*_GroupOverlapCount.nii.gz	Count map. Each voxel holds the number of participants whose tract included it.	Reporting participant overlap

The 50% threshold is the default for corridor construction. Higher thresholds (75%, 90%) reduce the map toward a single central trajectory and no longer represent between-participant variation. The 25% threshold retains greater extent and is appropriate when a warped 50% core is short or discontinuous.

Modes of use

Three modes of use are supported.

1. *Corridor-constrained tractography*. The warped and dilated atlas defines the region within which participant-level tractography may travel. Streamlines are estimated from the participant's own data. This mode supports along-tract analysis and is the subject of the workflow section.
2. *Whole-tract extraction*. A scalar map is averaged within the warped atlas mask. This mode is suitable for descriptive summaries; spatial variation along the tract is not retained. See the appendix.
3. *Atlas-guided synthetic streamlines*. A centerline scaffold constructed inside the warped atlas is sampled at 100 nodes. This mode is suitable when participant-level tractography fails in a subset of the sample. The scaffold is a sampling coordinate system rather than a reconstruction. See the appendix.

Coordinate spaces and interpolation

Atlas space is FSL MNI152 1 mm space unless stated otherwise. Native space refers to a participant's T1, diffusion or NODDI grid (NODDI: neurite orientation dispersion and density imaging; Zhang et al., 2012). Releases in MNI 2 mm and MNI152NLin2009cAsym space are planned.

Probabilistic maps are warped with linear interpolation and thresholded in native space. Binary maps are warped with nearest-neighbour interpolation. Spline interpolation is not appropriate for label images. In registration software the moving image is the image being transformed (here the atlas) and the fixed image defines the target grid (the participant); flag names differ between packages. Warped masks should be overlaid on the participant's image and inspected before use.

Interpretive limits

A group atlas warped to native space does not substitute for participant-level tractography when the research question depends on individual anatomy. Dilation increases sensitivity at the cost of specificity, and the smallest dilation that supports reliable tracking should be used. Whole-tract averages are summary descriptors. Synthetic streamlines are sampling scaffolds and should be described as such in reports.

Tract families and regions of interest

The atlas contains 10 tract families (Table S3). Each is processed with the same nine-step workflow; the seed, target and atlas file differ per family. The exclusion masks used during atlas construction are already reflected in each atlas corridor. They are listed here so that users can anticipate where a reconstruction may leak when the corridor is dilated too far. Throughout, the ventral tegmental area is abbreviated VTA and a region of interest is abbreviated ROI.

Table S3

Tract Families in the MesoConnect Atlas

Family	Files (left / right)	Seed	Target	Construction notes
VTA → posterior (body) hippocampus	l_vta_l_hipp, r_vta_r_hipp	VTA	hippocampus	Exclusions: fornix, optic tract and nerve, amygdala, ventral pallidum, accumbens, dorsal striatum, thalamus, cortex and cerebellum, brainstem inferior to the VTA, red nucleus, contralateral hemisphere. A ventral secondary bundle occurs in some participants (see Step 6).
VTA → anterior hippocampus	anterior_l_vta_l_hipp, anterior_r_vta_r_hipp	VTA	hippocampus	Shares its course with the posterior tract for approximately the first 60 to 80 of 100 nodes. Same exclusions.
VTA → amygdala	l_vta_l_amygdala, r_vta_r_amygdala	VTA	amygdala (hippocampus)	Optic tract exclusion is required. Fornix

Family	Files (left / right)	Seed	Target	Construction notes
			subtracted)	excluded.
Superior VTA → nucleus accumbens	superior_l_vta_l_accumbens , superior_r_vta_r_accumbens	VTA	accumbens (ventral pallidum subtracted)	Superior and inferior divisions are defined by anterior- commissure inclusion and exclusion masks.
Inferior VTA → nucleus accumbens	inferior_l_vta_l_accumbens , inferior_r_vta_r_accumbens	VTA	accumbens	As above.
Ventral pallidum → VTA	l_vp_l_vta, r_vp_r_vta	ventral pallidum	VTA	Ventral pallidum divided by hemisphere masks.
Hippocampus → nucleus accumbens	l_hipp_l_accumbens, r_hipp_r_accumbens	hippocampus	accumbens	Cutoff 0.08 at 7 T during construction. A QuickBundles division into two clusters was frequently required before cleaning.
Hippocampus → ventral pallidum	l_hipp_l_vp, r_hipp_r_vp	hippocampus	ventral pallidum	As above.
Precommissural fornix	TractSeg fornix $\cap$ hippocampus –accumbens overlap			Derived map, not a corridor target. Currently on an HCP 1.05 mm grid; resample before use.
Postcommissural fornix	TractSeg fornix minus hippocampus–accumbens overlap			As above.

Note. HCP = Human Connectome Project. The example dataset covers the first two families; the downloads page lists the files distributed with this site.

Regions of interest

Seeds and targets are derived from published atlases, thresholded and binarized in Montreal Neurological Institute (MNI) 1 mm space (Table S4). Where a source cannot be redistributed, the region should be rebuilt from the source and the source cited.

Table S4

Sources and Preparation of the Regions of Interest

Region	Source	File stem	Preparation
VTA	7 T probabilistic VTA atlas (Trutti et al., 2021), 25% threshold	left_VTA_0.25_bin, right_VTA_0.25_bin	Red nucleus subtracted where overlapping. The VTA is also subtracted from the lateral hypothalamus and mammillary body exclusion masks.
Hippocampus	Harvard–Oxford subcortical atlas (Frazier et al., 2005; Makris et al., 2006), 50% threshold	HPC_L_0.5_bin, HPC_R_0.5_bin	
Amygdala	Harvard–Oxford subcortical atlas, 50% threshold	amygdala_0.5_bin → left_amygdala_bin, right_amygdala_bin	Hippocampus subtracted; divided by hemisphere masks.
Nucleus accumbens (limbic striatum)	Connectivity-based striatal parcellation (Tziortzi et al., 2014), or FSL accumbens	left_accumbens_bin, right_accumbens_bin	Ventral pallidum subtracted.
Ventral pallidum	Subcortical atlas (Pauli et al., 2018)	ventral_pallidum_bin → left_, right_	Divided by hemisphere masks.
SN/VTA (alternative)	Functional–anatomical SN/VTA masks (Murty et al., 2014)		Comparison resource for the dopaminergic midbrain.

Note. SN = substantia nigra.

Exclusion masks used during atlas construction

The corridor workflow does not require these masks. They are listed so that users know what the atlas already excludes and can reintroduce any of them as an additional `-exclude` argument if a reconstruction enters a specific structure.

- Thalamus; cortical gray matter with cerebellum (single mask); brainstem inferior to the VTA.
- Dorsal striatum: caudate and putamen, with accumbens and ventral pallidum subtracted.
- Lateral hypothalamus and mammillary bodies, with the VTA subtracted.
- Red nucleus, with the VTA subtracted.
- Fornix body, with hippocampus and amygdala subtracted (VTA → hippocampus and VTA → amygdala).
- Optic tract and optic nerve, per hemisphere (VTA → hippocampus and VTA → amygdala).
- Contralateral hemisphere, enforcing ipsilateral tracking.
- Anterior commissure inclusion and exclusion masks, defining the superior and inferior VTA → accumbens divisions.

The region-tidying operations (subtractions and hemisphere divisions) will be distributed as a script with the full atlas release.

Downloads

Files distributed with this site

The ventral tegmental area (VTA) → hippocampus files used for the example dataset are listed in Table S5. All files are in Montreal Neurological Institute (MNI) space on the FSL MNI152 1 mm grid and are binary unless stated otherwise.

Table S5

Atlas and Region Files Distributed With This Site

File	Hemisphere	Contents
left_VTA_0.25_bin.nii.gz	Left	VTA seed (Trutti et al., 2021; 25% threshold)
right_VTA_0.25_bin.nii.gz	Right	VTA seed (Trutti et al., 2021; 25% threshold)
HPC_L_0.5_bin.nii.gz	Left	Hippocampus target (Harvard–Oxford; 50% threshold)
HPC_R_0.5_bin.nii.gz	Right	Hippocampus target (Harvard–Oxford; 50% threshold)
l_vta_l_hipp_1mm_MNI_GroupMean_thr50.nii.gz	Left	Posterior VTA → hippocampus atlas, 50% threshold
r_vta_r_hipp_1mm_MNI_GroupMean_thr50.nii.gz	Right	Posterior VTA → hippocampus atlas, 50% threshold
l_vta_l_hipp_1mm_MNI_GroupMean_OverlapProp.nii.gz	Left	Posterior VTA → hippocampus atlas, probabilistic (not binary)
r_vta_r_hipp_1mm_MNI_GroupMean_OverlapProp.nii.gz	Right	Posterior VTA → hippocampus atlas,

File	Hemisphere	Contents
		probabilistic (not binary)
anterior_l_vta_l_hipp_1mm_MNI_GroupMean_thr50.nii.gz	Left	Anterior VTA → hippocampus atlas, 50% threshold
anterior_r_vta_r_hipp_1mm_MNI_GroupMean_thr50.nii.gz	Right	Anterior VTA → hippocampus atlas, 50% threshold

The following commands retrieve all files listed above into the folder layout of the full release, which is the layout that `00_config.sh` expects (ATLAS_DIR is the MesoConnectAtlas folder).

```
mkdir -p MesoConnectAtlas/roi_maps MesoConnectAtlas/tracts_thresholded_binary_50 \
        MesoConnectAtlas/tracts_probabilistic
cd MesoConnectAtlas
base="https://diffusiontensorimaging-repos.github.io/MesoConnect-Tutorial/atlas"

for f in left_VTA_0.25_bin right_VTA_0.25_bin HPC_L_0.5_bin HPC_R_0.5_bin; do
    curl -sSL "$base/$f.nii.gz" -o "roi_maps/$f.nii.gz"
done
for t in l_vta_l_hipp r_vta_r_hipp anterior_l_vta_l_hipp anterior_r_vta_r_hipp; do
    f="${t}_1mm_MNI_GroupMean_thr50.nii.gz"
    curl -sSL "$base/$f" -o "tracts_thresholded_binary_50/$f"
done
for t in l_vta_l_hipp r_vta_r_hipp; do
    f="${t}_1mm_MNI_GroupMean_OverlapProp.nii.gz"
    curl -sSL "$base/$f" -o "tracts_probabilistic/$f"
done
```

Full release

The complete atlas package will follow the layout below. The remaining families (amygdala, accumbens divisions, ventral pallidum, hippocampus → accumbens, hippocampus → ventral pallidum, fornix divisions), the count maps, the 25% and 75% thresholds, the endpoint maps and the exclusion masks are available from the atlas authors in the interim.

```
MesoConnectAtlas_v1.0/
  Atlases/FSL_MNI152_1mm/
    tracts_probabilistic/          *_GroupMean_OverlapProp.nii.gz
    tracts_thresholded_binary_25/  *_thr25.nii.gz
    tracts_thresholded_binary_50/  *_thr50.nii.gz
    tracts_thresholded_binary_75/  *_thr75.nii.gz
    tracts_overlap_count/          *_GroupOverlapCount.nii.gz
    endpoint_maps/ roi_maps/ exclusion_masks/
  Atlases/HCP_T1_1p05mm/fornix_split_current_space/
  Scripts/ Examples/ Metadata/ Documentation/
```

Source atlases

- Trutti et al. (2021): probabilistic VTA in MNI space; threshold at 25% for a seed region.
- Pauli et al. (2018): subcortical atlas including ventral pallidum, VTA and substantia nigra.
- Harvard–Oxford subcortical atlas (Frazier et al., 2005; Makris et al., 2006), distributed with FSL: hippocampus, amygdala, accumbens, caudate, putamen, thalamus.
- Tziortzi et al. (2014): connectivity-based striatal parcellation; limbic striatum as an accumbens definition.
- Murty et al. (2014): functional–anatomical substantia nigra/VTA masks.
- TractSeg (Wasserthal et al., 2018): fornix and major white-matter tracts.
- MNI152 templates (FSL, `$FSLDIR/data/standard`): the 1 mm brain image and brain mask used for registration.

Workflow overview

The workflow comprises nine steps applied to one tract at a time (Figure S1). Step 1 is performed once per participant and reused across tracts. Steps 2 through 9 are repeated for each tract by changing the tract name and the seed, target and atlas files in the configuration.

Figure S1

Sequence of the Corridor Workflow

1. Step 1. Registration of the MNI template to native T1 space (ANTs SyN).
2. Step 2. Warping of the seed, target and atlas into diffusion space.
3. Step 3. Corridor construction (dilate, add seed and target, invert).
4. Step 4. Cutoff selection by pilot sweep and side-by-side comparison.
5. Step 5. Corridor-constrained tractography (tckgen).
6. Step 6. Bundle cleaning (Mahalanobis outlier removal).
7. Step 7. Quality control (tract-density overlays).
8. Step 8. Along-tract profiles (100 nodes; FA and NODDI).
9. Step 9. Group-level inference (whole tract, quartiles, nodes), with node-wise results viewed in the Node-wise Tract Explorer.

Note. MNI = Montreal Neurological Institute; FA = fractional anisotropy; NODDI = neurite orientation dispersion and density imaging. The Node-wise Tract Explorer is the browser-based results viewer described in the Explorer section.

Inputs

Table S6 lists the per-participant inputs. When the T1 image is already aligned to the diffusion image, as in Human Connectome Project data, no affine matrix is needed: `T1_TO_DWI` is set to `header` in the configuration and the warp script resamples by image header. With the default, `matrix`, a participant without a matrix is reported as missing an input.

Table S6

Per-Participant Inputs

Input	Origin	Path used by the scripts
Skull-stripped T1-weighted image	preprocessing	<code>\$PROJECT/anat/<subj>/<subj>_T1w_brain.nii.gz</code>
Normalized white-matter fibre orientation distribution (FOD; .mif)	MRtrix3 multi-shell multi-tissue CSD and <code>mtnormalise</code>	<code>\$PROJECT/dwi/<subj>/wm_fod_norm.mif</code>

Input	Origin	Path used by the scripts
Diffusion-space brain mask	preprocessing	\$PROJECT/dwi/<subj>/ nodif_brain_mask.nii.gz
Preprocessed diffusion data with <i>b</i> -values and gradient directions (needed only to estimate FODs or fit NODDI)	preprocessing	\$PROJECT/dwi/<subj>/ data.nii.gz, bvals, bvecs
Mean <i>b</i> = 0 image (optional; background of the quality- control images)	preprocessing	\$PROJECT/dwi/<subj>/ mean_b0.nii.gz
T1 → diffusion affine matrix (when T1_TO_DWI=matrix)	FLIRT	\$PROJECT/xfm/<subj>/ str2diff.mat
Scalar maps for profiling	DTIFIT; AMICO NODDI	\$PROJECT/dwi/<subj>/fa.nii.gz; \$PROJECT/noddi/<subj>/fit_*.nii.gz

Note. CSD = constrained spherical deconvolution.

Outputs

```

$OUT/<subj>/
  reg/          mni2t1_0GenericAffine.mat, mni2t1_1Warp.nii.gz, mni2t1_1InverseWarp.nii.gz
  rois/         <tract>_{seed,target,atlas}_{t1,diff}.nii.gz
                <tract>_atlas_dilated.nii.gz  <tract>_inclusion_zone.nii.gz
<tract>_exclusion_mask.nii.gz
  tckgen/<tract>/
                <tract>_pilot_<cutoff>.tck          (Step 4)
                <tract>_<cutoff>.tck  <tract>_<cutoff>_cleaned.tck
$OUT/qc/<tract>_cutoff_pilot/  cutoff comparison panels, cutoff_summary.csv          (Step 4b)
$OUT/qc/<tract>_tckgen_summary.csv  streamlines selected and generated          (Step 5)
$OUT/qc/<tract>/                per-participant overlay images, qc_flags.csv          (Step 7)
$OUT/nodewise/                 <tract>_tract_stats.csv          cleaned-bundle Streamline_count,
                                Mean_length_mm          (Step 6)
                                <tract>_nodewise_all_subjects.csv
                                Subject, Tract, Node, FA, NDI, ODI, FWF          (Step 8)
$OUT/analysis/                 <tract>__<metric>__analysis.csv  one row per participant (Step
8b)
                                <tract>__<metric>__tract_models.csv          (Step 9a)
$OUT/permutation/              <tract>__<metric>__<outcome>_{nodewise,clusters,summary}.csv
                                results_long.csv          Explorer input          (Steps 9b, 9c)
$OUT/logs/                     one log per shell step script

```

A manifest should be kept for each analysis recording the atlas version and threshold, dilation, interpolation, transform files, cutoff, scalar maps, covariates and software versions.

Scripts

All scripts read a single configuration file, `00_config.sh`, which specifies project paths, the participant list, the tract definition and the tractography parameters. The configuration is edited once per project; the tract fields (`TRACT`, `SEED_MNI`, `TARGET_MNI`, `ATLAS_MNI`) are changed for each tract. The shell scripts write a log to `$OUT/logs/`. Steps 0b, 1, 2, 3, 5, 6, 7 and 8a skip participants whose output already exists, so an interrupted run can be restarted with the same command; tractograms and FOD images are written under a temporary name and renamed on completion, so a file left by an interrupted run is never taken for a finished one. Setting `FORCE=1` recomputes existing outputs. Steps 4, 8, 8b and 9 recompute on every run. The Python and R scripts read the settings exported by `source 00_config.sh`; the file must be sourced again after every edit, otherwise those scripts continue to use the previous values. Steps 0b, 1 and 5 require hours on a full sample and should be run under `tmux` or a job scheduler. The concurrency settings in the configuration (`NTHREADS`, `MAXJOBS`, `ANTS_THREADS`, `ANTS_JOBS`, `FOD_JOBS`) were chosen for a 48-core node and should be reduced on a workstation or a shared machine.

```
cd scripts
nano 00_config.sh          # project paths, tract definition, parameters
source 00_config.sh        # the Python and R scripts read these settings

bash 00b_fod_estimation.sh  # only if wm_fod_norm.mif does not yet exist
bash 01_register_mni_to_t1.sh
bash 02_warp_rois.sh
bash 03_build_corridor_mask.sh
bash 04_tune_cutoff.sh "sub-01 sub-02 sub-03 sub-04 sub-05" "0.1 0.08 0.06 0.01"
python 04b_compare_cutoffs.py "sub-01 sub-02 sub-03 sub-04 sub-05" "0.1 0.08 0.06 0.01"
bash 05_tractography.sh
python 06_clean_bundles.py
python 07_visual_qc.py
python 08a_noddi_fit.py      # omit to profile FA only; Step 8 drops metrics with no maps
python 08_node_profiles.py
python 08b_build_analysis_csv.py # analysis files for Step 9

python 09a_tract_models.py --metric NDI --outcomes memory_accuracy
mkdir -p "$OUT/permutation"
Rscript 09b_nodewise_permutation.R "$OUT/analysis/${TRACT}__NDI__analysis.csv" \
    memory_accuracy NDI "$OUT/permutation" "${TRACT}__NDI__memory_accuracy"
python 09c_stack_for_explorer.py "$OUT/permutation"
```

The configuration file follows.

The script `00_config.sh` is reproduced in the Supplementary Scripts appendix.

Step 1. Registration of the MNI template to native T1 space

The atlas and region files are defined in Montreal Neurological Institute (MNI) space, whereas tractography is performed in each participant's diffusion space. Moving an MNI-space image into a participant therefore requires two transforms: a nonlinear warp from MNI to the participant's T1 image, computed in this step, and a linear transform from T1 to diffusion space, obtained during preprocessing and applied in Step 2.

A nonlinear registration is necessary because affine transforms account for global rotation, scaling and shear but not for individual differences in the size and position of subcortical structures. Positional errors of a few millimetres in the midbrain displace the ventral tegmental area (VTA) region outside the VTA.

Procedure

Registration is performed with `antsRegistrationSyNQuick.sh` (Avants et al., 2008), with the participant's skull-stripped T1 image as the fixed image and the skull-stripped MNI152 1 mm template as the moving image. Image coverage should match: registering a skull-stripped participant image to a whole-head template biases the solution.

```
antsRegistrationSyNQuick.sh -d 3 \
  -f "$PROJECT/anat/$s/${s}_T1w_brain.nii.gz" \
  -m "$FSLDIR/data/standard/MNI152_T1_1mm_brain.nii.gz" \
  -o "$OUT/$s/reg/mni2t1_" -n 4
```

The outputs are the affine component (`mni2t1_0GenericAffine.mat`), the forward warp from MNI to T1 (`mni2t1_1Warp.nii.gz`, used in Step 2), the inverse warp from T1 to MNI (`mni2t1_1InverseWarp.nii.gz`, used to return participant-level results to atlas space) and the template resampled into participant space (`mni2t1_Warped.nii.gz`).

If the T1 image has not been skull-stripped, this should be done first; atlas construction used SynthStrip (Hoopes et al., 2022) and the example dataset used ANTs brain extraction. If the T1 image is already aligned to the diffusion image, this warp is the only transform required (`T1_TO_DWI=header` in the configuration).

The full script follows. Each participant requires approximately 10 to 15 min with four threads, and the script runs four participants concurrently.

The script `01_register_mni_to_t1.sh` is reproduced in the Supplementary Scripts appendix.

Verification

The script concludes with a per-participant table of the three transform files. Registration quality is assessed by overlaying `mni2t1_Warped.nii.gz` on the participant's T1 image; the ventricles,

corpus callosum and brainstem outline should coincide. A registration that is globally acceptable but locally displaced in the midbrain will manifest in Step 2 as a misplaced VTA region, so this inspection supplements rather than replaces the region-level inspection described there. In the example dataset all 57 participants produced complete transforms.

Step 2. Warping the seed, target and atlas into diffusion space

Three Montreal Neurological Institute (MNI) space images are transformed for each tract: the seed region, the target region and the atlas's 50% binary map. Each passes through the nonlinear warp from Step 1 and then the linear T1 → diffusion transform. Nearest-neighbour interpolation is used at both stages because the images are labels, and the result is re-binarized.

Procedure

```
# MNI -> T1 (ANTs, nonlinear)
antsApplyTransforms -d 3 -i "$ATLAS_MNI" -r "$T1" -o "$d/${TRACT}_atlas_t1.nii.gz" \
  -t "$OUT/$s/reg/mni2t1_1Warp.nii.gz" -t "$OUT/$s/reg/mni2t1_0GenericAffine.mat" -n
NearestNeighbor

# T1 -> diffusion (FLIRT, linear); with T1_TO_DWI=header, -usesqform replaces -init <matrix>
flirt -in "$d/${TRACT}_atlas_t1.nii.gz" -ref "$PROJECT/dwi/$s/nodif_brain_mask.nii.gz" \
  -applyxfm -init "$PROJECT/xfm/$s/str2diff.mat" -out "$d/${TRACT}_atlas_diff.nii.gz" -interp
nearestneighbour
fslmaths "$d/${TRACT}_atlas_diff.nii.gz" -thr 0.5 -bin "$d/${TRACT}_atlas_diff.nii.gz"
```

Two successive nearest-neighbour resamplings of a binary mask have negligible effect on its extent. If the probabilistic map is required in native space, it should be warped with linear interpolation and thresholded afterwards. The full script follows; it completes in seconds per participant.

The script 02_warp_rois.sh is reproduced in the Supplementary Scripts appendix.

Verification

The script ends with a per-participant table of the voxel counts of the warped seed, target and atlas and of the seed–target overlap, which must be zero; participants with missing outputs are listed as such. Voxel counts far from the sample mean (more than two standard deviations is a useful criterion) indicate a failed registration and call for inspection of that participant. Table S7 gives the counts obtained in the example dataset. In that dataset image dimensions, binariness and laterality (left-hemisphere regions located in the left hemisphere) were also checked for every participant, and a cross-correlation below .60 between the warped template and the T1 image was flagged; two participants scored .59 and were retained after visual inspection. These additional checks are not part of the distributed script.

Table S7

Voxel Counts of Warped Regions in the Example Dataset (N = 57), VTA → Hippocampus

Region	Minimum	<i>M</i>	Maximum	<i>SD</i>
VTA (L / R)	32 / 34	44 / 44	56 / 56	5.5 / 5.2

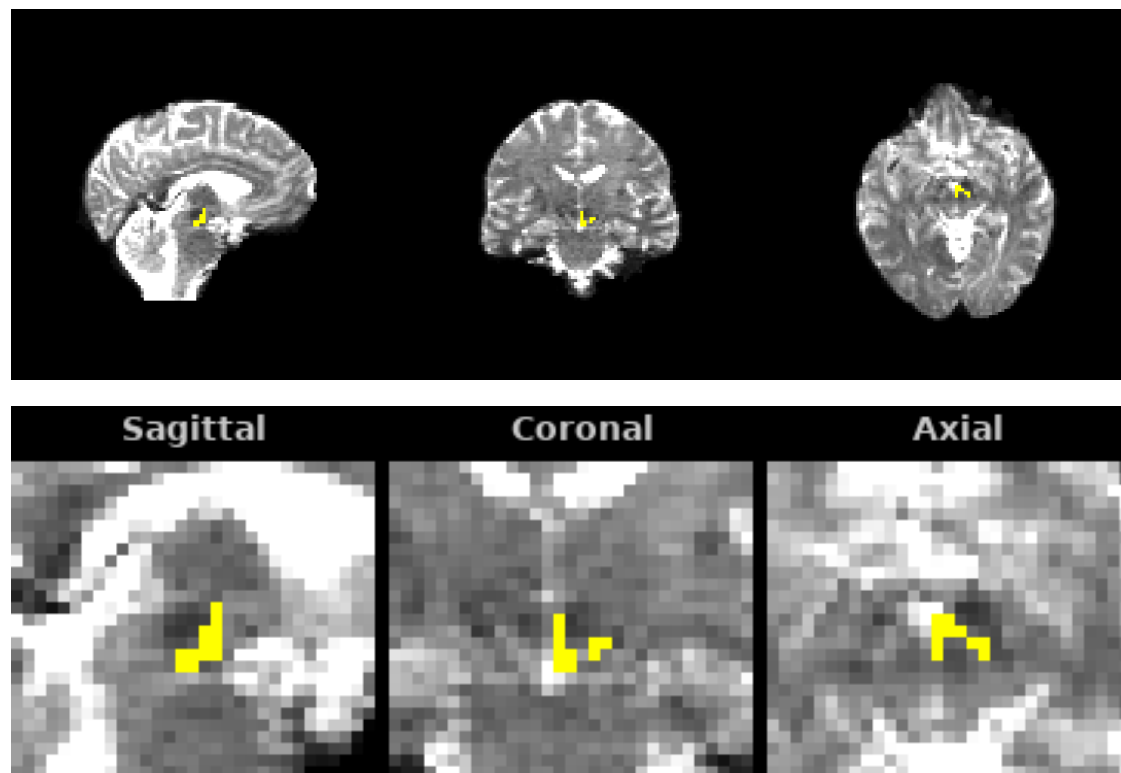
Region	Minimum	<i>M</i>	Maximum	<i>SD</i>
Hippocampus (L / R)	308 / 332	395 / 407	497 / 493	33 / 34
Atlas, 50% (L / R)	107 / 99	126 / 123	146 / 149	9.9 / 10.6

Note. VTA = ventral tegmental area; L = left; R = right.

Visual inspection is performed for every participant by overlaying each warped region on the mean $b = 0$ image. Three views are informative: a whole-brain view for gross placement, a magnified view (5 $\times$) around the region for voxel-level placement, and an orthogonal render for the record (Figures S2 to S4). The VTA should lie in the ventral midbrain anterior to the red nucleus near the midline and occupy a small number of voxels; placement in the cerebral peduncle, pons or outside the brainstem indicates a failed registration. The hippocampus should follow the medial temporal lobe along the floor of the lateral ventricle; overlap with the ventricle or placement in white matter indicates an error. The atlas should form a narrow corridor from the VTA through the midbrain to the target. A region located in ventricle, white matter or cortex, a region with zero voxels, or a left-hemisphere region on the right returns the participant to Step 1.

Figure S2

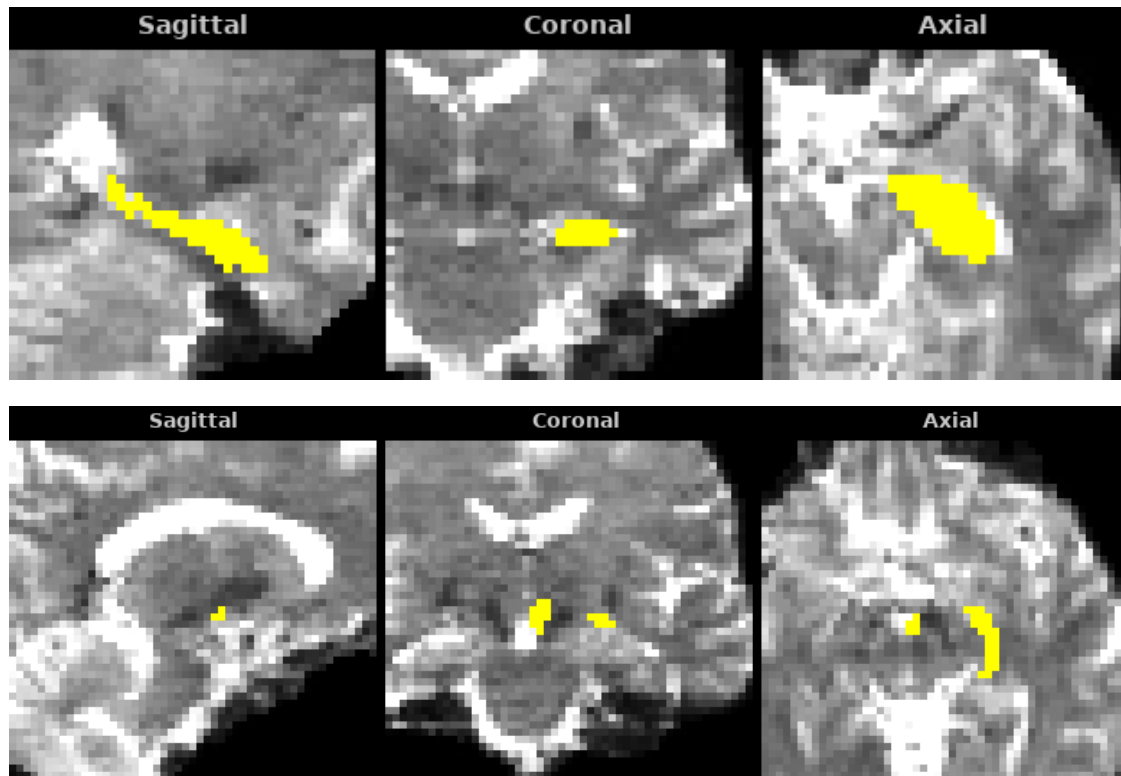
Warped Left VTA Seed Region Over the Mean $b = 0$ Image



Note. Whole-brain view (top) and magnified view (bottom), example dataset.

Figure S3

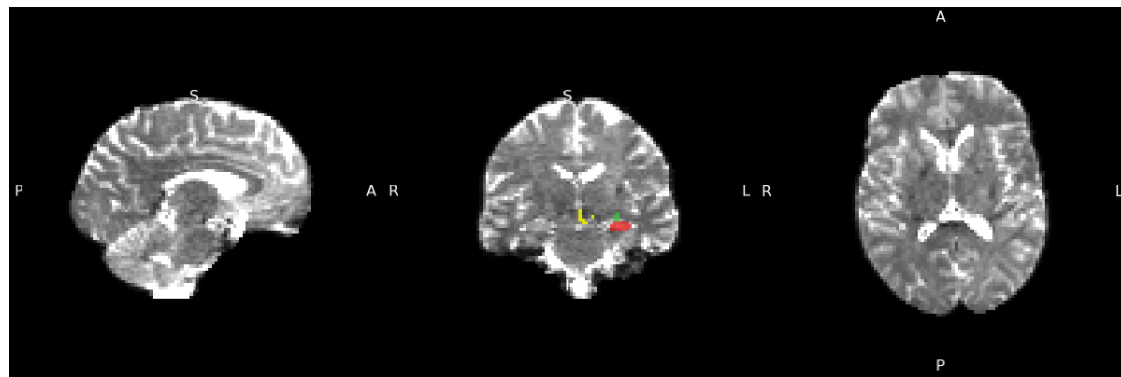
Warped Left Hippocampus Target and Left VTA → Hippocampus Atlas



Note. Hippocampus (top) and atlas at the 50% threshold (bottom), magnified views.

Figure S4

Orthogonal Render of the Warped Regions

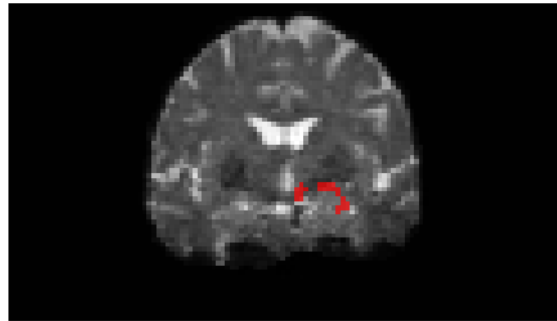
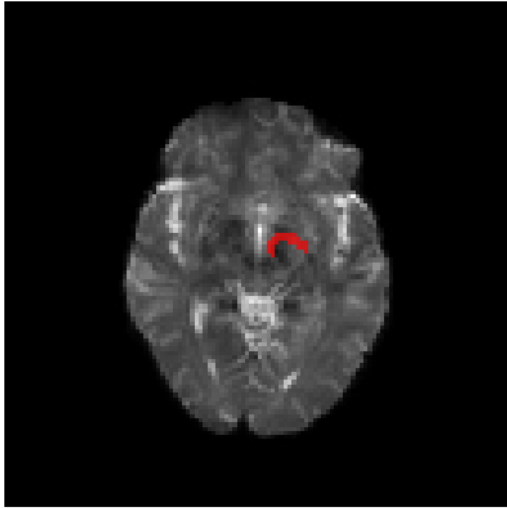


Note. VTA in yellow, hippocampus in red, atlas in green.

For the anterior hippocampus tract the same procedure is repeated with the anterior atlas file; the seed and target regions are shared (Figure S5).

Figure S5

Warped Anterior VTA → Hippocampus Atlas in Diffusion Space



Note. Atlas in red over the mean $b = 0$ image. Left: axial view. Right: coronal view.

Step 3. Corridor construction

Atlas construction used approximately a dozen individually prepared exclusion regions per tract. The corridor workflow replaces them with a single mask derived from the warped atlas. The atlas is dilated, the seed and target regions are added, the union is binarized to form an inclusion zone, and the inclusion zone is inverted. The inverted image serves as the sole exclusion mask for tractography: any streamline leaving the corridor is discarded.

Procedure

```
fslmaths "$d/${TRACT}_atlas_diff.nii.gz" -dilm -dilm "$d/${TRACT}_atlas_dilated.nii.gz"
fslmaths "$d/${TRACT}_atlas_dilated.nii.gz" -add "$d/${TRACT}_seed_diff.nii.gz" \
    -add "$d/${TRACT}_target_diff.nii.gz" -bin "$d/${TRACT}_inclusion_zone.nii.gz"
fslmaths "$d/${TRACT}_inclusion_zone.nii.gz" -binv "$d/${TRACT}_exclusion_mask.nii.gz"
```

The seed and target must be added before inversion; otherwise the corridor terminates short of them and tracking stops at the boundary. The full script follows; the `DILATE_VOX` setting in the configuration controls the number of `-dilm` passes.

The script `03_build_corridor_mask.sh` is reproduced in the Supplementary Scripts appendix.

Dilation

A dilation of two voxels was used in the example dataset at 2 mm isotropic resolution. One voxel is appropriate when registration is accurate and pilot reconstructions appear loose; four voxels serves as a sensitivity setting when registration is uncertain. The 50% atlas is already somewhat wider than the median participant's core, which is why two voxels is generally sufficient. The dilation used should be reported.

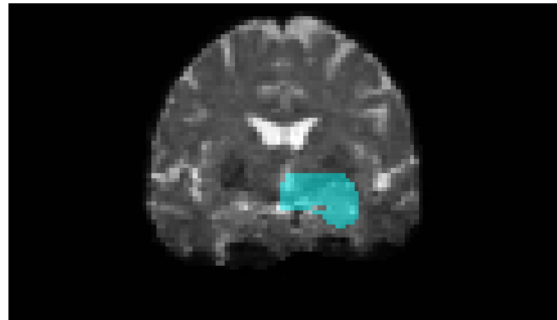
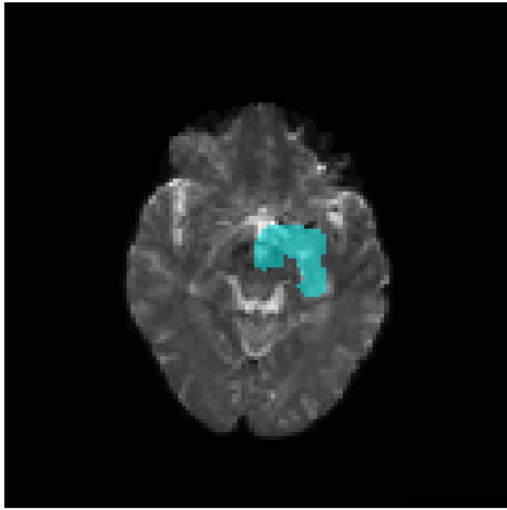
Verification

The script confirms that the seed and target voxels have value 0 in the exclusion mask (that is, are not excluded) and reports the corridor size. Additional checks confirm binariness, agreement of dimensions with the diffusion image, and an inclusion zone below 20% of brain volume. In the example dataset corridor sizes ranged from 1,526 to 1,934 voxels ($M = 1,720$) for the posterior tract and approximately 1,400 voxels for the anterior tract. An automated coverage check that flags inclusion zones below 1% of brain volume flags every participant; the corridor is that small by design.

Visual inspection confirms that the corridor follows a plausible path from seed to target and contains both regions (Figure S6). Contact with the ventricle or extension into cortex indicates excessive dilation or a registration error.

Figure S6

Inclusion Zone for the Anterior VTA $\rightarrow$ Hippocampus Tract



Note. Inclusion zone in cyan over the mean $b = 0$ image, with the seed and target contained within it. Left: axial view. Right: coronal view. VTA = ventral tegmental area.

Step 4. Selection of the FOD amplitude cutoff

The fibre orientation distribution (FOD) amplitude cutoff determines the minimum FOD amplitude at which tracking continues. A high cutoff terminates streamlines before they reach the target; a low cutoff, in unconstrained tracking, produces spurious streamlines. The corridor alters this trade-off. With tracking confined to the corridor, a permissive cutoff produces well-formed bundles while consuming far fewer seeds than a conservative cutoff. This behaviour should be verified for each dataset and tract family by a pilot sweep on a small number of participants, followed by tabulation and side-by-side inspection of the reconstructions.

Cutoff in routine use

The value in routine use with the corridor mask is 0.01, and it is the default in `00_config.sh`. It is the recommended starting value for any tract reconstructed with this workflow; the pilot sweep described below confirms it for a new dataset or tract family and takes under an hour. Table S8 lists the cutoffs that have been used with the atlas.

Table S8

FOD Amplitude Cutoffs in Use

Setting	Cutoff	Outcome
Corridor workflow, 3 T multi-shell data (example dataset): posterior and anterior ventral tegmental area (VTA) → hippocampus	0.01	All 228 runs (57 participants, two tracts, two hemispheres) reached 2,500 streamlines; no participant excluded
Corridor workflow, VTA → accumbens (other users of the atlas)	0.01	Reported to reconstruct the tract
Atlas construction, 7 T: VTA → hippocampus and VTA → accumbens	0.06	Value used to build the atlas
Atlas construction, 7 T: hippocampus → accumbens and hippocampus → ventral pallidum	0.08	Value used to build the atlas
MRtrix3 default for FOD-based tracking	0.05	For reference; not used here

Note. A cutoff of 0.01 is usable only with the corridor's exclusion mask. Without the mask it produces streamlines throughout the brain.

Because lower field strength yields noisier FOD estimates, a cutoff higher than the 7 T values (approximately 0.08) was anticipated for 3 T data. The pilot sweep did not support that expectation: the conservative cutoffs failed to reach the target in several participants, and 0.01

reached it in all of them with about one fifth of the seeds.

Pilot sweep

The sweep uses five participants, four cutoffs and reduced budgets (`-select 1000, -seeds 5000000`); all other parameters match the production run.

```
bash 04_tune_cutoff.sh "sub-01 sub-02 sub-03 sub-04 sub-05" "0.1 0.08 0.06 0.01"
```

The script reports streamlines selected, streamlines generated and mean length for each participant and cutoff. `tcngen` generates one streamline per seed, so the number generated is also the number of seeds consumed. Table S9 gives the streamline counts obtained in the example dataset. At 0.1 most runs exhausted the seed budget before reaching the target. At 0.08 some participants reached the target and one stopped at 309. At 0.06 and at 0.01 all runs reached the target; the 0.01 runs consumed approximately one fifth of the seeds (about 415,000 versus 2.1 million or more at 0.06). Reaching the target in every pilot participant is a necessary condition for a cutoff; the reconstructions must also correspond to the tract, which is assessed next.

Table S9

Streamlines Reached by Cutoff in the Pilot Sweep, Posterior VTA $\rightarrow$ Hippocampus

Participant	Hemisphere	0.1	0.08	0.06	0.01
P1	L	118	556	1000	1000
P1	R	30	434	1000	1000
P2	L	274	1000	1000	1000
P2	R	165	906	1000	1000
P3	L	1000	1000	1000	1000
P3	R	1000	1000	1000	1000
P4	L	88	309	1000	1000
P4	R	731	1000	1000	1000

Note. Target 1,000 streamlines; seed limit 5 million. P1 to P4 = pilot participants with all four cutoffs completed; the fifth pilot participant was run at 0.06 and 0.01 only and reached the target at both. L = left; R = right.

Side-by-side comparison

For each participant, the tract-density image (TDI) at each cutoff is rendered on the same axial and coronal slices, one column per cutoff. Dice overlap of each cutoff against the most

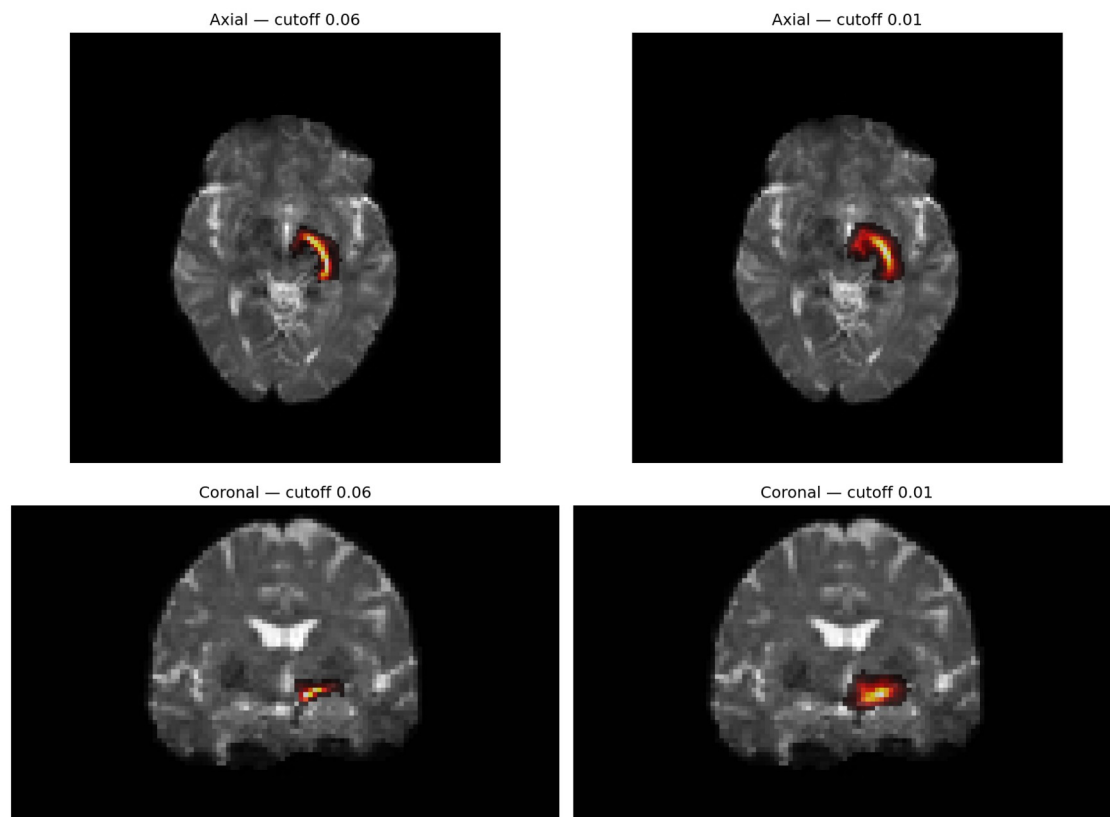
conservative one, the number of TDI voxels, and the mean and standard deviation of streamline length are computed and summarized.

```
python 04b_compare_cutoffs.py "sub-01 sub-02 sub-03 sub-04 sub-05" "0.1 0.08 0.06 0.01"
```

The script writes one panel per participant, `cutoff_summary.csv`, and a summary chart (Figures S7 and S8). In the example dataset the two cutoffs traced the same trajectory in both views. The 0.01 reconstruction was somewhat thicker (mean Dice = .66 against 0.06), mean lengths were similar (approximately 44 versus 47 mm), and the length standard deviation was slightly greater at 0.01 (7 versus 6 mm). Bundle cleaning (Step 6) removes the outlying streamlines responsible for the additional thickness; the cleaned 0.01 bundle is more compact than the uncleaned 0.06 bundle, as shown on that page.

Figure S7

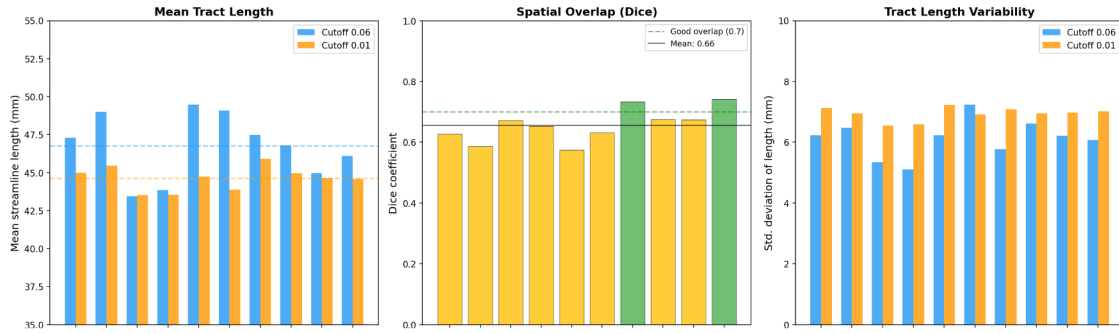
Tract-Density Images at Two Cutoffs, Uncleaned



Note. Cutoff 0.06 (left) and 0.01 (right) for one participant, left posterior VTA → hippocampus.

Figure S8

Summary Statistics by Cutoff Across the Pilot Participants



Note. Bars are grouped by pilot participant and hemisphere. Left: mean streamline length. Middle: Dice overlap between cutoffs. Right: standard deviation of streamline length.

Selection criterion

The cutoff selected is the most permissive value that reaches the streamline target in every pilot participant and whose reconstruction matches the conservative reconstruction. In the example dataset this was 0.01, the value in routine use (Table S8). A higher value is warranted only when the 0.01 reconstruction departs from the conservative reconstruction in the side-by-side images, for example by filling the corridor rather than following the tract; the next value in Table S8 (0.06) is then tried. The sweep should be repeated for each tract family, since a cutoff confirmed for VTA → hippocampus is a starting value for other pathways.

Options not used

Anatomically constrained tractography (ACT) was evaluated during atlas construction and abandoned because the tissue segmentation's white-matter mask was too restrictive relative to the gray-matter mask and tracts did not reconstruct. The `-backtrack` and `-crop_at_gmwmi` options and custom `-angle` and `-step_size` values were left at MRtrix3 defaults. These options should be introduced only in response to a specific, observed failure.

Scripts

The script `04_tune_cutoff.sh` is reproduced in the Supplementary Scripts appendix.

The script `04b_compare_cutoffs.py` is reproduced in the Supplementary Scripts appendix.

Step 5. Corridor-constrained tractography

Tractography is performed with `tckgen` using second-order integration over fibre orientation distributions (iFOD2; Tournier et al., 2010). Seeding is unidirectional from the seed region, the target region is required, and the inverted corridor from Step 3 is supplied as the exclusion mask.

Procedure

```
tckgen "$PROJECT/dwi/$s/wm_fod_norm.mif" "$tck" \
  -algorithm iFOD2 \
  -seed_image "$rois/${TRACT}_seed_diff.nii.gz" -seed_unidirectional \
  -include "$rois/${TRACT}_target_diff.nii.gz" \
  -exclude "$rois/${TRACT}_exclusion_mask.nii.gz" \
  -select 2500 -seeds 25000000 -cutoff 0.01 \
  -minlength 35 -maxlength 65 -stop \
  -nthreads 8 -force
```

Table S10 gives the rationale for each option, and Table S11 the values used per tract family. The script records the number of streamlines selected and the number generated (equal to the number of seeds consumed) per participant in `$OUT/qc/<tract>_tckgen_summary.csv`. The streamline count and mean length that enter the group-level models as covariates are taken from the cleaned bundle in Step 6, because every uncleaned tractogram that reaches the target contains the same number of streamlines. The full script follows. At 3 T with eight threads each participant and hemisphere requires approximately 2 min; 57 participants with both hemispheres required about 4 hr run serially.

Table S10

Tractography Parameters

Option	Value	Rationale
-cutoff	from Step 4 (0.01 in the example dataset)	The corridor permits a low cutoff.
-select	2500	Sufficient for stable profiles and endpoint maps. Streamline count is not a measure of connectivity; participants who stop short are retained.
-seeds	25,000,000	Ceiling on seeding attempts. The default (1,000 × select) is sometimes insufficient to reach 2,500.
-minlength, -maxlength	35, 65 mm	Set for VTA → hippocampus at 7 T. The example dataset retained them at 3 T, with mean

Option	Value	Rationale
		lengths of 44 to 47 mm. Set per tract family from pilot lengths.
-seed_unidirectional	on	Tracking proceeds away from the seed only.
-stop	on	Tracking terminates on entering the target.
gradient table	embedded in the .mif	-fslgrad is omitted when the FOD image was built from a .mif containing gradients.

Note. VTA = ventral tegmental area; FOD = fibre orientation distribution.

Table S11

Parameters by Tract Family

Family	Cutoff with the corridor	Cutoff at 7 T (atlas construction)	Length bounds	Notes
VTA → hippocampus, posterior and anterior	0.01	0.06	35–65 mm	Used for all 57 participants of the example dataset at 3 T.
VTA → amygdala	0.01, confirm by pilot	0.06	determine by pilot	Shorter tract; lower bounds expected.
VTA → accumbens, superior and inferior	0.01	0.06	determine by pilot	0.01 has been used with the corridor by other users of the atlas.
Ventral pallidum → VTA	0.01, confirm by pilot	0.06	determine by pilot	
Hippocampus → accumbens; hippocampus → ventral pallidum	0.01, confirm by pilot	0.08	determine by pilot	Frequently reconstructs as two clusters; see Step 6.

Note. Where a pilot is indicated, the length bounds are taken from the mean lengths reported in the Step 4 summary.

The script 05_tractography.sh is reproduced in the Supplementary Scripts appendix.

Verification

The script reports the number of participants reaching the streamline target. In the example dataset all 114 hemisphere runs reached 2,500 streamlines, consuming 0.5 to 2.0 million seeds (about 4% of the ceiling). The anterior tract also reached 2,500 in all 114 runs, with higher seed consumption (1.5 to 11 million) consistent with its smaller corridor. A participant reaching a lower count, for example 1,800, is retained; a participant reaching a very low count, for example 50, is reviewed in Step 7.

Step 6. Bundle cleaning

Some streamlines within the corridor follow atypical trajectories. Cleaning is performed with `clean_bundle` from pyAFQ (Kruyer et al., 2021), which implements the procedure of Yeatman et al. (2012): each streamline is resampled to 100 points, its Mahalanobis distance from the bundle's mean trajectory is computed, and streamlines exceeding a distance threshold or a length threshold are removed iteratively (Table S12).

Procedure

Table S12

Cleaning Parameters

Parameter	Value	Meaning
<code>n_points</code>	100	Resampling prior to comparison.
<code>clean_rounds</code>	5	Maximum iterations; stops earlier when no streamlines are removed.
<code>distance_threshold</code>	3	Streamlines more than 3 <i>SD</i> (Mahalanobis) from the bundle centroid are removed.
<code>length_threshold</code>	2	Streamlines longer than the mean length by more than 2 <i>SD</i> are removed.
<code>stat</code>	mean	Centroid statistic.
<code>core_only</code>	0	Distance is computed over the full length of each streamline. pyAFQ 3 and later default to the middle 60%; the script sets 0 so that results do not depend on the installed version.

```
from AFQ.recognition.cleaning import clean_bundle
cleaned, keep = clean_bundle(sft, n_points=100, clean_rounds=5,
                             distance_threshold=3, length_threshold=2,
                             stat="mean", core_only=0, return_idx=True)
```

The `core_only` argument exists in pyAFQ 3 and later; the script omits it under earlier versions, whose behaviour it reproduces. In pyAFQ 1.3.2 and earlier the function is imported from `AFQ.segmentation`; the script tries both locations.

The full script follows. It requires pyAFQ and DIPY; on older systems `pip` may require the `zipp` package to be upgraded first.

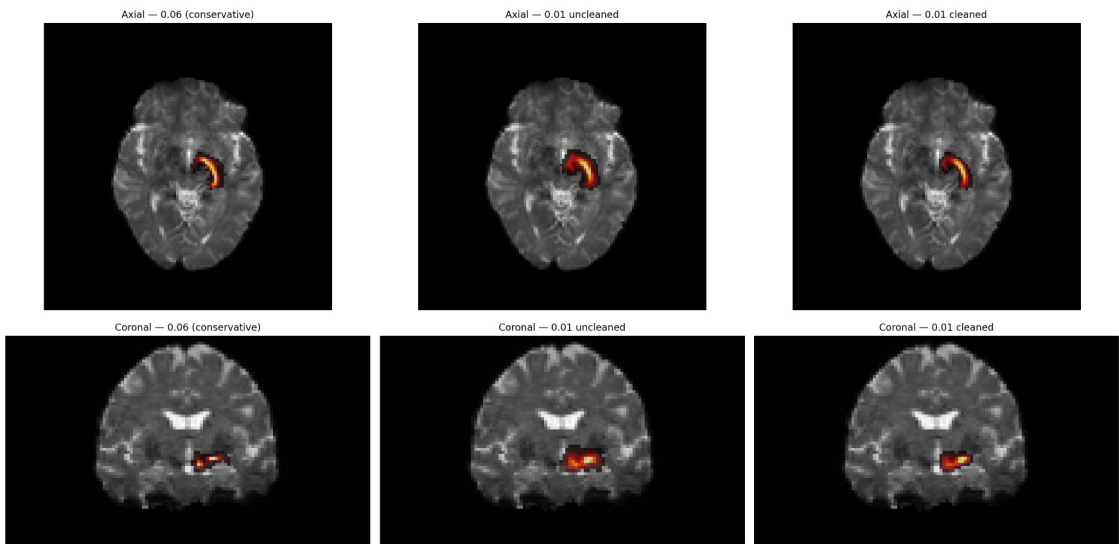
The script `06_clean_bundles.py` is reproduced in the Supplementary Scripts appendix.

Retention

In the example dataset, cleaning retained 26% to 58% of streamlines ($M = 39\%$), with a minimum cleaned count of 659. These values are typical for the thresholds in Table S12, and every participant retained several hundred streamlines. Figures S9 and S10 present the comparison referred to in Step 4: the cleaned 0.01 bundle is more compact than the uncleaned 0.06 bundle, with length standard deviations of approximately 3.5 to 4.5 mm for cleaned bundles against 5 to 7 mm for either uncleaned condition.

Figure S9

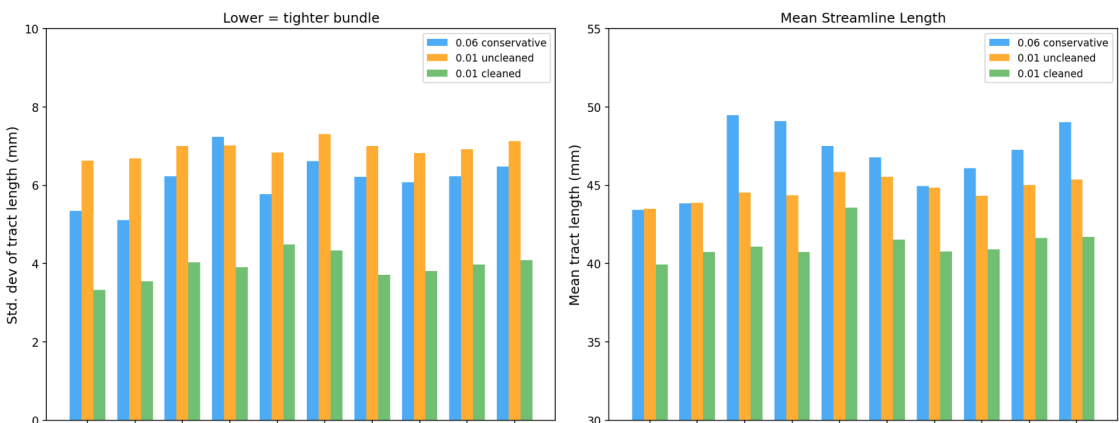
Effect of Cutoff and Cleaning on One Bundle



Note. Left: cutoff 0.06, uncleaned (1,000 streamlines). Middle: cutoff 0.01, uncleaned (2,500). Right: cutoff 0.01, cleaned (824). One participant, left posterior VTA → hippocampus. VTA = ventral tegmental area.

Figure S10

Standard Deviation of Streamline Length by Condition Across the Five Pilot Participants



Note. Bars are grouped by pilot participant and hemisphere. Cleaned 0.01 in green.

Tracts with two bundles

Some tracts reconstruct as two distinct bundles rather than a single bundle with outliers. VTA → hippocampus has a ventral secondary bundle in a subset of participants, and hippocampus → accumbens frequently produces two bundles. Mahalanobis cleaning applied to such a mixture retains the wrong bundle or a blend. The procedure used during atlas construction clusters the streamlines with QuickBundles (Garyfallidis et al., 2012; threshold 5 mm, streamlines resampled to 100 points), cleans the two largest clusters separately, and retains the anatomically correct one after inspection. Setting `QB_SPLIT = True` in the script produces `_cluster1_cleaned.tck` and `_cluster2_cleaned.tck` per participant. The correct cluster is copied to `<tract>_<cutoff>_cleaned.tck` and the script is run again to rebuild the covariate table; the selection should be recorded per participant in the manifest. In the example dataset the corridor and cleaning were sufficient for VTA → hippocampus and the division was not required.

Verification

The cleaned file must exist and contain more than zero streamlines for every participant. Counts before and after cleaning and the retention percentage are reported; retention below 15% or above 80% warrants inspection. The script also writes `$OUT/nodewise/<tract>_tract_stats.csv`, which holds the streamline count and mean length of each cleaned bundle (`Streamline_count`, `Mean_length_mm`). Both index reconstruction quality, vary across participants, and are entered as covariates in Step 9.

Step 7. Quality control of cleaned bundles

Every participant and hemisphere is inspected after cleaning. The script converts each cleaned bundle to a tract-density image (TDI) with `tckmap`, selects the axial and coronal slices of greatest density, overlays the image on the mean $b = 0$ image, and writes one image per participant together with automatic flags (Table S13). Flags appear in the image title and are collected in `qc_flags.csv`.

Table S13

Automatic Flags

Flag	Criterion
LOW_COVERAGE	fewer than 50 TDI voxels
HIGH_COVERAGE	more than 5,000 TDI voxels
LOW_DENSITY	maximum density below 5 streamlines per voxel

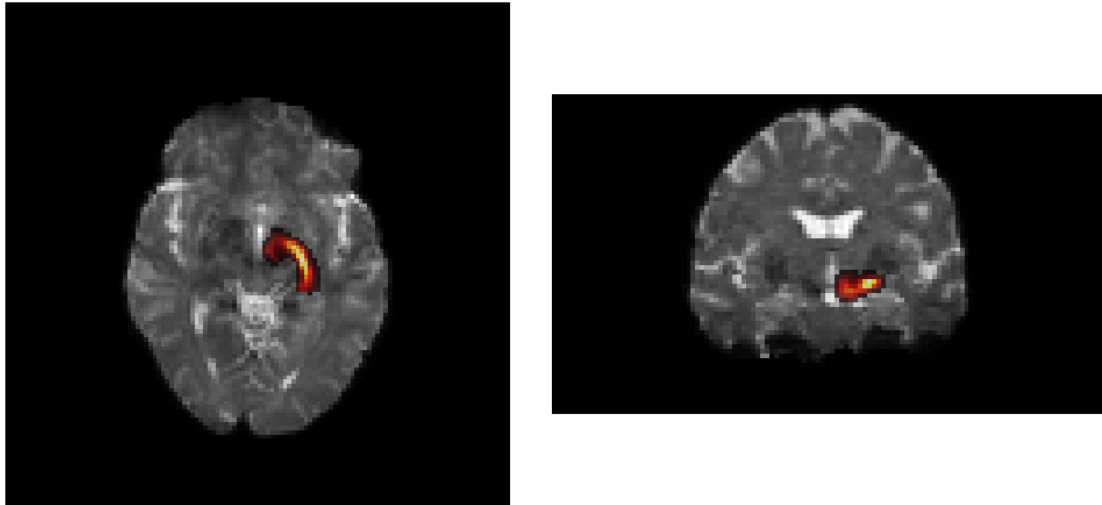
The script `07_visual_qc.py` is reproduced in the Supplementary Scripts appendix.

Criteria

A correct reconstruction shows the expected shape for the tract (for ventral tegmental area [VTA] → hippocampus, an arc from the ventral midbrain laterally into the medial temporal lobe), consistent shape across participants with variation in size, and no isolated voxel clusters, no density in the contralateral hemisphere or frontal lobe, and no empty tracts (Figures S11 to S13).

Figure S11

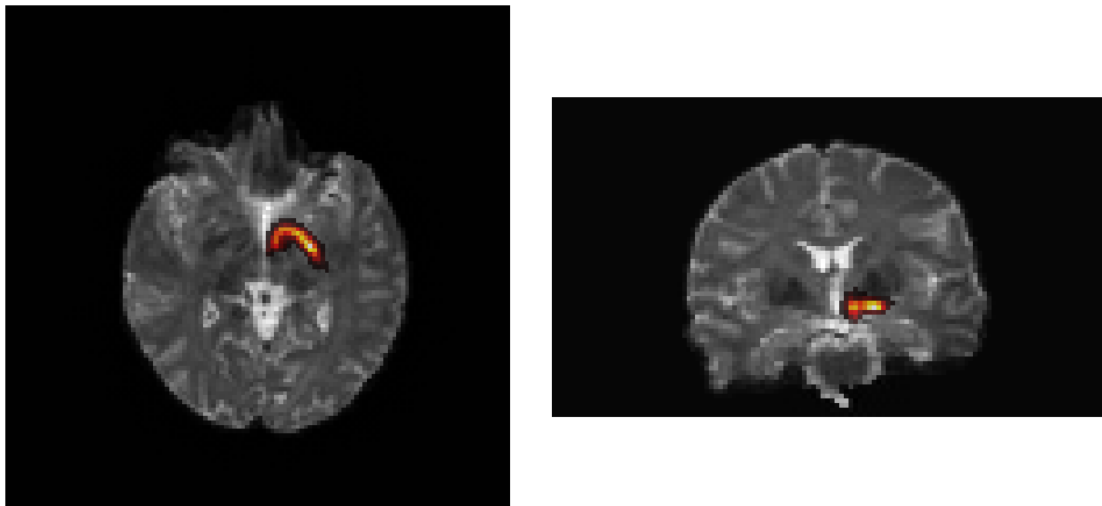
Cleaned Left Posterior VTA → Hippocampus Tract, One Participant



Note. Streamline density over the mean $b = 0$ image. Left: axial view. Right: coronal view. The same layout applies to Figures S12 and S13.

Figure S12

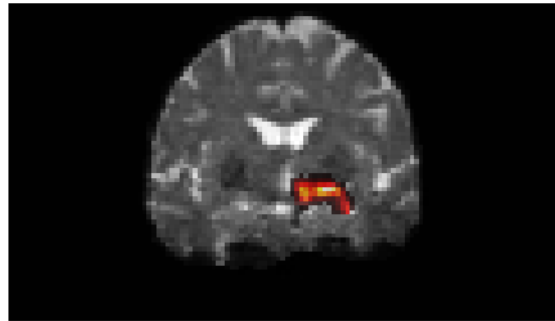
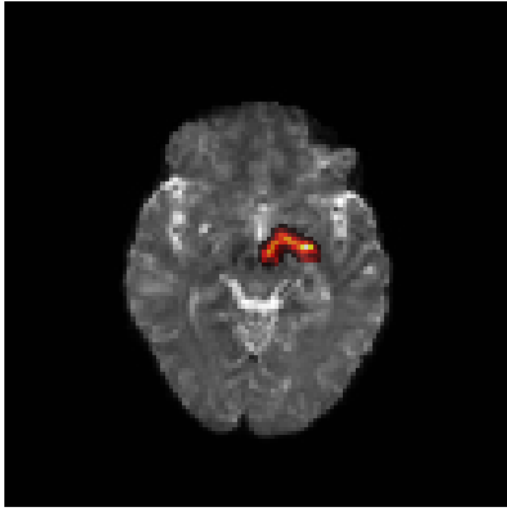
Participant With the Lowest Cleaning Retention



Note. Retention 26%; 659 streamlines.

Figure S13

Cleaned Anterior VTA $\rightarrow$ Hippocampus Tract



Note. Same participant as Figure S11.

Interactive review

The overlay images support a batch review. Flagged participants, and a sample of unflagged participants, should additionally be examined in a viewer with slice navigation, magnification and opacity control.

```
fsleyes "$PROJECT/dwi/$s/mean_b0.nii.gz" \
"$OUT/$s/tckgen/$TRACT/${TRACT}_${CUTOFF}_cleaned.tck" -cm hot -a 70 &

mrview "$PROJECT/dwi/$s/mean_b0.nii.gz" \
-tractography.load "$OUT/$s/tckgen/$TRACT/${TRACT}_${CUTOFF}_cleaned.tck" &
```

Comparison with the atlas

For a quantitative check, each participant's cleaned TDI can be warped to Montreal Neurological Institute (MNI) space with the inverse warp from Step 1 and compared with the atlas by Dice overlap against the 50% map, or by the fraction of the participant tract lying within the 25% probability map. This overlap should not be reported as validation of the atlas, because the reconstruction was constrained by it.

```
tckmap "$tck" "$tdi" -template "$PROJECT/dwi/$s/nodif_brain_mask.nii.gz" -force
fslmaths "$tdi" -thr 1 -bin "${tdi%.nii.gz}_bin.nii.gz"
```

Example dataset

All 114 posterior and 114 anterior tracts passed without flags. The two participants with borderline registration scores in Step 2 passed.

Step 8. Along-tract profiles

Each cleaned bundle is converted to a 100-point profile of a scalar map: fractional anisotropy (FA) from the tensor fit; the neurite density index (NDI), orientation dispersion index (ODI) and free-water fraction (FWF) from neurite orientation dispersion and density imaging (NODDI; Zhang et al., 2012); or any other map on the diffusion grid. Profiling follows the tract-profile approach of Yeatman et al. (2012) as implemented in DIPY (Garyfallidis et al., 2014).

Two operations make profiles comparable across participants. First, because streamlines in a tractogram file may run in either direction, each bundle is oriented against its QuickBundles centroid (Garyfallidis et al., 2012; `threshold=np.inf`, yielding one cluster), and the centroid is itself oriented so that it begins at the end nearer the warped seed region. Node 0 therefore corresponds to the seed end and node 99 to the target end in every participant. Second, `afq_profile` with `gaussian_weights` weights each streamline's contribution at a node by its Mahalanobis distance from the bundle core, reducing the influence of outlying streamlines.

Procedure

```
metric = AveragePointwiseEuclideanMetric(ResampleFeature(nb_points=100))
centroid = QuickBundles(threshold=np.inf, metric=metric).cluster(streamlines).centroids[0]
if distance(centroid[0], seed_centre) > distance(centroid[-1], seed_centre):
    centroid = centroid[::-1] # node 0 at the seed end
oriented = orient_by_streamline(streamlines, centroid)
weights = gaussian_weights(oriented, n_points=100)
profile = afq_profile(data, oriented, affine, n_points=100, weights=weights)
```

The `METRICS` dictionary in the script specifies the scalar maps; the script writes one long-format CSV per tract with one column per metric. Processing 57 participants and four tracts required 20 to 40 min.

The script `08_node_profiles.py` is reproduced in the Supplementary Scripts appendix.

Node range

Nodes near the ends of the profile (0 to 4 and 95 to 99) lie in or adjacent to gray matter and are affected by partial-volume contamination from the seed and target regions. Deep white matter spans approximately nodes 25 to 75. Whether end nodes are trimmed should be decided before modelling.

NODDI inputs

NODDI is fitted with AMICO (Daducci et al., 2015) on the eddy-corrected data; the script `08a_noddi_fit.py`, shown below, performs the fit. The modulated NDI and ODI maps (`fit_NDI_modulated.nii.gz`, `fit_ODI_modulated.nii.gz`), which incorporate the tissue-

weighted partial-volume correction of Parker et al. (2021), are used for profiling; FWF has no modulated form and `fit_FWF.nii.gz` is used. Settings used in the example dataset were `bStep=200` during scheme conversion (rounding jittered *b*-values), `b0_thr=100`, `doSaveModulatedMaps=True`, `doComputeRMSE=True` and `BLAS_nthreads=1`. By default AMICO writes its response kernels to a directory shared by all participants, and concurrent jobs then delete one another's files; the script instead generates the kernels in a working directory for each participant, so several instances can be run at once on different participants (`python 08a_noddi_fit.py sub-01 sub-02`). AMICO also empties its output directory when saving, so nothing else should be stored there.

When NODDI is sampled within a gray-matter region (for example the hippocampus), the model should be refitted with the gray-matter intrinsic parallel diffusivity (approximately 1.1×10^{-3} mm²/s rather than the white-matter default of 1.7×10^{-3} mm²/s) by setting `NODDI_DPAR=1.1e-3`; the script then writes to `$PROJECT/noddi_gm` so that the white-matter maps are retained. The white-matter fit is not appropriate in gray matter and the two fits can yield different results.

The script `08a_noddi_fit.py` is reproduced in the Supplementary Scripts appendix.

Analysis file

Inference in Step 9 reads a wide file with one row per participant: the covariates and outcomes, the streamline count and mean length of the cleaned bundle from Step 6, and the 100 node values as columns `<METRIC>_0` to `<METRIC>_99`. The script `08b_build_analysis_csv.py`, shown below, builds one such file per tract and metric from the long profile CSV, the tract statistics file and a participant-level covariates file (`COVARIATES_CSV` in the configuration; columns `Subject` plus covariates and outcomes).

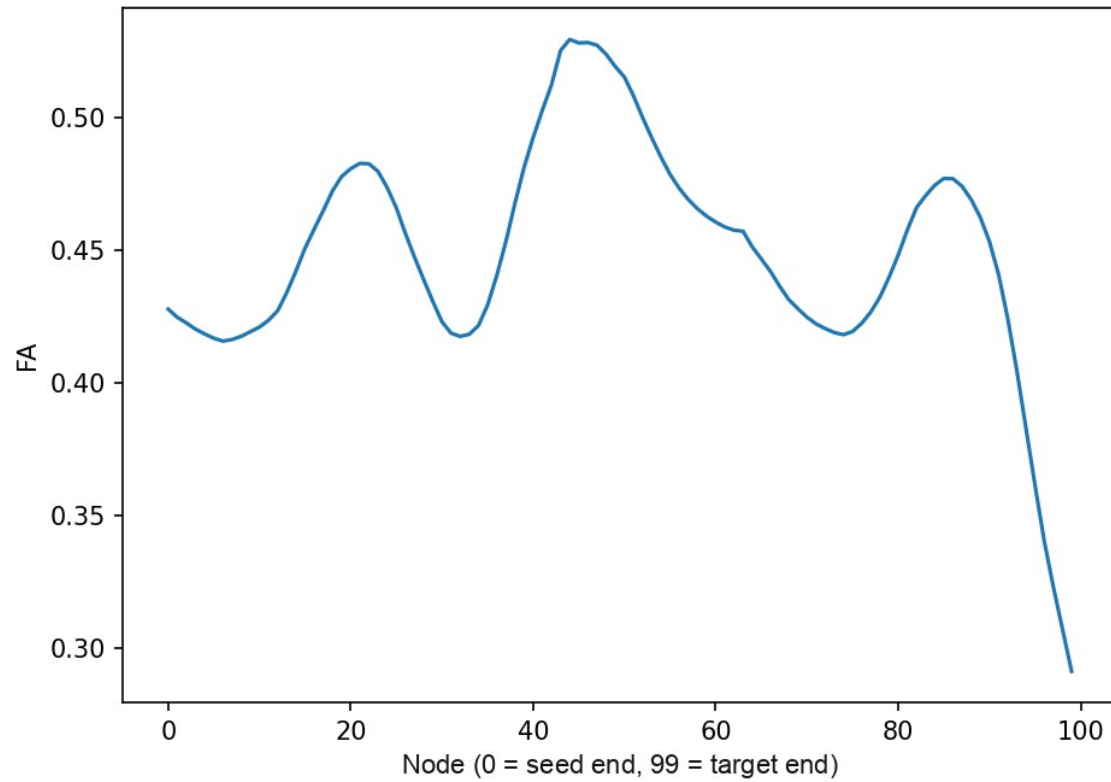
The script `08b_build_analysis_csv.py` is reproduced in the Supplementary Scripts appendix.

Example profiles

Figures S14 to S16 show profiles from one participant in the example dataset.

Figure S14

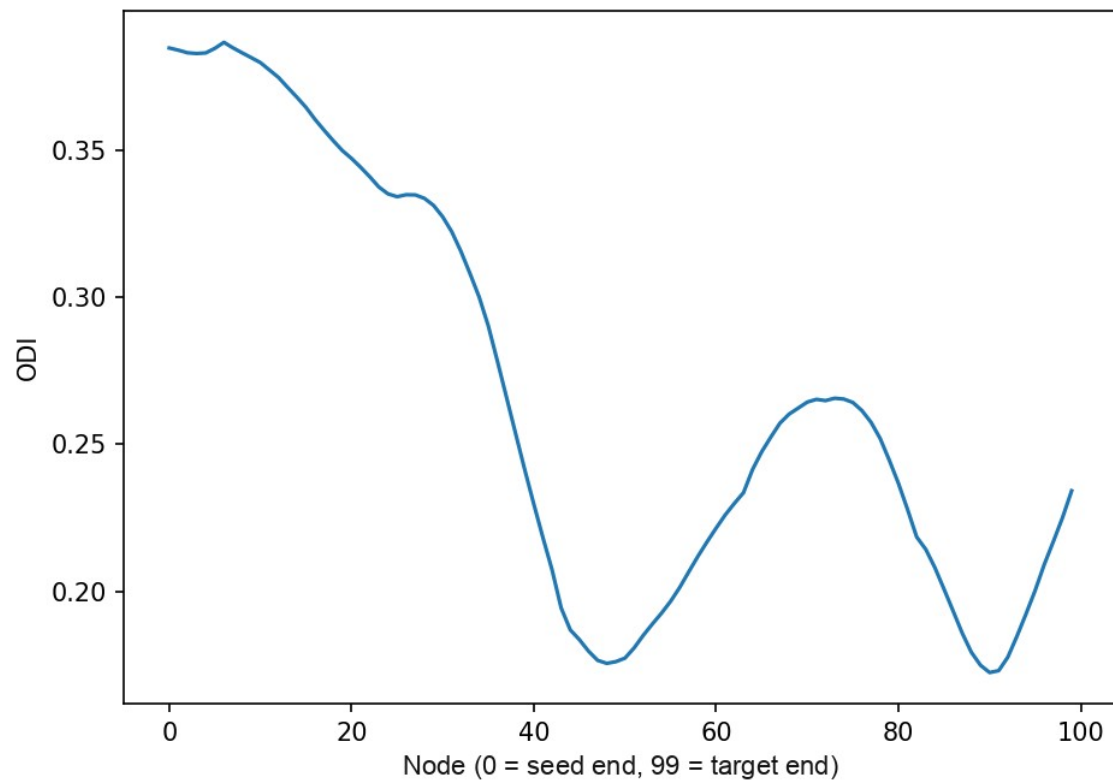
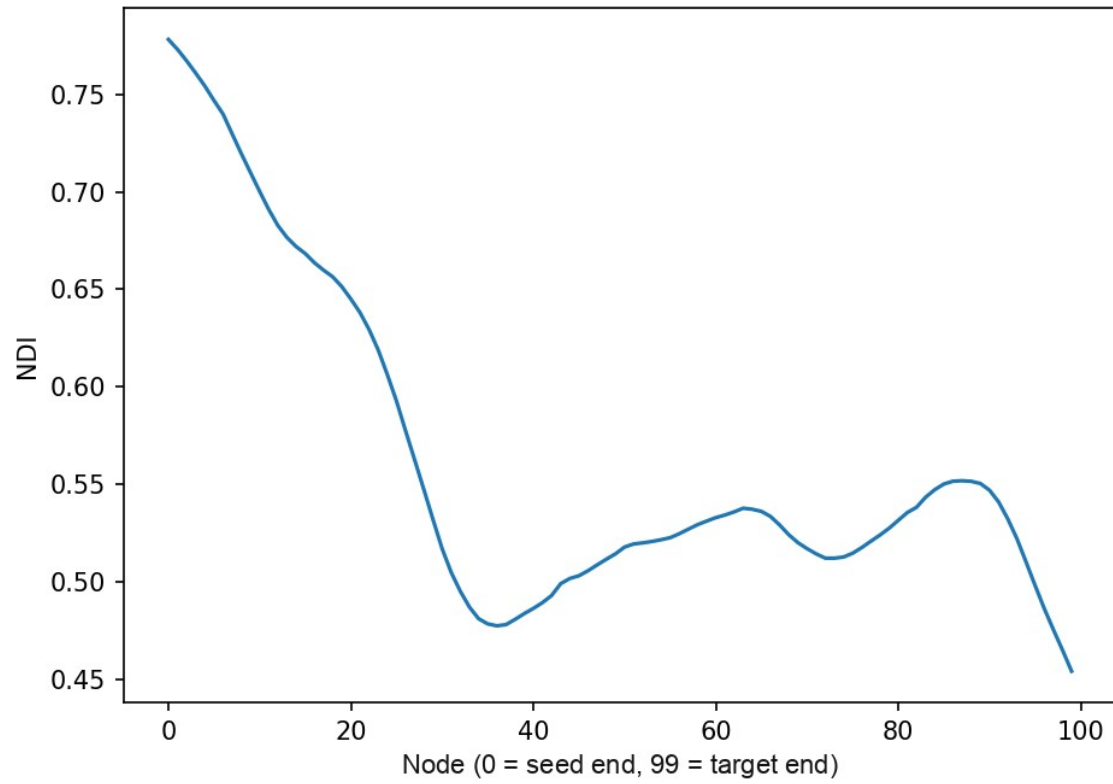
Fractional Anisotropy Along the Left Posterior VTA → Hippocampus Tract

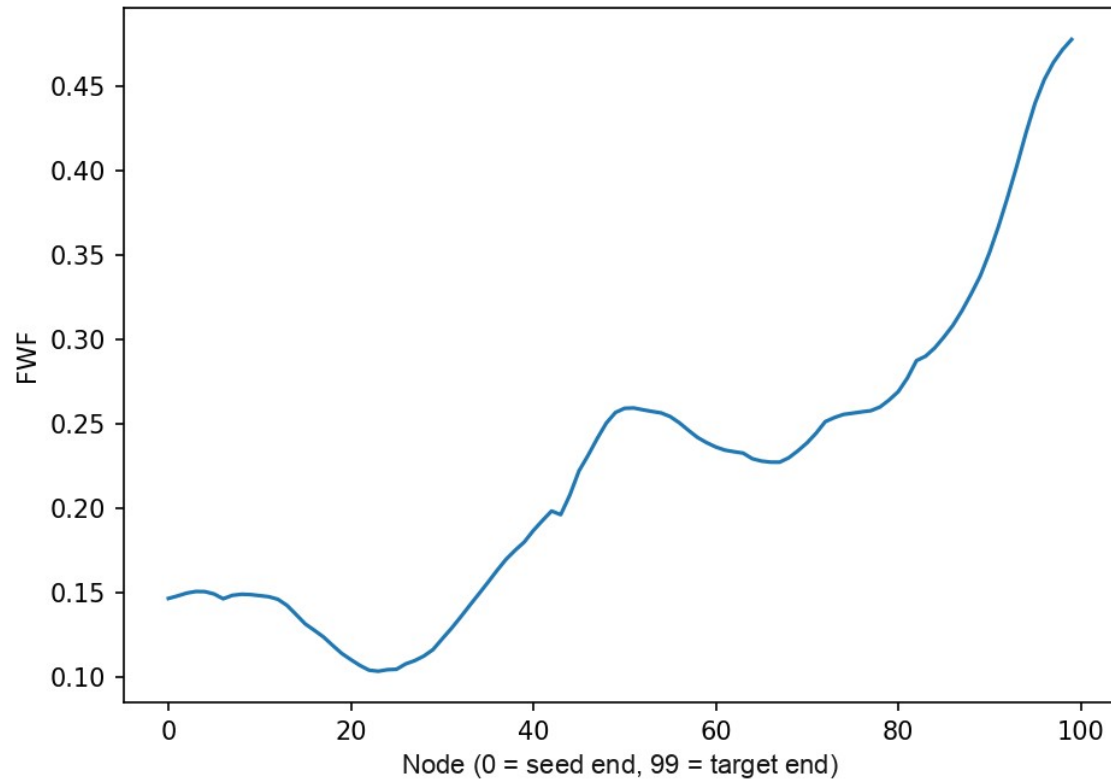


Note. FA peaks in deep white matter and declines toward the hippocampal end. VTA = ventral tegmental area.

Figure S15

NODDI Indices Along the Same Bundle

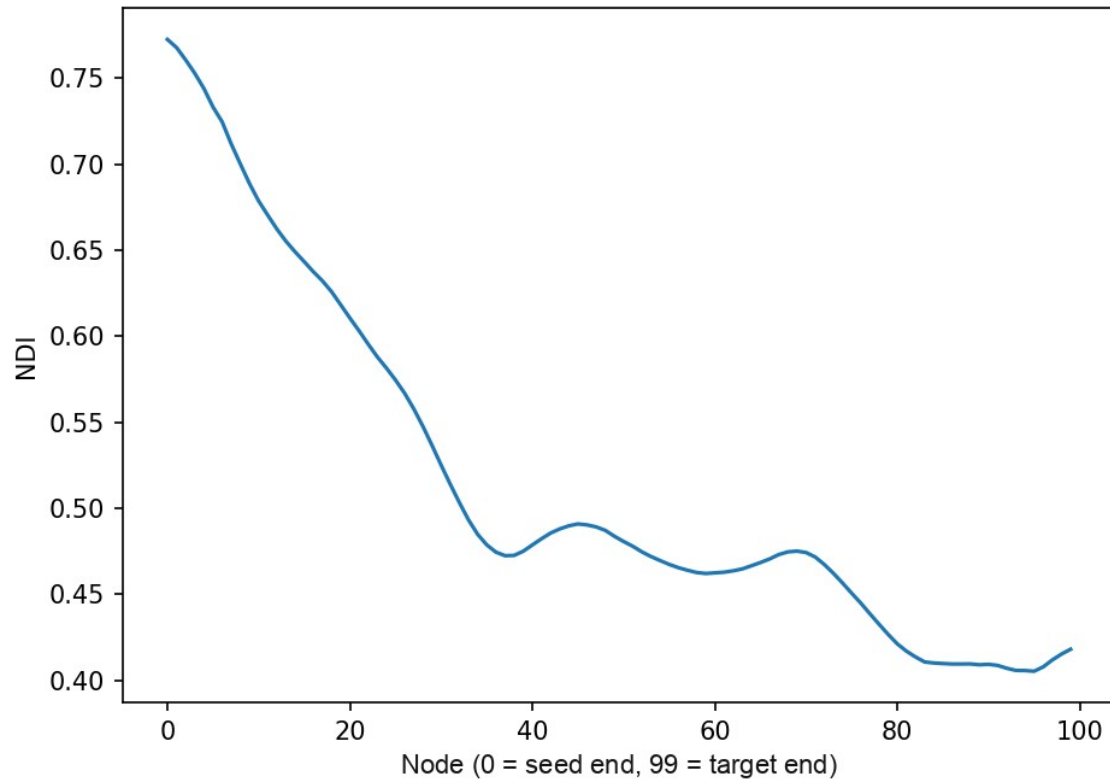




Note. NDI (top), ODI (middle) and FWF (bottom). NDI is high near the VTA and increases again toward the hippocampus; ODI is higher at both ends where fibres fan; FWF rises sharply at the hippocampal end adjacent to cerebrospinal fluid.

Figure S16

NDI Along the Anterior Tract for the Same Participant



Note. The anterior tract shares the early trajectory and diverges late.

Verification

The row count of each CSV should equal the number of participants $\times$ 100. In the example dataset every tract and metric produced 5,700 rows with no skipped participants.

When both hemispheres are processed, alignment should be confirmed before averaging. The across-participant correlation between node i on the left and node i on the right is strongly positive when the profiles are aligned (approximately $r = .98$ in the example dataset) and strongly negative when one side is reversed. Mid-tract averages (nodes 25 to 74) correlated across hemispheres at approximately $r = .80$ to $.90$ for NDI and ODI and $r = .50$ for FA.

Step 9. Group-level inference

Along-tract data can be analysed at three spatial resolutions: the whole tract, quartiles of the tract, and individual nodes (Table S14). The three address different questions, and no single resolution is appropriate for every study. The resolution, or combination of resolutions, should be specified before results are examined, and the same covariates should be used throughout. Where the neurite density index (NDI) is mentioned below, it refers to the neurite orientation dispersion and density imaging metric profiled in Step 8; VTA denotes the ventral tegmental area.

Table S14

Three Resolutions for Along-Tract Inference

Resolution	Unit of analysis	Question	Correction	Script
Whole tract	Metric averaged across nodes; one value per participant per tract	Is the metric associated with the outcome; does the association differ between subregional tracts	None within tract; across outcomes as pre-specified	09a_tract_models.py
Quartiles	Metric averaged within nodes 0–24, 25–49, 50–74, 75–99	Does the association vary along the tract; is it confined to a segment	False discovery rate (FDR) or maximum-statistic permutation across the four quartiles	09a_tract_models.py
Nodes	Metric at each of 100 nodes	Where along the tract is the association located	Cluster-extent family-wise error (FWE) by permutation	09b_nodewise_permutation. R

Whole-tract model

The metric is averaged across all 100 nodes, or across the retained range if end nodes are trimmed (`--trim`), and the outcome is regressed on the averaged metric and the covariates. All

variables are standardized, so coefficients are standardized betas. When two subregional tracts share most of their course (posterior and anterior VTA → hippocampus), both are entered in one mixed model with a subregion term, two rows per participant, a random intercept for participant, and each row carrying its own streamline length and count. The outcome × subregion interaction is tested by likelihood ratio; when it does not improve fit, the main effect is reported. In the mixed model the metric is the dependent variable, because each participant contributes one metric value per tract and a single outcome value. This resolution has the greatest power when the effect is distributed along the tract, and it does not localize the effect.

Quartile model

The same model is fitted with the metric averaged within each quartile. Because adjacent quartiles are correlated, correction across the four should either use the FDR or a maximum-statistic permutation that preserves their dependence. Uniformity along the tract is tested directly by contrasting the seed-end quartile against the target-end quartile with the mean and the difference entered together: when the mean carries the effect and the difference does not differ from zero, the effect is diffuse rather than localized.

Both resolutions are fitted by one script, which reads the analysis files from Step 8b and the covariate list (COVARIATES) from the configuration. With one tract it reports the whole-tract and quartile regressions, the FDR-corrected quartile *p* values and the uniformity contrast; with two tracts it adds the mixed model and the likelihood-ratio test of the interaction for every segment.

```
source 00_config.sh
python 09a_tract_models.py --metric NDI --outcomes memory_accuracy,memory_bias
python 09a_tract_models.py --metric NDI --outcomes memory_accuracy \
    --tracts 1_vta_1_hipp,anterior_1_vta_1_hipp
```

Results are printed and written to \$OUT/analysis/<tract>__<METRIC>__tract_models.csv, one row per model term, with the columns model (ols, uniformity or mixed), tract, segment, outcome, term, n, beta, statistic, p, p_fdr and interaction_p.

Node-wise model

At each node the outcome is regressed on the node's metric and the covariates, and the fit is compared with a reduced model omitting the metric.

```
full      ~ metric_node + ICV + tract_length + streamline_count + motion + age
reduced ~      ICV + tract_length + streamline_count + motion + age
```

Each node yields a *t* statistic and a parametric *p* value, and adjacent nodes with *p* < .05 form clusters. Family-wise error is controlled at the cluster level with a Freedman–Lane permutation procedure (Freedman & Lane, 1983; Winkler et al., 2014): the residuals of the reduced model are permuted, all 100 nodes are refitted, the largest cluster of each permutation is recorded, and the procedure is repeated 5,000 times. A cluster's *p* value is the proportion of permutations whose

largest cluster is at least as long, and a cluster is retained when that value does not exceed .05; the smallest extent that meets this criterion is reported as the extent threshold. The permutations are drawn once from a fixed seed, so results do not depend on the number of cores. Variables are not standardized in this script; the node-wise estimates are in the units of the outcome per unit of the metric.

The script reads the wide analysis file produced in Step 8b and is called once per outcome, tract and metric: `Rscript 09b_nodewise_permutation.R <analysis.csv> <outcome>`
`<METRIC>_ <out_dir> <label>`, with the label written as `<tract>__<metric>__<outcome>`. Covariate column names and the number of permutations are taken from the `COVARIATES` and `N_PERMUTATIONS` settings in the configuration. Outputs per analysis are `_nodewise.csv` (node, estimate, t , p), `_clusters.csv` and `_summary.csv`; each call requires a few minutes on one core.

```
source 00_config.sh
mkdir -p "$OUT/permutation"
for outcome in memory_accuracy memory_bias; do
  Rscript 09b_nodewise_permutation.R "$OUT/analysis/${TRACT}__NDI__analysis.csv" \
    "$outcome" NDI_ "$OUT/permutation" "${TRACT}__NDI__$outcome"
done
```

This resolution localizes an effect along the tract. Its limitation is that cluster-extent correction detects contiguous runs of supra-threshold nodes; when an effect is uniform along the tract, per-node t statistics remain near threshold at every node and the outcome becomes sensitive to small changes in model specification. In that situation node-wise results are descriptive and the whole-tract or quartile model carries the inference.

Covariates

Covariates comprise intracranial volume (ICV), the mean streamline length and streamline count of the cleaned bundle (written by Step 6), absolute head motion and age. Length and count are tract-specific and index reconstruction quality. Streamline count can be strongly correlated with the metric under test ($r = .63$ to $.87$ with tract-mean NDI across the tracts of the example dataset), which reduces the residual variance available to the test. Its inclusion should be decided in advance and both specifications reported.

Choosing a resolution

A hypothesis about a particular segment of a pathway calls for the quartile or node-wise model. A hypothesis about the pathway as a whole calls for the whole-tract model. Exploratory work is often best served by the whole-tract model for inference with the node-wise profile reported as description. Tests across several metrics, tracts and outcomes accumulate rapidly at any resolution; fixing the analytic hierarchy in advance (primary metric, primary outcome family) is more readily justified than correcting across all tests afterwards.

In the example dataset, three social-memory measures were associated with NDI in the whole-

tract model with no subregion interaction. In the quartile model one of them (positivity bias in false memories) was significant in all four quartiles and the seed-end versus target-end contrast was null for all three, indicating uniform effects. Node-wise clusters were located at nodes 27 to 55 and changed with covariate specification. The results were therefore reported with the whole-tract model as primary, the quartile model as confirmation of uniformity, and node-wise profiles as description. Applied to the analysis files of that dataset, `09a_tract_models.py` reproduces the reported whole-tract and quartile estimates.

Preparing node-wise results for the Explorer

Node-wise outputs are stacked into a single long-format file for the Node-wise Tract Explorer, the browser-based viewer described in the Explorer section.

```
python 09c_stack_for_explorer.py "$OUT/permutation" # writes results_long.csv in that directory
```

The file contains one row per node with the columns `outcome`, `tract`, `metric`, `node`, `t`, `p`, together with `hemisphere`, `N`, `covariates`, `extent_threshold`, `cluster_p` and `passed`.

Scripts

The script `09a_tract_models.py` is reproduced in the Supplementary Scripts appendix.

The script `09b_nodewise_permutation.R` is reproduced in the Supplementary Scripts appendix.

The script `09c_stack_for_explorer.py` is reproduced in the Supplementary Scripts appendix.

Node-wise Tract Explorer

The Node-wise Tract Explorer is a browser-based viewer for node-wise statistical results distributed with this site. It reads a results file in the browser, without uploading it, and presents each analysis as a table row that expands to show the node-wise t -value profile, the clusters identified, and, where both hemispheres are present, a left–right comparison. Whole-tract and quartile results are tabular and do not require it.

Open the Explorer · [sample file](#) · [example dataset](#)

Input format

The input is a long-format CSV with one row per node. Six columns are required: `outcome`, `tract`, `metric`, `node`, `t` and `p`. Optional columns are `hemisphere` (L or R), `N`, `covariates`, `extent_threshold`, `cluster_p` and `passed`. Any additional column is treated as a grouping variable and becomes a filter. The column names produced by `09b_nodewise_permutation.R` (`Node`, `t_value`, `p_value`) are accepted without renaming. `09c_stack_for_explorer.py` (Step 9) produces this file from the permutation outputs.

Derived quantities

From the node rows the viewer derives the number of significant nodes, clusters (contiguous runs with $p < .05$), the largest cluster, which clusters exceed the extent threshold, and, when a hemisphere column is present, the counts of significant nodes per hemisphere and their overlap.

Display

Selecting a row shows the t -value profile with a labelled axis and reference lines at ± 2 , significant nodes highlighted, retained clusters shaded, and a cluster table. For lateralized data the left and right profiles are drawn on a common axis so that both hemispheres' full t -value profiles are visible. The hemisphere panel is descriptive: it reports the number of significant nodes per side and their overlap and does not constitute a test of lateralization.

Page labels may be supplied in the URL, for example `?title=...&method=Freedman-Lane&node0=VTA&node1=hippocampus&n_perms=5000` for a ventral tegmental area (VTA) → hippocampus analysis, and `?data=URL` loads a hosted file directly.

Whole-tract extraction

In this approach an atlas map is warped into a participant and a scalar map is averaged within it. It is suitable for descriptive summaries, replication checks and visualization. It is not recommended as a primary endpoint because spatial variation along the tract is not retained.

Procedure

The 50% binary map provides a conservative core; the probabilistic map supports probability-weighted extraction. The map is warped to T1 space with the Step 1 transforms, using nearest-neighbour interpolation for binary maps and linear interpolation followed by thresholding for probabilistic maps.

```
antsApplyTransforms -d 3 -i "$ATLAS_PROB" -r "$T1" -o prob_in_T1.nii.gz \
  -t mni2t1_1Warp.nii.gz -t mni2t1_0GenericAffine.mat -n Linear
fslmaths prob_in_T1.nii.gz -thr 0.50 -bin prob50_in_T1_bin.nii.gz
```

The mask is then moved to the scalar map's grid. When the T1 and the scalar map are header-aligned and differ only in grid, the mask is resampled (`flirt -in mask -ref scalar -applyxfm -usesqform -interp nearestneighbour`); otherwise the T1 → diffusion transform is applied. Voxel count, volume and the mean and standard deviation of the scalar within the mask are then extracted.

```
fslstats mask_on_scalar_grid.nii.gz -V # voxel count, mm3
fslstats "$NDI" -k mask_on_scalar_grid.nii.gz -M -S # mean, SD
```

Reporting

For multi-shell data with a fitted neurite orientation dispersion and density imaging (NODDI) model, the neurite density index (NDI) is generally preferred to fractional anisotropy (FA) as a whole-tract summary. It remains a model-derived quantity sensitive to partial volume, registration and mask boundaries, and should be reported with covariates for age, sex, motion, intracranial volume and whole-white-matter NDI and described as a summary measure.

Atlas-guided synthetic streamlines

This approach is intended for studies that require along-tract profiles but cannot reconstruct the tract in every participant. The warped atlas serves as a geometric scaffold: an ordered centerline is estimated from one endpoint to the other within the mask, 100 nodes are placed along it, jittered streamlines may be generated within the mask, and scalar maps are sampled at each node. The resulting streamlines constitute a reproducible sampling coordinate system rather than a tractography reconstruction, and the approach is less established than corridor-constrained tractography.

Procedure

The probabilistic map is warped onto the scalar grid with linear interpolation and thresholded, at 50% for a conservative core or 25% when the core is short or discontinuous; lower thresholds require justification. The scaffold is built from the mask, the probability map, and native-space start and end regions, with node 0 assigned to the seed side and node 99 to the target side for every participant; an image of the node ordering is saved. Each scalar map is sampled at every node using pyAFQ, DIPY or a custom function, producing one row per participant, tract, metric and node in the long format read by the Explorer. Modelling proceeds as in Step 9 of the main workflow, with cluster or permutation correction.

Reporting

Recommended language: "Atlas-guided streamline scaffolds were used to define a consistent along-tract coordinate system for node-wise sampling." The scaffold should not be described as participant-specific tractography.

Parameters

Table S15 consolidates the parameters used in atlas construction at 7 T and in the example dataset at 3 T.

Table S15

Consolidated Parameters

Step	Parameter	Atlas construction (7 T)	Example dataset (3 T)	Notes
Atlas	threshold	50%	50%	25% when a wa core is short or discontinuous
Registration	tool	antsRegistrationSyNQuick.s h	same	Skull-stripped participant imag against MNI152_T1_1mm n
Warping	interpolation	nearest-neighbour	same	Linear for proba maps, then thres
Corridor	dilation	1–2 voxels	2	4 when registrat uncertain
Tractography	algorithm	iFOD2 (default)	same	
Tractography	-cutoff	0.06 (VTA → hippocampus); 0.08 (hippocampus → accumbens)	0.01	0.01 is the value routine use with corridor mask an default in 00_config.sh; confirm by pilot 4). MRtrix3 def 0.05
Tractography	-select	2500	2500	
Tractography	-seeds	25,000,000	25,000,000	
Tractography	-minlength, -maxlength	35, 65 mm	35, 65 mm	Per tract family
Tractography	-seed_unidirectional	on	on	

Step	Parameter	Atlas construction (7 T)	Example dataset (3 T)	Notes
	, -stop			
Tractography	ACT, backtrack, angle, step size	off / default	off / default	ACT did not reconstruct with masks
Cleaning	n_points, clean_rounds	100, 5	same	
Cleaning	distance_threshold, length_threshold	3, 2	same	Mahalanobis SD length SD
Cleaning	QuickBundles threshold	5 mm	not used	For two-bundle
Quality control	flags		voxels < 50 or > 5,000; maximum density < 5	
Profiling	nodes	100	100	End nodes 0–4 and 99 may be trimmed
Profiling	weighting	Gaussian (Mahalanobis)	same	
NODDI	intrinsic parallel diffusivity, white / gray matter	$1.7 / 1.1 \times 10^{-3} \text{ mm}^2/\text{s}$	same	Refit for gray-matter regions
NODDI	maps	modulated NDI and ODI; FWF	same	
Inference, whole tract	model		mixed model with subregion term; likelihood-ratio test for interaction	one value per participant per tract
Inference, quartiles	segments and correction		nodes 0–24, 25–49, 50–74, 75–99; FDR or maximum-statistic permutation	seed-end versus end contrast for uniformity
Inference, nodes	permutations	5,000	5,000	Freedman–Lane
Inference,	cluster criterion	extent $\geq$ 95th percentile of null	same	$\alpha = .05$ per node

Step	Parameter	Atlas construction (7 T)	Example dataset (3 T)	Notes
nodes		maximum		
Inference, nodes	covariates		ICV, length, count, motion, age	Count is collinearity; NDI; decide in advance

Note. VTA = ventral tegmental area; ACT = anatomically constrained tractography; NODDI = neurite orientation dispersion and density imaging; NDI = neurite density index; ODI = orientation dispersion index; FWF = free-water fraction; FDR = false discovery rate; ICV = intracranial volume.

Troubleshooting

Table S16 lists common problems, their likely causes and remedies. A quality-assurance checklist follows.

Table S16

Common Problems, Causes and Remedies

Symptom	Likely cause	Remedy
Warped atlas displaced anteriorly or posteriorly	Poor T1 → MNI registration, or mismatched coverage (skull-stripped participant image with a whole-head template)	Check brain extraction; use the brain template; overlay the warped template on the T1 before warping labels
Left and right appear reversed	Incorrect transform, header orientation error, mislabelled hemisphere	Check with <code>fslorient</code> and <code>fslhd</code> ; overlay known asymmetric anatomy
Atlas mask empty after warping	Threshold too strict after linear interpolation, or grid mismatch	Inspect the probability map before thresholding; use the 25% map; warp the binary map with nearest-neighbour interpolation
Mask includes ventricle or gray matter	Registration error or excessive dilation	Reduce dilation; inspect in native space
VTA in the cerebral peduncle or pons	Local registration failure	Rerun registration for the participant; consider a midbrain-weighted mask via <code>-x</code>
<code>tckgen</code> reaches far fewer streamlines than the target	Cutoff too high for the field strength; corridor too narrow; seed or target disconnected from the corridor	Run the pilot sweep; confirm the seed and target lie within the inclusion zone; try the 25% atlas or one additional voxel of dilation
Tract appears thick at a permissive cutoff	Expected before cleaning	Compare cleaned bundles; if still thick, reduce dilation
Cleaning removes nearly all streamlines	Two bundles cleaned as one, or a sparse raw tract	Apply the QuickBundles division; check the raw streamline count
Profile endpoints	Orientation failed	Assign node 0 by the seed region;

Symptom	Likely cause	Remedy
reversed in some participants		check centroid orientation per participant
Whole-tract NDI implausibly high or low	Grid mismatch, partial volume, global tissue differences	Confirm a common grid; add a whole-white-matter NDI covariate; inspect the overlay
AMICO <code>FileNotFoundError</code> during parallel runs	Kernel directory regenerated by every job	Generate kernels once on one participant before starting the pool
<code>pip install pyAFQ</code> fails on zipp	Outdated system package	<code>pip install --user --upgrade zipp</code>
Coverage audit flags every corridor as too small	Corridors are below 1% of brain volume by design	Raise the threshold or disregard the flag

Note. MNI = Montreal Neurological Institute; VTA = ventral tegmental area; NDI = neurite density index.

Quality-assurance checklist

- Record atlas version, threshold, dilation, interpolation, transforms, cutoff, scalar maps, covariates and software versions.
- Inspect the atlas in MNI space before warping.
- Inspect every warped region over the T1 and diffusion images.
- Check voxel counts and volumes after warping.
- Save at least one overlay image per participant and step.
- Inspect streamlines in native space and compare with the atlas in MNI space.
- Verify node ordering and coverage.
- Check missingness by tract and participant before group models.

Software

Table S17 lists the software used in the workflow. The workflow depends on FSL (Jenkinson et al., 2012), ANTs (Avants et al., 2008), MRtrix3 (Tournier et al., 2019) and Python with DIPY (Garyfallidis et al., 2014) and pyAFQ (Krupe et al., 2021); AMICO (Daducci et al., 2015) is required only for neurite orientation dispersion and density imaging (NODDI) fitting and R only for the node-wise permutation test.

Table S17

Software Used in the Workflow

Software	Use
FSL	Templates; <code>fslmaths</code> , <code>fslstats</code> , <code>flirt</code> ; <code>probtrackx2</code> for the FSL route
ANTs	<code>antsRegistrationSyNQuick.sh</code> , <code>antsApplyTransforms</code>
MRtrix3	<code>mrconvert</code> , <code>dwi2response</code> , <code>responsemean</code> , <code>dwi2fod</code> , <code>mtnormalise</code> , <code>tckgen</code> , <code>tckinfo</code> , <code>tckstats</code> , <code>tckmap</code> , <code>mrview</code>
Python 3 with DIPY, pyAFQ, nibabel, numpy, scipy, pandas, statsmodels, matplotlib	Cleaning, profiling, quality control, whole-tract and quartile models
AMICO	NODDI fitting
R with readr, dplyr, stringr, tibble, foreach, doParallel	<code>09b_nodewise_permutation.R</code>
FSLeyes, mrview or ITK-SNAP	Inspection
TractSeg (Wasserthal et al., 2018; optional)	Fornix and control tracts

Fibre orientation distribution estimation

When preprocessing ends at the tensor fit, as in the TUBRIC pipeline, the following MRtrix3 commands produce the normalized white-matter fibre orientation distribution (FOD) image required by the workflow. Response functions are estimated per participant with the unsupervised method of Dhollander et al. (2016), averaged across participants, and used for multi-shell multi-tissue constrained spherical deconvolution (Jeurissen et al., 2014) followed by intensity normalisation (Raffelt et al., 2017). The script `00b_fod_estimation.sh`, shown below, runs these commands across participants.

```

# 1. conversion with gradients embedded
mrconvert data.nii.gz dwi.mif -fslgrad bvecs bvals
mrconvert nodif_brain_mask.nii.gz mask.mif

# 2. tissue response functions (multi-shell)
dwi2response dhollander dwi.mif wm_response.txt gm_response.txt csf_response.txt -mask mask.mif

# 3. group-average response functions (once, across participants)
responsemean */wm_response.txt group_wm_response.txt
responsemean */gm_response.txt group_gm_response.txt
responsemean */csf_response.txt group_csf_response.txt

# 4. multi-shell multi-tissue CSD
dwi2fod msmt_csd dwi.mif group_wm_response.txt wm_fod.mif \
    group_gm_response.txt gm_fod.mif group_csf_response.txt csf_fod.mif -mask mask.mif

# 5. intensity normalisation
mtnormalise wm_fod.mif wm_fod_norm.mif gm_fod.mif gm_fod_norm.mif csf_fod.mif csf_fod_norm.mif
-mask mask.mif

```

Single-shell data may use `dwi2response tournier` and `dwi2fod csd`. The corridor workflow is independent of how the FOD image was estimated.

The script `00b_fod_estimation.sh` is reproduced in the Supplementary Scripts appendix.

FSL route

`probtrackx2` accepts the dilated corridor as a `--waypoints` mask and the exclusions as `--avoid`, with a BEDPOSTX model as input. Its output is voxel-wise connectivity rather than streamlines, so along-tract profiling is not available from it; it is appropriate when a whole-tract summary is sufficient.

Scripts

Each script appears in full on the workflow page for its step. Table S18 lists the set in run order. All scripts read a single configuration file, `00_config.sh`, which is edited once per project. The shell scripts source it themselves; the Python and R scripts read the exported settings, so `source 00_config.sh` is run in the shell before they are called, and again after every edit to the file. Each shell script writes a log to `$OUT/logs/`. Steps 0b, 1, 2, 3, 5, 6, 7 and 8a skip participants whose output already exists (`FORCE=1` recomputes); Steps 4, 8, 8b and 9 recompute on every run. Missing inputs are reported by participant.

Table S18

Scripts, in Run Order

File	Step	Requires
<code>00_config.sh</code>	Project paths, tract definition, parameters, covariate list	bash
<code>00b_fod_estimation.sh</code>	0b. Fibre orientation distributions, when preprocessing ended at the tensor	MRtrix3
<code>01_register_mni_to_t1.sh</code>	1. Registration	ANTs
<code>02_warp_rois.sh</code>	2. Warping of seed, target and atlas	ANTs, FSL
<code>03_build_corridor_mask.sh</code>	3. Corridor and exclusion mask	FSL
<code>04_tune_cutoff.sh</code>	4. Pilot sweep of the cutoff	MRtrix3
<code>04b_compare_cutoffs.py</code>	4. Side-by-side images, Dice overlap, summary table	Python, MRtrix3
<code>05_tractography.sh</code>	5. Tractography	MRtrix3
<code>06_clean_bundles.py</code>	6. Cleaning; streamline count and length of the cleaned bundle	Python, pyAFQ, DIPY
<code>07_visual_qc.py</code>	7. Overlay images and flags	Python, MRtrix3
<code>08a_noddi_fit.py</code>	8. NODDI fit	Python, AMICO
<code>08_node_profiles.py</code>	8. Along-tract profiles	Python, DIPY
<code>08b_build_analysis_csv.py</code>	8. Analysis files for Step 9	Python
<code>09a_tract_models.py</code>	9. Whole-tract and quartile models	Python, statsmodels

File	Step	Requires
09b_nodewise_permutation.R	9. Node-wise permutation test	R
09c_stack_for_explorerer.py	9. Explorer input	Python

Note. NODDI = neurite orientation dispersion and density imaging.

The complete set can be retrieved with the following command.

```
mkdir -p mesoconnect_scripts && cd mesoconnect_scripts
base="https://diffusiontensorimaging-repos.github.io/MesoConnect-Tutorial/scripts"
for f in 00_config.sh 00b_fod_estimation.sh 01_register_mni_to_t1.sh 02_warp_rois.sh \
    03_build_corridor_mask.sh 04_tune_cutoff.sh 04b_compare_cutoffs.py \
    05_tractography.sh 06_clean_bundles.py 07_visual_qc.py 08a_noddi_fit.py \
    08_node_profiles.py 08b_build_analysis_csv.py 09a_tract_models.py \
    09b_nodewise_permutation.R 09c_stack_for_explorerer.py; do
    curl -sSLO "$base/$f"
done
```

Validation status

The scripts generalize those used to process the example dataset, in which every step completed for all 57 participants. The generalized set was run from Step 0b to Step 8b on three participants of that dataset (one tract, left posterior ventral tegmental area → hippocampus), starting from the preprocessed diffusion data and the skull-stripped T1 image, and its outputs were compared with those the example dataset's own scripts had produced for the same participants (Table S19). The group-level scripts, which need a full sample, were checked against the example dataset's stored results (Table S20). The run used MRtrix3 3.0.7, FSL 6.0.5.1, ANTs 2.3.5, Python 3.8.10, DIPY 1.8.0, pyAFQ 1.3.5, AMICO 2.1.1 and bash 5.0 on Linux, with four threads and one participant at a time; it took under an hour.

Table S19

Generalized Scripts Compared With the Example Dataset's Outputs, Three Participants

Output	Agreement
Warped seed, target and atlas (Dice overlap)	.93 to .99
Corridor (Dice overlap)	.98 to .99
Pilot sweep, 0.06 versus 0.01: streamlines generated to reach 1,000	0.39 and 3.0 million versus 0.31 and 0.43 million; Dice between cutoffs .68 and .65 (.66 in the example dataset)
Tractography at 0.01	2,500 streamlines in all three; 0.8 to 1.3 million streamlines generated

Output	Agreement
Cleaned bundle, streamline count	864, 886, 705 versus 857, 824, 733
Cleaned bundle, mean length	within 0.4 mm
Quality-control overlays	No flags; bundles follow the expected arc
NODDI maps	Identical, voxel for voxel
Along-tract profiles (correlation across the 100 nodes)	FA .98; NDI .99 to 1.00; ODI .99; FWF .94 to 1.00

Note. Tractography is stochastic, so streamline counts and profiles are not expected to match exactly. NODDI = neurite orientation dispersion and density imaging; FA = fractional anisotropy; NDI = neurite density index; ODI = orientation dispersion index; FWF = free-water fraction.

Table S20

Checks of the Group-Level Scripts

Script	Check	Result
09a_tract_models.py	Run on the analysis files of the example dataset	Reproduces the reported whole-tract and quartile estimates and interaction tests to three decimals
09b_nodewise_permutation.R	Run on an analysis file of the example dataset	Node-wise <i>t</i> values identical to the stored results; same clusters; identical output with one, three and four cores
09c_stack_for_explorer.py	Run on the outputs of 09b	Explorer file written and loaded
08b_build_analysis_csv.py	Run in the three-participant test	Analysis files written for all four metrics

Two defects surfaced during validation and are fixed in the distributed scripts: on bash 5.0 to 5.2 the shell steps stalled at the end of their first participant loop, and the mean streamline length was read from the wrong line of the `tckstats` output. The scripts were also reviewed independently; the defects found are recorded in the repository history.

The example dataset's own per-step scripts, with its cluster paths and participant list, are in the SDN-IMPACT-DTI repository, Steps 20 to 30 of the ReadMe.

Suggested methods text

The paragraphs below describe the corridor workflow in the form required for a Method section and may be adapted. Values in square brackets are study-specific and should be replaced with the values used; values stated without brackets are those of the workflow as documented here.

The whole tutorial is also available as a single document, with tables and figures numbered S1, S2 and so on and the scripts in an appendix: PDF and Word.

Tractography

Mesolimbic pathways were reconstructed in each participant using the MesoConnect Atlas as an anatomical constraint on participant-level tractography. For each pathway, the seed region, the target region and the atlas's 50% group-overlap map were transformed from Montreal Neurological Institute (MNI) space to each participant's T1-weighted image using symmetric diffeomorphic registration (ANTs; Avants et al., 2008) and then to diffusion space using the affine transform estimated during preprocessing, with nearest-neighbour interpolation. The warped atlas was dilated by [two] voxels and combined with the seed and target regions to define an inclusion zone; its inverse served as the sole exclusion mask. Fibre orientation distributions were estimated with multi-shell multi-tissue constrained spherical deconvolution (Jeurissen et al., 2014) using group-averaged response functions (Dhollander et al., 2016) and intensity normalisation in MRtrix3 (Tournier et al., 2019). Probabilistic tractography (iFOD2; Tournier et al., 2010) was seeded unidirectionally from the seed region, required to enter the target region, and terminated on entry, with [2,500] streamlines selected from a maximum of [25 million] seeds, a fibre orientation distribution amplitude cutoff of [0.01], and length bounds of [35] to [65] mm. The cutoff was selected from a pilot sweep in [five] participants as the most permissive value that reached the streamline target in every pilot participant and reproduced the trajectory obtained at the most conservative cutoff.

Bundle cleaning and quality control

Streamlines were resampled to 100 points and cleaned with pyAFQ (Kruyer et al., 2021; Yeatman et al., 2012) by iteratively removing streamlines more than 3 *SD* (Mahalanobis distance) from the bundle centroid or longer than the mean length by more than 2 *SD*, for up to five iterations. [Where a tract reconstructed as two distinct bundles, streamlines were first clustered with QuickBundles (Garyfallidis et al., 2012; 5 mm threshold), each cluster was cleaned separately, and the anatomically correct cluster was retained after inspection.] Every cleaned bundle was inspected as a tract-density overlay on the mean $b = 0$ image; bundles with fewer than 50 or more than 5,000 tract-density voxels, or a maximum density below 5 streamlines per voxel, were flagged for review. [n] participants were excluded at this stage.

Along-tract profiles

Each cleaned bundle was oriented from seed to target against its QuickBundles centroid and sampled at 100 equidistant nodes using Gaussian-weighted tract profiling (Yeatman et al., 2012) as implemented in DIPY (Garyfallidis et al., 2014). [Fractional anisotropy was taken from the diffusion tensor fit. Neurite density index, orientation dispersion index and free-water fraction were obtained from NODDI (Zhang et al., 2012) fitted with AMICO (Daducci et al., 2015), using tissue-weighted modulated maps for neurite density and orientation dispersion (Parker et al., 2021).] Streamline count and mean streamline length were recorded for each bundle as indices of reconstruction quality.

Statistical analysis

[Whole tract: The metric was averaged across nodes and related to the outcome in a linear model (or, for subregional tracts sharing most of their course, a mixed model with a subregion term and a random intercept for participant, in which the outcome $\times$ subregion interaction was tested by likelihood ratio).] [Quartiles: The metric was averaged within nodes 0–24, 25–49, 50–74 and 75–99 and the same model was fitted per quartile, with correction across quartiles by false discovery rate (or maximum-statistic permutation).] [Nodes: At each node the outcome was regressed on the metric and covariates. Adjacent nodes with parametric $p < .05$ formed clusters, and family-wise error was controlled at the cluster level: a null distribution of maximum cluster extent was obtained from 5,000 Freedman–Lane permutations (Freedman & Lane, 1983; Winkler et al., 2014), and clusters were retained when the proportion of permutations with a maximum cluster at least as long did not exceed .05.] Covariates were [intracranial volume, the mean streamline length and streamline count of the cleaned bundle, absolute head motion and age]. [Variables in the whole-tract and quartile models were standardized.] The resolution of analysis and the covariate set were specified before results were examined.

Abbreviations

Abbreviation	Term
ACT	anatomically constrained tractography
AMICO	Accelerated Microstructure Imaging via Convex Optimization
CSD	constrained spherical deconvolution
DIPY	Diffusion Imaging in Python
FA	fractional anisotropy
FDR	false discovery rate
FOD	fibre orientation distribution
FWE	family-wise error
FWF	free-water fraction
HCP	Human Connectome Project
ICV	intracranial volume
iFOD2	second-order integration over fibre orientation distributions
MNI	Montreal Neurological Institute
NDI	neurite density index
NODDI	neurite orientation dispersion and density imaging
ODI	orientation dispersion index
pyAFQ	Python Automated Fiber Quantification
ROI	region of interest
SN	substantia nigra
TDI	tract-density image
VTA	ventral tegmental area

Citation

The atlas manuscript is in preparation. Work that uses the atlas or this workflow should cite that paper; the full reference will be added here when it is published.

Source atlases and software used in the workflow are cited on the pages where they are used and listed on the references page.

References

- Avants, B. B., Epstein, C. L., Grossman, M., & Gee, J. C. (2008). Symmetric diffeomorphic image registration with cross-correlation: Evaluating automated labeling of elderly and neurodegenerative brain. *Medical Image Analysis*, 12(1), 26–41. <https://doi.org/10.1016/j.media.2007.06.004>
- Daducci, A., Canales-Rodríguez, E. J., Zhang, H., Dyrby, T. B., Alexander, D. C., & Thiran, J.-P. (2015). Accelerated microstructure imaging via convex optimization (AMICO) from diffusion MRI data. *NeuroImage*, 105, 32–44. <https://doi.org/10.1016/j.neuroimage.2014.10.026>
- Dhollander, T., Raffelt, D., & Connelly, A. (2016). Unsupervised 3-tissue response function estimation from single-shell or multi-shell diffusion MR data without a co-registered T1 image. *ISMRM Workshop on Breaking the Barriers of Diffusion MRI*, 5.
- Frazier, J. A., Chiu, S., Breeze, J. L., Makris, N., Lange, N., Kennedy, D. N., Herbert, M. R., Bent, E. K., Koneru, V. K., Dieterich, M. E., Hodge, S. M., Rauch, S. L., Grant, P. E., Cohen, B. M., Seidman, L. J., Caviness, V. S., & Biederman, J. (2005). Structural brain magnetic resonance imaging of limbic and thalamic volumes in pediatric bipolar disorder. *American Journal of Psychiatry*, 162(7), 1256–1265. <https://doi.org/10.1176/appi.ajp.162.7.1256>
- Freedman, D., & Lane, D. (1983). A nonstochastic interpretation of reported significance levels. *Journal of Business & Economic Statistics*, 1(4), 292–298. <https://doi.org/10.1080/07350015.1983.10509354>
- Garyfallidis, E., Brett, M., Correia, M. M., Williams, G. B., & Nimmo-Smith, I. (2012). QuickBundles, a method for tractography simplification. *Frontiers in Neuroscience*, 6, 175. <https://doi.org/10.3389/fnins.2012.00175>
- Garyfallidis, E., Brett, M., Amirbekian, B., Rokem, A., van der Walt, S., Descoteaux, M., Nimmo-Smith, I., & DIPY Contributors. (2014). Dipy, a library for the analysis of diffusion MRI data. *Frontiers in Neuroinformatics*, 8, 8. <https://doi.org/10.3389/fninf.2014.00008>
- Hoopes, A., Mora, J. S., Dalca, A. V., Fischl, B., & Hoffmann, M. (2022). SynthStrip: Skull-stripping for any brain image. *NeuroImage*, 260, 119474. <https://doi.org/10.1016/j.neuroimage.2022.119474>
- Jenkinson, M., Beckmann, C. F., Behrens, T. E. J., Woolrich, M. W., & Smith, S. M. (2012). FSL. *NeuroImage*, 62(2), 782–790. <https://doi.org/10.1016/j.neuroimage.2011.09.015>
- Jeurissen, B., Tournier, J.-D., Dhollander, T., Connelly, A., & Sijbers, J. (2014). Multi-tissue constrained spherical deconvolution for improved analysis of multi-shell diffusion MRI data. *NeuroImage*, 103, 411–426. <https://doi.org/10.1016/j.neuroimage.2014.07.061>
- Kruper, J., Yeatman, J. D., Richie-Halford, A., Bloom, D., Grotheer, M., Caffarra, S., Kiar, G.,

- Karipidis, I. I., Roy, E., Chandio, B. Q., Garyfallidis, E., & Rokem, A. (2021). Evaluating the reliability of human brain white matter tractometry. *Aperture Neuro*, 2021(1). <https://doi.org/10.52294/e6198273-b8e3-4b63-babb-6e6b0da10669>
- Makris, N., Goldstein, J. M., Kennedy, D., Hodge, S. M., Caviness, V. S., Faraone, S. V., Tsuang, M. T., & Seidman, L. J. (2006). Decreased volume of left and total anterior insular lobule in schizophrenia. *Schizophrenia Research*, 83(2–3), 155–171. <https://doi.org/10.1016/j.schres.2005.11.020>
- Murty, V. P., Shermohammed, M., Smith, D. V., Carter, R. M., Huettel, S. A., & Adcock, R. A. (2014). Resting state networks distinguish human ventral tegmental area from substantia nigra. *NeuroImage*, 100, 580–589. <https://doi.org/10.1016/j.neuroimage.2014.06.047>
- Parker, C. S., Veale, T., Bocchetta, M., Slattery, C. F., Malone, I. B., Thomas, D. L., Schott, J. M., Cash, D. M., & Zhang, H. (2021). Not all voxels are created equal: Reducing estimation bias in regional NODDI metrics using tissue-weighted means. *NeuroImage*, 245, 118749. <https://doi.org/10.1016/j.neuroimage.2021.118749>
- Pauli, W. M., Nili, A. N., & Tyszka, J. M. (2018). A high-resolution probabilistic in vivo atlas of human subcortical brain nuclei. *Scientific Data*, 5, 180063. <https://doi.org/10.1038/sdata.2018.63>
- Raffelt, D., Dhollander, T., Tournier, J.-D., Tabbara, R., Smith, R. E., Pierre, E., & Connelly, A. (2017). Bias field correction and intensity normalisation for quantitative analysis of apparent fibre density. *Proceedings of the International Society for Magnetic Resonance in Medicine*, 25, 3541.
- Tournier, J.-D., Calamante, F., & Connelly, A. (2010). Improved probabilistic streamlines tractography by 2nd order integration over fibre orientation distributions. *Proceedings of the International Society for Magnetic Resonance in Medicine*, 18, 1670.
- Tournier, J.-D., Smith, R., Raffelt, D., Tabbara, R., Dhollander, T., Pietsch, M., Christiaens, D., Jeurissen, B., Yeh, C.-H., & Connelly, A. (2019). MRtrix3: A fast, flexible and open software framework for medical image processing and visualisation. *NeuroImage*, 202, 116137. <https://doi.org/10.1016/j.neuroimage.2019.116137>
- Trutti, A. C., Fontanesi, L., Mulder, M. J., Bazin, P.-L., Hommel, B., & Forstmann, B. U. (2021). A probabilistic atlas of the human ventral tegmental area (VTA) based on 7 Tesla MRI data. *Brain Structure and Function*, 226, 1155–1167. <https://doi.org/10.1007/s00429-021-02231-w>
- Tziortzi, A. C., Haber, S. N., Searle, G. E., Tsoumpas, C., Long, C. J., Shotbolt, P., Douaud, G., Jbabdi, S., Behrens, T. E. J., Rabiner, E. A., Jenkinson, M., & Gunn, R. N. (2014). Connectivity-based functional analysis of dopamine release in the striatum using diffusion-weighted MRI and positron emission tomography. *Cerebral Cortex*, 24(5), 1165–1177. <https://doi.org/10.1093/cercor/bhs397>

Vu, A. T., Auerbach, E., Lenglet, C., Moeller, S., Sotiropoulos, S. N., Jbabdi, S., Andersson, J., Yacoub, E., & Ugurbil, K. (2015). High resolution whole brain diffusion imaging at 7T for the Human Connectome Project. *NeuroImage*, 122, 318–331.

<https://doi.org/10.1016/j.neuroimage.2015.08.004>

Wasserthal, J., Neher, P., & Maier-Hein, K. H. (2018). TractSeg: Fast and accurate white matter tract segmentation. *NeuroImage*, 183, 239–253.

<https://doi.org/10.1016/j.neuroimage.2018.07.070>

Winkler, A. M., Ridgway, G. R., Webster, M. A., Smith, S. M., & Nichols, T. E. (2014). Permutation inference for the general linear model. *NeuroImage*, 92, 381–397.

<https://doi.org/10.1016/j.neuroimage.2014.01.060>

Yeatman, J. D., Dougherty, R. F., Myall, N. J., Wandell, B. A., & Feldman, H. M. (2012). Tract profiles of white matter properties: Automating fiber-tract quantification. *PLoS ONE*, 7(11), e49790. <https://doi.org/10.1371/journal.pone.0049790>

Zhang, H., Schneider, T., Wheeler-Kingshott, C. A., & Alexander, D. C. (2012). NODDI: Practical in vivo neurite orientation dispersion and density imaging of the human brain. *NeuroImage*, 61(4), 1000–1016. <https://doi.org/10.1016/j.neuroimage.2012.03.072>

Supplementary scripts

The scripts below are those distributed with the tutorial, in run order.

00_config.sh

```
#!/bin/bash
# =====
# MesoConnect corridor workflow: shared configuration
# =====
# Every step script sources this file. Edit this file only; the step scripts
# contain no project-specific values.
#
# What this file is. It holds the project paths, the participant list, the tract
# definition, the tractography parameters, the covariate list and the concurrency
# settings, plus six small shell helpers the step scripts share. Nothing in here
# computes anything; sourcing it takes a fraction of a second.
#
# How to use it. The shell steps source it themselves. The Python and R steps only
# see the exported variables, so run "source 00_config.sh" in your shell before calling
# them, and run it AGAIN after every edit: a sourced value stays in the shell until it is
# re-sourced, and the Python steps will happily keep using a stale TRACT or CUTOFF.
# "bash 00_config.sh" runs it in a throwaway shell and sets nothing; the shebang above
# is only there for editors and syntax checkers.
#
# Expected project layout (one directory per participant):
# $PROJECT/anat/<subj>/<subj>_T1w_brain.nii.gz      skull-stripped T1
# $PROJECT/dwi/<subj>/data.nii.gz, bvals, bvecs    preprocessed diffusion data
# $PROJECT/dwi/<subj>/nodif_brain_mask.nii.gz      diffusion-space brain mask
# $PROJECT/dwi/<subj>/mean_b0.nii.gz               background for QC images (optional)
# $PROJECT/dwi/<subj>/fa.nii.gz                    tensor FA map
# $PROJECT/dwi/<subj>/wm_fod_norm.mif               normalized WM FOD (written by 00b)
# $PROJECT/noddi/<subj>/fit_*.nii.gz               NODDI maps (written by 08a)
# $PROJECT/xfm/<subj>/str2diff.mat                 FLIRT T1 -> diffusion affine
#                                                    (omit if T1 and DWI share a grid)
# Plus two files at the top of $PROJECT: subjects.txt (one ID per line) and
# covariates.csv (a Subject column plus every covariate and outcome you will model).
#
# Everything the workflow writes lands under $OUT (full table on the overview page),
# except the FOD (0b) and NODDI maps (8a), which go beside the raw data as listed above:
# $OUT/<subj>/reg/                                ANTs transforms (Step 1)
# $OUT/<subj>/rois/                                warped seed, target, atlas; corridor; exclusion mask (2, 3)
# $OUT/<subj>/tckgen/<tract>/                       raw and cleaned tractograms (4, 5, 6)
# $OUT/qc/                                         cutoff panels, tckgen summary, overlays, flags (4b, 5, 7)
# $OUT/nodewise/                                  tract stats and 100-node profiles (6, 8)
# $OUT/analysis/                                  one wide CSV per metric; whole-tract/quartile models (8b, 9a)
# $OUT/permutation/                              node-wise permutation results and the Explorer file (9b, 9c)
# $OUT/logs/                                      one log per shell step, appended on every run
#
# Quick sanity check after editing:
# source 00_config.sh; echo "$OUT"; wc -l "$SUBJECTS_FILE"
# ls "$SEED_MNI" "$TARGET_MNI" "$ATLAS_MNI" "$MNI_TEMPLATE"
# If any of those complain, fix the paths here before running Step 1.
# =====

# --- project -----
# "export" makes a variable visible to every program this shell starts, which is how the
# Python and R scripts read these values. A plain VAR=... would stay inside bash.
# PROJECT and ATLAS_DIR are the two paths you set by hand; the rest are built from them.
```

```

export PROJECT="/path/to/project"
# The IDs double as directory names under anat/, dwi/, noddi/ and xfm/, and as the
# Subject column in covariates.csv, so they must match exactly (sub-01 and sub01 are two
# different participants as far as the scripts know). read_subjects() below turns this
# file into the SUBJECTS array; the Python steps read the file themselves, because bash
# cannot export an array.
export SUBJECTS_FILE="$PROJECT/subjects.txt"      # one participant ID per line
# The atlas download (Atlas > Downloads) with its roi_maps/ and
# tracts_thresholded_binary_50/ subfolders. Everything in it sits on the FSL MNI152
# 1 mm grid, the same space as the template Step 1 registers.
export ATLAS_DIR="/path/to/MesoConnectAtlas"      # atlas and region files, MNI 1 mm
# Keeping outputs out of the raw-data tree means a botched run can be deleted with one
# rm -rf and nothing under $PROJECT/dwi is touched.
export OUT="$PROJECT/derivatives/mesoconnect"     # everything this workflow writes
# ${FORCE:-0} means "use FORCE if it is already set and non-empty, otherwise 0". So by
# default a participant whose output exists is skipped (a restart after a crash picks up
# where it stopped), and "FORCE=1 bash 05_tractography.sh" redoes everyone without
# editing this file. Steps 0b, 1, 2, 3, 5, 6, 7 and 8a honour it; Steps 4, 8, 8b and 9
# recompute on every run anyway.
export FORCE="${FORCE:-0}"                        # 1 = recompute existing outputs

# --- tract definition (one tract per run; edit and rerun for another tract) ---
# The workflow does one tract at a time. Step 1 (registration) is per participant and is
# reused; for a second tract change these four lines, re-source, and rerun Steps 2
# through 9. Right hemisphere: right_VTA_0.25_bin, HPC_R_0.5_bin, r_vta_r_hipp.
# Anterior division: the anterior_l_vta_l_hipp atlas file with the same seed and target.
#
# TRACT is a label, not a lookup key. It is spliced into every output name
# (<TRACT>_seed_diff.nii.gz, <TRACT>_<CUTOFF>.tck, <TRACT>_NDI_analysis.csv, ...),
# so pick something without spaces and keep it stable once Step 2 has run.
export TRACT="l_vta_l_hipp"                      # name used for all outputs
# The seed is the left VTA from the 7 T probabilistic atlas of Trutti et al. (2021),
# thresholded at 25% and binarized (the 0.25 in the name); the target is the left
# Harvard-Oxford hippocampus at 50%. Both are binary masks, so Step 2 warps them with
# nearest-neighbour interpolation and re-binarizes, and its audit table checks that they
# do not overlap.
export SEED_MNI="$ATLAS_DIR/roi_maps/left_VTA_0.25_bin.nii.gz"
export TARGET_MNI="$ATLAS_DIR/roi_maps/HPC_L_0.5_bin.nii.gz"
# The group-mean tract map thresholded at 50% of participants (thr50). Step 3 dilates
# this into the corridor. If the warped 50% core comes out short or broken in someone,
# the 25% version of the same tract is the documented fallback (not in the current
# download; the Downloads page says to ask the atlas authors). The trailing backslash
# only continues the string on the next line; do not put a comment between the two.
export ATLAS_MNI="$ATLAS_DIR/tracts_thresholded_binary_50/\
l_vta_l_hipp_1mm_MNI_GroupMean_thr50.nii.gz"

# --- T1 -> diffusion transform applied in Step 2 -----
# matrix apply $PROJECT/xfm/<subj>/str2diff.mat (a missing matrix is a missing input)
# header T1 and diffusion images already share a space: resample by image header only
# Step 2 takes each region MNI -> T1 with the ANTs warp, then T1 -> diffusion with FLIRT.
# "matrix" runs flirt -applyxfm -init str2diff.mat, the affine you made with FLIRT during
# preprocessing. "header" runs flirt -applyxfm -usesqform, which takes the alignment
# from the image headers instead of a matrix file and only resamples onto the diffusion
# grid; that is the HCP-style case where the T1 was already aligned to the diffusion
# data. Any other value makes Step 2 stop with an error before touching a participant.
export T1_TO_DWI="matrix"

# --- corridor and tractography parameters (see Reference: Parameters) -----
# How far Step 3 grows the warped atlas before inverting it into the exclusion mask; one
# fslmaths -dilM pass per voxel. Two voxels at 2 mm isotropic worked for the example
# dataset. Too little and the corridor clips the participant's own tract, so tracking
# stops at the wall and counts drop; too much and the corridor reaches the ventricle or
# cortex (the Step 3 page's sign of over-dilation). Report whatever you use.

```

```

export DILATE_VOX=2          # corridor dilation, voxels (1-2; 4 if registration is uncertain)
# tckgen -cutoff. Tracking stops where the FOD amplitude drops below this. 0.01 is far
# below the MRtrix default (0.05) and is only sane because the exclusion mask keeps
# streamlines inside the corridor; unconstrained, a cutoff this low tracks spurious
# streamlines. Higher values failed to reach the hippocampus in some pilot participants
# and burned many more seeds (Step 4). The value is also baked into filenames
# (<TRACT>_0.01.tck), so if you change it after Step 5 has run, Steps 6 to 8 will look
# for tractograms that do not exist.
export CUTOFF=0.01          # FOD amplitude cutoff selected in Step 4 (MRtrix default 0.05)
# tckgen -select, the number of accepted streamlines to keep before stopping. 2,500 is
# enough for stable node profiles and endpoint maps. It is a target, not a measurement:
# a participant who stops short (say 1,800) is kept, and Step 5 lists who fell short.
export SELECT=2500          # streamlines to retain
# tckgen -seeds, the ceiling on seeding attempts before tckgen gives up on reaching
# SELECT. MRtrix's own default (1,000 x select = 2.5 million) was sometimes not enough,
# so this is ten times that. The example dataset used 0.5 to 2 million per run (about 4%
# of the ceiling); the smaller anterior corridor needed up to 11 million. Raising it
# costs nothing when the target is reached early; lowering it makes more participants
# stop short.
export SEEDS=25000000       # maximum seeding attempts
# tckgen -minlength / -maxlength, in mm. Streamlines outside this window are rejected
# outright. 35 to 65 mm was set for VTA -> hippocampus at 7 T and held at 3 T, where the
# mean lengths came out at 44 to 47 mm. A different tract family needs its own window;
# read it off the pilot lengths that Step 4 prints rather than guessing.
export MINLEN=35            # minimum streamline length, mm
export MAXLEN=65            # maximum streamline length, mm

# --- group-level models (Steps 8b and 9) -----
# One row per participant, a Subject column whose values match subjects.txt, and one
# column per covariate and outcome. Step 8b joins it to the tract statistics from Step 6
# and the 100-node profiles from Step 8 to make $OUT/analysis/<TRACT>_<METRIC>_
# analysis.csv. A participant missing from this file drops out of that join; Step 8b
# prints who that was.
export COVARIATES_CSV="$PROJECT/covariates.csv" # Subject, covariates, outcomes
# Model covariates: columns of $COVARIATES_CSV, plus Mean_length_mm and
# Streamline_count, which Step 6 computes from the cleaned bundle of this tract.
# Comma-separated, no spaces. Steps 9a and 9b split on the comma and look each name up
# as a column, so a typo fails there ("columns not found"), not here. The same list is
# used at every resolution (whole tract, quartiles, nodes) so the models are comparable.
# Streamline_count is the one to think about: it correlates strongly with NDI (r = .63
# to .87 in the example dataset), which eats residual variance. Decide before looking
# at results whether it stays in, and report both specifications.
export COVARIATES="ICV,Mean_length_mm,Streamline_count,absolute_motion,age"
# Freedman-Lane permutations in Step 9b: the reduced-model residuals are shuffled this
# many times, all 100 nodes are refitted each time, and the largest cluster per shuffle
# forms the null for cluster extent. Cluster p values are proportions of these draws,
# so fewer permutations means coarser p values (the smallest is 1/N) and a noisier
# extent threshold; more means proportionally longer runs (a few minutes per call on one
# core at 5,000). The R script seeds its RNG, so a rerun with the same N is identical.
export N_PERMUTATIONS=5000

# --- software and concurrency -----
# Step 1 registers this skull-stripped 1 mm MNI brain (moving) to the participant's
# skull-stripped T1 (fixed); both must be brain-only, since a stripped image against a
# whole-head template biases the registration. $FSLDIR is set by FSL's own setup
# script, so FSL has to be configured in the shell before this file is sourced; if it
# is not, the path silently becomes /data/standard/... and Step 1 fails.
export MNI_TEMPLATE="$FSLDIR/data/standard/MNI152_T1_1mm_brain.nii.gz"
# Same ${var:-default} trick as FORCE: keep an ANTSPATH the shell already has, otherwise
# assume the usual install location. Then put it at the front of PATH so
# antsRegistrationSynQuick.sh and antsApplyTransforms resolve. ANTSPATH is exported on
# its own as well, not just folded into PATH, since the ANTs shell wrappers can look it
# up directly.

```

```

export ANTSPATH="${ANTSPATH:-/usr/local/ants/bin}"
export PATH="$ANTSPATH:$PATH"
# Peak load is roughly threads x concurrent participants; lower these on a shared machine.
# These were tuned for a 48-core node. Worked out: Step 1 runs ANTS_JOBS x ANTS_THREADS
# = 16 threads, the FOD phase of Step 0b runs FOD_JOBS x NTHREADS = 16, and Step 5 is
# serial so it peaks at NTHREADS = 8. On a workstation scale them down so that jobs x
# threads stays within the cores you can spare. Steps 0b, 1 and 5 take hours on a full
# sample whatever you choose; run them under tmux or a scheduler.
#
# Passed as -nthreads to dwi2fod (0b) and tckgen (4, 5). Step 5 at eight threads took
# about 2 min per participant and hemisphere at 3 T. NODDI fitting (8a) does not read
# this; it uses NODDI_NTHREADS (default 4).
export NTHREADS=8          # threads per MRtrix command
# How many participants the light steps (warping, corridor building) run at once via
# throttle() below. Step 0b phase 1 (response functions) uses it too.
export MAXJOBS=8           # participants processed concurrently in light steps (2, 3)
# antsRegistrationSynQuick.sh -n (threads) and how many registrations run side by side.
# Each registration is roughly 10 to 15 min at four threads.
export ANTS_THREADS=4      # threads per registration (Step 1)
export ANTS_JOBS=4         # registrations run concurrently (Step 1)
# Multi-shell dwi2fod is the memory-hungry step, so this is kept low on purpose; raise
# it only after watching memory use on your machine with a couple of participants.
export FOD_JOBS=2          # FOD estimations run concurrently (Step 0b; memory-intensive)

# --- helper functions used by the step scripts -----
# A function is "name() { ... }"; inside it, $1 is the first argument. These exist once
# the file is sourced, so you can call them from an interactive shell too, except
# start_log, which would redirect your terminal.

# Append everything the calling script prints to $OUT/logs/<script>.log.
# Argument: the script path, which every step passes as "$0" (its own name).
# "exec > file" with no command re-points the whole shell's stdout for the rest of the
# run. ">(tee -a ...)" is process substitution: bash starts tee in the background and
# hands us a path that feeds it; tee -a appends to the log AND echoes to the terminal;
# 2>&1 sends stderr down the same pipe so tool warnings are captured too. $(...) runs
# a command and pastes its output in place; basename strips the directory, so
# 05_tractography.sh logs to $OUT/logs/05_tractography.sh.log. Logs accumulate across
# reruns; the "==" line with the date and TRACT marks where each run starts.
start_log() {
    mkdir -p "$OUT/logs"
    exec > >(tee -a "$OUT/logs/${basename "$1"}.log") 2>&1
    echo "==" $(date '+%F %T') $(basename "$1") TRACT=$TRACT"
}

# Participant IDs as the array SUBJECTS. Tolerates CRLF line endings, blank lines
# and a missing final newline, none of which a "while read" loop handles.
# tr -d '\r' strips the carriage returns a Windows editor leaves behind; the unquoted
# $(...) inside ( ) is then split on whitespace into array elements, which is what makes
# blank lines and a missing newline harmless (and why IDs must not contain spaces).
# The step scripts loop with for s in "${SUBJECTS[@]}" (every element, one word each);
# Step 4 takes the first five as pilots with ${SUBJECTS[*]:0:5}; ${#SUBJECTS[@]} is the
# element count. Not exported: the Python steps re-read $SUBJECTS_FILE themselves.
read_subjects() {
    SUBJECTS=$(tr -d '\r' < "$SUBJECTS_FILE")
    echo "==" ${#SUBJECTS[@]} participants in $SUBJECTS_FILE"
}

# Block until fewer than N background jobs are running. Usage: throttle N
# The step scripts launch one participant in the background with "work "$s" &" and call
# throttle right after, so at most N participants run at once. "jobs -r" lists this
# shell's running background jobs, wc -l counts the lines, and -ge is >= for integers.
# Polling once a second is plenty for jobs that take minutes.
throttle() {

```

```

while [ "$(jobs -r | wc -l)" -ge "$1" ]; do
    sleep 1
done
}

# Block until every background job has finished. A bare "wait" must not be used:
# bash 5.0 to 5.2 would also wait for the logging process started by start_log,
# which never exits while the script is running.
# (That is the stall recorded on the Scripts reference page: the shell steps hung at the
# end of their first participant loop.) throttle 1 waits until zero jobs are running,
# which is the same thing without that problem. Every parallel step calls this before
# printing its audit table.
wait_for_jobs() {
    throttle 1
}

# Number of non-zero voxels in an image.
# Argument: a NIfTI path. fslstats -V prints two numbers, the non-zero voxel count and
# the same volume in mm3; awk '{print $1}' keeps the first field. Steps 2 and 3 use it
# for their audit tables (seed, target and atlas sizes, seed-target overlap, corridor
# size), where a 0 or a count wildly unlike the rest of the sample is the first sign of
# a bad registration.
nvox() {
    fslstats "$1" -V | awk '{print $1}'
}

# "ok" if the file exists, otherwise "MISSING"; used in the audit tables.
# [ -f path ] is true for an existing regular file. Steps 0b and 1 print one row per
# participant with this, so the end of the log tells you who to rerun.
present() {
    if [ -f "$1" ]; then echo ok; else echo MISSING; fi
}

```

01_register_mni_to_t1.sh

```

#!/bin/bash
# =====
# Step 1. Nonlinear registration of the MNI template to each participant's T1
# =====
# Fixed image: participant T1 (skull-stripped). Moving image: MNI152 1 mm brain.
# The forward transforms (mni2t1_0GenericAffine.mat, mni2t1_1Warp.nii.gz) carry
# MNI-space files into T1 space in Step 2; the inverse warp returns
# participant-level results to MNI space.
#
# Why we need this: the atlas, seed and target are drawn in MNI space, but we track
# in each participant's own diffusion space. An affine handles global size, rotation
# and shear, not where one person's midbrain sits, and a few mm of error there puts
# the VTA seed outside the VTA. So we compute a nonlinear warp (ANTs SyN) from MNI to
# the T1 here, and Step 2 chains it with the T1 -> diffusion transform from
# preprocessing. If the T1 already sits on the diffusion grid, this warp is the only
# transform needed (T1_TO_DWI=header in 00_config.sh).
#
# Needs before it runs:
#   00_config.sh edited for this project (PROJECT, OUT, SUBJECTS_FILE, ANTS_*)
#   $PROJECT/anat/<subj>/<subj>_T1w_brain.nii.gz skull-stripped T1, one per participant
#   $MNI_TEMPLATE, which 00_config.sh points at $FSLDIR/data/standard/MNI152_T1_1mm_brain,
#   so FSL must be set up in this shell before you run this. ANTPATH must
#   point at your ANTs bin directory (00_config.sh assumes /usr/local/ants/bin)
#   Both images must be brain-only. A stripped T1 against a whole-head template (or the
#   reverse) drags the solution toward the skull and biases the warp. If your T1 still
#   has a skull, strip it first (the example dataset used ANTs brain extraction).

```

```

#
# Writes, per participant, into $OUT/<subj>/reg/:
#   mni2t1_0GenericAffine.mat   linear part of the MNI -> T1 transform
#   mni2t1_1Warp.nii.gz        nonlinear MNI -> T1 warp; Step 2 applies this + the affine
#   mni2t1_1InverseWarp.nii.gz T1 -> MNI warp, for taking participant results back to MNI
#   mni2t1_Warped.nii.gz       the template resampled into this participant's T1 space;
#                               nothing downstream reads it, but it is the QC image
# Everything printed also lands in $OUT/logs/01_register_mni_to_t1.sh.log.
#
# Run:   bash 01_register_mni_to_t1.sh          (FORCE=1 bash 01... redoes everyone)
# Time:  about 10-15 min per participant at 4 threads, 4 participants at a time, so a
#         full sample takes hours. Run it under tmux or a job scheduler. Peak load is
#         about ANTS_THREADS x ANTS_JOBS threads (16 by default); lower both in
#         00_config.sh on a workstation or a shared node.
# Check: the audit table at the end should read "ok" three times per participant (all
#         57 did in the example dataset). Then open mni2t1_Warped.nii.gz over the T1 for
#         each participant: ventricles, corpus callosum and brainstem outline should
#         coincide. A warp that looks fine globally can still be off in the midbrain,
#         which is why Step 2 has you inspect the warped VTA itself as well.
# =====

# Pull in the paths, parameters and helper functions (start_log, read_subjects,
# throttle, wait_for_jobs, present). "$(...)" runs a command and pastes its output into
# the line; "$(dirname "$0")" is the directory this script lives in, so it finds
# 00_config.sh no matter which directory you call it from.
source "$(dirname "$0")/00_config.sh"
# From here on everything printed goes to the terminal and to the log file.
start_log "$0"
# Fills the SUBJECTS array from $SUBJECTS_FILE (one ID per line).
read_subjects

# Register one participant. Argument: the participant ID. Writes the mni2t1_* files
# into $OUT/<id>/reg/ and prints one status line. It returns early, without stopping
# the script, if the T1 is missing or the work is already done. Several copies run at
# once in the background, so it only ever touches its own participant's directory.
register_one() {
    local s=$1                                # "local": visible inside this function only
    local t1="$PROJECT/anat/$s/${s}_T1w_brain.nii.gz"
    local d="$OUT/$s/reg"
    # No T1, no registration. "[ ! -f X ]" is true when X is not an existing file. The
    # "!!" prefix flags problems in the log; the audit table will also show MISSING.
    # "return" leaves this function only; the script itself keeps going.
    if [ ! -f "$t1" ]; then
        echo "!! $s missing $t1"
        return
    fi
    # Skip participants who already have a warp, so an interrupted run can be restarted
    # with the same command. "&&" runs the second test only if the first passes; FORCE
    # defaults to 0 in 00_config.sh and "FORCE=1 bash ..." sets it for one run. The warp is
    # the done marker. Unlike Steps 0b and 5, nothing here writes to a temp name and renames
    # on completion, so if you killed a run midway and are not sure what it left behind,
    # delete that participant's reg/ directory or rerun with FORCE=1.
    if [ "$FORCE" != 1 ] && [ -f "$d/mni2t1_1Warp.nii.gz" ]; then
        echo "== $s already registered (FORCE=1 to recompute)"
        return
    fi
    mkdir -p "$d"                                # -p: make parents too, no error if present
    # antsRegistrationSynQuick.sh is the ANTs wrapper that runs an affine stage and then a
    # SyN (symmetric diffeomorphic) nonlinear stage, which is why we get both a .mat and a
    # warp. Its registration settings live inside the wrapper; this call only passes the
    # options below. It is what the tutorial was validated with (ANTs 2.3.5). Option by
    # option (trailing "\" continues the command on the next line):
    #   -d 3   image dimension; 3 for a volume

```

```

# -f      fixed image, the participant's T1: the space everything is moved INTO
# -m      moving image, the MNI brain template: the space the atlas files start in
# -o      output prefix; ANTs appends 0GenericAffine.mat, 1Warp.nii.gz and so on
# -n      threads for this one registration (ANTS_THREADS in 00_config.sh)
# Fixed and moving are deliberately this way round: the forward transforms take
# MNI-space images into the T1, which is what Step 2 applies to the seed, target and
# atlas. Swap them and you get a T1 -> MNI warp that Step 2 cannot use.
antsRegistrationSyNQuick.sh -d 3 \
    -f "$t1" \
    -m "$MNI_TEMPLATE" \
    -o "$d/mni2t1_" \
    -n "$ANTS_THREADS"
# This prints even if ANTs failed partway, so trust the audit table, not this line.
echo ">> $s registered"
}

# Launch the registrations. "${SUBJECTS[@]}" expands the array to one word per ID, and
# the quotes keep an odd ID from splitting. The trailing "&" starts register_one in the
# background and moves straight on; throttle then sleeps until fewer than ANTS_JOBS
# jobs are running, so at most ANTS_JOBS registrations are ever active at once. The log
# is shared, so output from different participants interleaves; the status lines carry
# the ID, and the audit below is the tidy summary.
for s in "${SUBJECTS[@]}"; do
    register_one "$s" &
    throttle "$ANTS_JOBS"
done
# Block until the last background registration finishes before we audit. This is a
# helper rather than a bare "wait" because "wait" would also hang on the tee process
# that start_log opened; 00_config.sh explains.
wait_for_jobs

# Audit
# One row per participant, "ok" or "MISSING" for each of the three transform files.
# present() is the helper from 00_config.sh and "$(present ...)" drops its answer into
# the printf; "\t" makes tab-separated columns that paste cleanly into a spreadsheet.
# Step 2 skips any participant whose affine or warp is MISSING, so fix those first
# (usually a missing T1 or an ANTs crash; the log has the details). The table does not
# check mni2t1_Warped.nii.gz, the QC image; that one you look at by eye. "d" is not
# "local" here because we are back at the top level of the script, not in a function.
printf "\nSubject\tAffine\tWarp\tInverseWarp\n"
for s in "${SUBJECTS[@]}"; do
    d="$OUT/$s/reg"
    printf "%s\t%s\t%s\t%s\n" "$s" \
        "$(present "$d/mni2t1_0GenericAffine.mat")" \
        "$(present "$d/mni2t1_1Warp.nii.gz")" \
        "$(present "$d/mni2t1_1InverseWarp.nii.gz")"
done

```

02_warp_rois.sh

```

#!/bin/bash
# =====
# Step 2. Warp the seed, target and tract atlas into diffusion space
# =====
# MNI -> T1 with the ANTs transforms from Step 1, then T1 -> diffusion with FLIRT,
# using the matrix or the image header according to T1_TO_DWI in 00_config.sh.
# Nearest-neighbour interpolation throughout; outputs are re-binarized.
#
# Three MNI-space label images go through this for every participant: the seed, the
# target and the atlas's 50% binary map (SEED_MNI, TARGET_MNI and ATLAS_MNI in
# 00_config.sh). They are 0/1 masks, which is why both resamplings use nearest

```

```

# neighbour: linear interpolation would smear the edges into fractions and shift the
# boundary wherever we threshold. If you ever need the probabilistic atlas in native
# space instead, warp it with linear interpolation and threshold afterwards.
#
# Needs, per participant <subj>:
#   $OUT/<subj>/reg/mni2t1_0GenericAffine.mat      written by Step 1
#   $OUT/<subj>/reg/mni2t1_1Warp.nii.gz           written by Step 1
#   $PROJECT/anat/<subj>/<subj>_T1w_brain.nii.gz   skull-stripped T1 (grid for stage 1)
#   $PROJECT/dwi/<subj>/nodif_brain_mask.nii.gz    diffusion grid the output lands on
#   $PROJECT/xfm/<subj>/str2diff.mat              only when T1_TO_DWI=matrix
# Writes, per participant, into $OUT/<subj>/rois/:
#   ${TRACT}_{seed,target,atlas}_t1.nii.gz       T1-space intermediates (kept for checks)
#   ${TRACT}_{seed,target,atlas}_diff.nii.gz      diffusion space, binary; the real outputs
# Run:  bash 02_warp_rois.sh                      (FORCE=1 bash 02_warp_rois.sh redoes everyone)
# Time: seconds per participant, MAXJOBS participants at a time.
# Check: the audit table at the end of the log ($OUT/logs/02_warp_rois.sh.log). Every
# row should have non-zero seed, target and atlas counts of similar size across the
# sample, and seed_target_overlap must be 0. Then look at the _diff images over the
# mean b=0 (the tutorial page shows what good placement looks like). Step 3 reads the
# three _diff files to build the corridor; Step 5 seeds from seed_diff and includes
# target_diff.
# =====
# 00_config.sh sits next to this script. $(...) runs a command and pastes its output in
# place; $0 is the path you invoked the script by and dirname strips the file name off
# it, so this finds the config next to the script file no matter which directory you
# call it from. "source" runs the file in this same shell, which is what makes its
# exports and helper functions available below.
source "$(dirname "$0")/00_config.sh"
# From here on everything printed also lands in $OUT/logs/02_warp_rois.sh.log.
start_log "$0"
# Fills the SUBJECTS array from $SUBJECTS_FILE (one ID per line).
read_subjects

# Warp one participant. Argument: the participant ID. Writes the six files listed in
# the header into $OUT/<subj>/rois/ and prints one status line. A missing input prints
# a "!!" line and returns, so one broken participant never stops the whole run.
warp_one() {
    # "local" keeps these names inside this call instead of leaking into the rest of the
    # script (the audit loop at the bottom reuses d and s). Each backgrounded call below
    # is its own process anyway, so this is hygiene, not a fix for a race.
    # $1 is the first argument handed to the function.
    local s=$1
    local d="$OUT/$s/rois"
    local t1="$PROJECT/anat/$s/${s}_T1w_brain.nii.gz"
    local ref="$PROJECT/dwi/$s/nodif_brain_mask.nii.gz"
    local warp="$OUT/$s/reg/mni2t1_1Warp.nii.gz"
    local aff="$OUT/$s/reg/mni2t1_0GenericAffine.mat"
    local mat="$PROJECT/xfm/$s/str2diff.mat"
    local name src in_t1 in_diff

    # Resume logic. The atlas is the last of the three files written, so if it exists this
    # participant finished on an earlier run and we skip. FORCE=1 overrides that.
    # [ -f X ] is true when X exists as a regular file; && means both tests must pass.
    if [ "$FORCE" != 1 ] && [ -f "$d/${TRACT}_atlas_diff.nii.gz" ]; then
        echo "== $s already warped"
        return
    fi

    # Step 1 must have produced both transforms, and we need the diffusion mask as the
    # output grid. "||" chains the tests: any one of them missing is enough to skip.
    if [ ! -f "$warp" ] || [ ! -f "$aff" ] || [ ! -f "$ref" ]; then
        echo "!! $s missing registration outputs or diffusion reference"
        return
    fi
}

```

```

# The FLIRT matrix is only an input in matrix mode; header mode does not use it.
if [ "$T1_TO_DWI" = matrix ] && [ ! -f "$mat" ]; then
    echo "!! $s missing $mat"
    return
fi
mkdir -p "$d"

# The same two-stage warp for each of the three images; only the source differs.
for name in seed target atlas; do
    # case/esac is the shell's switch statement: pick the MNI source for this name.
    case $name in
        seed)    src=$SEED_MNI ;;
        target) src=$TARGET_MNI ;;
        atlas)   src=$ATLAS_MNI ;;
    esac
    in_t1="$d/${TRACT}_${name}_t1.nii.gz"
    in_diff="$d/${TRACT}_${name}_diff.nii.gz"

    # Stage 1, MNI -> T1, with the registration from Step 1.
    #   -d 3                3-D image
    #   -i / -o            input in MNI space, output in T1 space
    #   -r "$t1"           reference: the output takes this image's grid (size, spacing)
    #   -t warp -t aff      the two Step 1 transforms. ANTs treats -t as a stack and
    #                       applies the last one listed first, so the affine goes on
    #                       before the warp. That is the order the Step 1 outputs are
    #                       meant to be combined in; do not swap them
    #   -n NearestNeighbor label image: never average 0s and 1s into fractions
    antsApplyTransforms -d 3 -i "$src" -r "$t1" -o "$in_t1" \
        -t "$warp" -t "$aff" -n NearestNeighbor

    # Stage 2, T1 -> diffusion, with FLIRT. -applyxfm means resample with the given
    # transform and do not estimate a new one. -ref sets the output grid (the diffusion
    # brain mask), and -interp nearestneighbour is there for the same reason as above.
    if [ "$T1_TO_DWI" = matrix ]; then
        # -init "$mat" is the T1 -> diffusion affine your preprocessing already estimated.
        flirt -in "$in_t1" -ref "$ref" -applyxfm -init "$mat" \
            -interp nearestneighbour -out "$in_diff"
    else
        # T1 and diffusion share a space: resample onto the diffusion grid by header.
        # -usesqform takes the alignment from the two images' sform/qform headers instead
        # of a matrix file, so this branch is only right when they already line up.
        flirt -in "$in_t1" -ref "$ref" -applyxfm -usesqform \
            -interp nearestneighbour -out "$in_diff"
    fi

    # Belt and braces. Nearest neighbour should already give 0/1, but -thr 0.5 (zero
    # anything below 0.5) then -bin (set what is left to 1) guarantees a clean binary
    # mask. Step 3 adds and inverts these and Step 5 seeds from them, so 0/1 matters.
    fslmaths "$in_diff" -thr 0.5 -bin "$in_diff"
done
echo ">> $s warped"
}

# Say which T1 -> diffusion mode this run uses, and stop now on a typo in 00_config.sh
# rather than after warping everyone the wrong way. "*" is the catch-all branch.
case "$T1_TO_DWI" in
    matrix) echo "== T1 -> diffusion: applying xfm/<sub>j>/str2diff.mat" ;;
    header) echo "== T1 -> diffusion: resampling by image header (no matrix)" ;;
    *)      echo "!! T1_TO_DWI must be matrix or header"; exit 1 ;;
esac

# Run the participants MAXJOBS at a time. "${SUBJECTS[@]}" expands to every ID as its own
# word. The trailing "&" sends warp_one to the background so the loop moves straight on;
# throttle (00_config.sh) then blocks whenever MAXJOBS copies are already running.

```

```

for s in "${SUBJECTS[@]}"; do
    warp_one "$s" &
    throttle "$MAXJOBS"
done
# Waits for every background warp_one. Do not swap this for a bare "wait": on some bash
# versions that also waits on the logging process from start_log, which never exits.
wait_for_jobs

# Audit: voxel counts and seed-target overlap (the overlap must be 0)
# One tab-separated row per participant so the table pastes straight into a spreadsheet.
# mktemp -d makes a fresh empty scratch directory for the overlap image; removed at the
# end.
tmp=$(mktemp -d)
printf "\nSubject\tseed_vox\ttarget_vox\tatlas_vox\tseed_target_overlap\n"
for s in "${SUBJECTS[@]}"; do
    d="$OUT/$s/rois"
    # A participant skipped above shows up as MISSING instead of handing fslstats files
    # that do not exist. "continue" jumps to the next participant.
    if [ ! -f "$d/${TRACT}_atlas_diff.nii.gz" ]; then
        printf "%s\tMISSING\n" "$s"
        continue
    fi
    # seed x target is 1 only where both masks are 1. Any non-zero count means the two
    # regions touch after warping. Step 5 seeds inside seed_diff and keeps a streamline
    # only once it reaches target_diff (-include, -stop), so a voxel in both would count
    # as "reached" before the streamline has gone anywhere.
    fslmaths "$d/${TRACT}_seed_diff.nii.gz" -mul "$d/${TRACT}_target_diff.nii.gz" "$tmp/overlap"
    # nvox (00_config.sh) is fslstats -V, first column: the number of non-zero voxels.
    # Read the counts against each other, not against a fixed number. One far from the
    # rest (more than 2 SD is a good rule) points to a bad Step 1 registration for that
    # participant; look at the _t1 and _diff images before going on.
    printf "%s\t%s\t%s\t%s\t%s\n" "$s" \
        "$(nvox "$d/${TRACT}_seed_diff.nii.gz")" \
        "$(nvox "$d/${TRACT}_target_diff.nii.gz")" \
        "$(nvox "$d/${TRACT}_atlas_diff.nii.gz")" \
        "$(nvox "$tmp/overlap")"
done
rm -rf "$tmp"

```

03_build_corridor_mask.sh

```

#!/bin/bash
# =====
# Step 3. Build the corridor and its exclusion mask
# =====
# Dilate the warped atlas, add the seed and target, binarize (inclusion zone),
# then invert. The inverted image is the single exclusion mask given to tckgen:
# every voxel outside the corridor terminates and discards a streamline.
#
# This is the whole trick of the corridor workflow. The atlas itself was built
# with a dozen or so separately prepared exclusion regions per tract (listed on
# the atlas Tracts page); here one mask cut from the warped atlas does that job,
# and anything that strays out of it is thrown away.
#
# Needs, per participant, in $OUT/<participant>/rois/ (all written by Step 2):
# <TRACT>_atlas_diff.nii.gz      50% group atlas, 0/1, on the diffusion grid
# <TRACT>_seed_diff.nii.gz      seed region, 0/1, diffusion grid
# <TRACT>_target_diff.nii.gz    target region, 0/1, diffusion grid
# Writes, into the same directory:
# <TRACT>_atlas_dilated.nii.gz  atlas grown by DILATE_VOX voxels
# <TRACT>_inclusion_zone.nii.gz  dilated atlas + seed + target, 0/1 (the corridor)

```

```

# <TRACT>_exclusion_mask.nii.gz the corridor inverted: 1 = forbidden
# Steps 4 and 5 hand <TRACT>_exclusion_mask.nii.gz to tckgen as -exclude. The
# inclusion zone is only for the audit below and for eyeballing in a viewer (the
# figure on the Step 3 page); no later script reads it.
#
# Run: bash 03_build_corridor_mask.sh (after Step 2; FORCE=1 to redo all)
# Light step: a handful of fslmaths calls per participant, each on a small mask,
# so a full sample takes minutes. Output is copied to
# $OUT/logs/03_build_corridor_mask.sh.log.
#
# What to look at: the audit table at the end. seed_excluded and target_excluded
# must be 0 for every participant, and corridor_vox should be similar across
# participants (1,526 to 1,934 at 2 mm for the posterior tract in the example
# data; about 1,400 for the anterior). The corridor is under 1% of the brain by
# design, so a generic coverage check will flag it; ignore that. Then open one
# inclusion zone over the mean b=0 and check it runs seed to target without
# touching ventricle or cortex (that means too much dilation or a bad
# registration).
# =====
# $(...) runs a command and pastes its output in place. $0 is this script's own
# path, so dirname "$0" is the folder it lives in: the config is found next to the
# script no matter where you call it from. "source" runs the config inside this
# shell, which is how its exports and helper functions (start_log, read_subjects,
# throttle, wait_for_jobs, nvox) become available here.
source "$(dirname "$0")/00_config.sh"
# From here on everything printed also lands in $OUT/logs/03_build_corridor_mask.sh.log
start_log "$0"
# Fills the SUBJECTS array from $SUBJECTS_FILE, one ID per line
read_subjects

# One -dilM pass (3 x 3 x 3 kernel) per voxel of requested dilation
# The script repeats -dilM once per voxel rather than changing the kernel, so
# DILATE_VOX=2 gives "-dilM -dilM". With that kernel, each pass on a 0/1 image
# turns every zero voxel touching the mask (face, edge or corner) into a 1, so N
# passes grow it by N voxels in every direction. for ((...)) is bash's C-style
# counting loop.
#
# Why 2: the 50% atlas is already a bit wider than most participants' core, so two
# voxels (4 mm at 2 mm isotropic) is enough slack for registration error. Use 1 if
# registration is tight and pilot bundles look loose; 4 only as a sensitivity run
# when registration is doubtful. Too wide and the corridor reaches ventricle or
# cortex; too narrow and tckgen struggles to reach its streamline target. Report
# whichever value you use.
DILATE_ARGS=""
for ((i = 0; i < DILATE_VOX; i++)); do
    DILATE_ARGS="$DILATE_ARGS -dilM"
done

# build_one <participant ID>
# Builds the three corridor images for one participant under
# $OUT/<participant>/rois/. Skips a participant whose mask already exists
# (unless FORCE=1) and prints a "!!" line when Step 2's atlas is missing. Prints
# one line per participant; returns nothing. It is launched in the background
# below, so it runs in its own copy of the shell: all it can do is write files and
# print, nothing it sets is visible to the main script.
build_one() {
    # "local" scopes a variable to this function call, so these names cannot leak
    # into or collide with the script's globals (the audit loop below also uses d).
    # $1 is the first argument passed to the function.
    local s=$1
    local d="$OUT/$s/rois"
    # [ -f path ] is true when the file exists. Restart-safe: the exclusion mask is
    # the last file written, so if it is there this participant is done and a re-run

```

```

# only fills in the gaps. FORCE=1 (exported by 00_config.sh) overrides that.
# "return" leaves the function right here.
if [ "$FORCE" != 1 ] && [ -f "$d/${TRACT}_exclusion_mask.nii.gz" ]; then
    echo "== $s corridor exists"
    return
fi
# No warped atlas means Step 2 did not finish for this participant. Say so and
# move on instead of letting fslmaths die with a less helpful message.
if [ ! -f "$d/${TRACT}_atlas_diff.nii.gz" ]; then
    echo "!! $s missing warped atlas (run Step 2)"
    return
fi
# $DILATE_ARGS is intentionally unquoted so that it expands to separate options
# (quoted, bash would hand fslmaths " -dilm -dilm" as one unknown option).
# Output: the atlas grown by DILATE_VOX voxels.
fslmaths "$d/${TRACT}_atlas_diff.nii.gz" $DILATE_ARGS "$d/${TRACT}_atlas_dilated.nii.gz"
# Corridor = dilated atlas + seed + target. -add sums the images voxel by voxel
# (a voxel in both atlas and seed becomes 2), then -bin sets every non-zero
# voxel to 1. The seed and target have to go in here, before the inversion:
# leave them out and the corridor ends short of them, so tracking stops at the
# boundary and never reaches the target.
fslmaths "$d/${TRACT}_atlas_dilated.nii.gz" \
    -add "$d/${TRACT}_seed_diff.nii.gz" \
    -add "$d/${TRACT}_target_diff.nii.gz" \
    -bin "$d/${TRACT}_inclusion_zone.nii.gz"
# -binv binarizes and inverts: 1 wherever the corridor is 0, and 0 inside it.
# This is the file tckgen gets as -exclude in Steps 4 and 5; a streamline that
# enters any 1 voxel is discarded.
fslmaths "$d/${TRACT}_inclusion_zone.nii.gz" -binv "$d/${TRACT}_exclusion_mask.nii.gz"
# nvox (00_config.sh) is fslstats -V, the number of non-zero voxels. A quick
# sanity number: a corridor of 0 or of half the brain is wrong either way.
echo ">> $s corridor: $(nvox "$d/${TRACT}_inclusion_zone.nii.gz") voxels"
}

# Participants run in parallel. "command &" starts build_one in the background
# and moves straight on; throttle (00_config.sh) then sleeps until fewer than
# MAXJOBS background jobs are running, so at most MAXJOBS participants are in
# flight at once. "${SUBJECTS[@]}" expands the array to one word per ID.
for s in "${SUBJECTS[@]}"; do
    build_one "$s" &
    throttle "$MAXJOBS"
done
# Block until the last background job is done; the audit needs every mask on disk.
# (Not a bare "wait": see the note on wait_for_jobs in 00_config.sh.)
wait_for_jobs

# Audit: no seed or target voxel may fall inside the exclusion mask (both counts 0)
# Multiplying the exclusion mask by the seed leaves a 1 only where a seed voxel is
# forbidden; same for the target. Anything but 0 means tckgen would discard
# streamlines the moment they start (seed) or arrive (target), so fix that
# participant before Step 4. The table is tab-separated so it pastes straight
# into a spreadsheet, and it is in the log as well.
# mktemp -d makes a fresh, uniquely named scratch directory (under $TMPDIR or
# /tmp) so the throwaway images here never collide with anything else.
tmp=$(mktemp -d)
# printf with \t writes real tab characters; the leading \n leaves a blank line
# between the per-participant messages above and the table.
printf "\nSubject\tseed_excluded\ttarget_excluded\tcorridor_vox\n"
for s in "${SUBJECTS[@]}"; do
    d="$OUT/$s/rois"
    excl="$d/${TRACT}_exclusion_mask.nii.gz"
    # A participant skipped above (no Step 2 output) gets a MISSING row rather than
    # crashing the audit; "continue" jumps to the next participant.

```

```

if [ ! -f "$excl" ]; then
    printf "%s\tMISSING\n" "$s"
    continue
fi
# -mul multiplies voxel by voxel. The outputs are named without an extension;
# FSL adds its own (normally .nii.gz) and fslstats, inside nvox, finds them the
# same way.
fslmaths "$excl" -mul "$d/${TRACT}_seed_diff.nii.gz" "$tmp/seed"
fslmaths "$excl" -mul "$d/${TRACT}_target_diff.nii.gz" "$tmp/target"
# One row: ID, forbidden seed voxels, forbidden target voxels, corridor size.
# The corridor should be a small fraction of the brain, well under 20%.
printf "%s\t%s\t%s\t%s\n" "$s" \
    "$ (nvox "$tmp/seed")" \
    "$ (nvox "$tmp/target")" \
    "$ (nvox "$d/${TRACT}_inclusion_zone.nii.gz")"
done
# Throw the scratch directory away; the only thing worth keeping is the table.
rm -rf "$tmp"

```

04_tune_cutoff.sh

```

#!/bin/bash
# =====
# Step 4. Pilot sweep of the FOD amplitude cutoff
# =====
# Runs reduced-budget tractography at several cutoffs in a few participants.
# "Selected" is the number of streamlines that met every criterion; "Generated" is
# the number tckgen had to generate (selected plus rejected) to obtain them. tckgen
# generates one streamline per seed, so it is also the number of seeds consumed.
#
# Why this exists. The cutoff is the minimum FOD amplitude at which tckgen keeps
# tracking. Too high and streamlines die before the target; too low and, without a
# corridor, they wander all over the brain. Inside the corridor a very low cutoff
# (0.01, the default in 00_config.sh) reached the target in every pilot participant of
# the example dataset with about a fifth of the seeds 0.06 needed, and later in all 57
# at the production budget. That has to be checked again on each new dataset or tract
# family, which is what this sweep is for.
#
# Needs, per pilot participant (Steps 0b, 2 and 3 must have run for them):
# $PROJECT/dwi/<s>/wm_fod_norm.mif          normalized WM FOD (Step 0b)
# $OUT/<s>/rois/<TRACT>_seed_diff.nii.gz    seed region in diffusion space (Step 2)
# $OUT/<s>/rois/<TRACT>_target_diff.nii.gz  target region in diffusion space (Step 2)
# $OUT/<s>/rois/<TRACT>_exclusion_mask.nii.gz inverted corridor (Step 3)
# Writes:
# $OUT/<s>/tckgen/<TRACT>/<TRACT>_pilot_<cutoff>.tck  one tractogram per cutoff
# $OUT/logs/04_tune_cutoff.sh.log                the table below, kept on disk
# Usage:
# bash 04_tune_cutoff.sh "sub-01 sub-02 sub-03 sub-04 sub-05" "0.1 0.08 0.06 0.01"
# Defaults: the first five participants in $SUBJECTS_FILE and the four cutoffs above.
# Runtime: under an hour for five participants and four cutoffs at the default NTHREADS.
# Expect the high cutoffs to take longest, since they burn the whole 5 million seed
# budget without reaching 1000; 0.01 used about 415,000 seeds in the example dataset.
# What to look for. A cutoff is a candidate only if Selected reaches 1000 in every
# pilot participant. Selected below 1000 with Generated at or near the 5 million seed
# limit means the budget ran out. MeanLen_mm should be similar across cutoffs and sit
# well inside MINLEN and MAXLEN; for a new tract family set the bounds from these means.
# Follow with 04b_compare_cutoffs.py for the side-by-side images and summary table.
# Then put the chosen value in CUTOFF in 00_config.sh; Step 5 reads it from there.
# =====
# 00_config.sh holds every project-specific value (paths, TRACT, MINLEN, NTHREADS...).
# "source" runs it in this shell so its exports are visible here. $(...) substitutes a

```

```

# command's output; dirname "$0" is the directory this script lives in, so the config is
# found no matter which directory you run from.
source "$(dirname "$0")/00_config.sh"
# Two helpers from 00_config.sh. start_log copies everything printed below into
# $OUT/logs/04_tune_cutoff.sh.log (via tee) while still showing it on screen, so the
# table survives after the terminal is closed. read_subjects fills the SUBJECTS array
# with the IDs in $SUBJECTS_FILE.
start_log "$0"
read_subjects

# Arguments, with defaults. ${1:-default} means "use the first argument if it was given
# and non-empty, otherwise the default". ${SUBJECTS[*]:0:5} slices the array (offset 0,
# length 5) and joins the five IDs into one space-separated string, the same shape as a
# quoted list passed on the command line.
PILOT=${1:-${SUBJECTS[*]:0:5}}
# Order matters downstream: 04b_compare_cutoffs.py takes the first cutoff as the
# conservative reference for its Dice overlap, so list the highest cutoff first.
CUTOFFS=${2:-"0.1 0.08 0.06 0.01"}
# Reduced budgets so the sweep is cheap; everything else matches the Step 5 production
# run (SELECT=2500, SEEDS=25000000 in 00_config.sh). 1000 streamlines is enough to see
# whether a cutoff reaches the target and what the bundle looks like. 5 million seeds
# is the ceiling on attempts, so a cutoff that cannot get there still stops in bounded
# time; tckgen's own default would be 1000 x select, i.e. 1 million here.
PILOT_SELECT=1000
PILOT_SEEDS=5000000

# Header row of the results table. printf with %-12s pads each field to a fixed width,
# left-justified, so the rows line up in the log.
printf "%-12s %-8s %-12s %-12s %-12s\n" Subject Cutoff Selected Generated MeanLen_mm
# $PILOT is deliberately unquoted: the shell splits the string on spaces, giving one loop
# iteration per participant ID.
for s in $PILOT; do
    rois="$OUT/$s/rois"
    tdir="$OUT/$s/tckgen/$TRACT"
    # [ ! -f file ] is true when the file does not exist. The exclusion mask is the last
    # image Step 3 writes, so if it is missing the corridor is not ready; skip this
    # participant ("continue" jumps to the next one) rather than let tckgen fail.
    # The FOD image is not checked here. If it is missing, tckgen fails, the row comes
    # out with blank counts and NA, and the error message is in the log.
    if [ ! -f "$rois/${TRACT}_exclusion_mask.nii.gz" ]; then
        echo "!! $s missing corridor (run Steps 2 and 3)"
        continue
    fi
    # mkdir -p creates parent directories as needed and is silent if it already exists.
    mkdir -p "$tdir"
    for c in $CUTOFFS; do
        # The cutoff goes in the file name, so the pilot runs sit side by side in the same
        # directory Step 5 will use later but never collide with the production tractogram
        # (<TRACT>_<CUTOFF>.tck). 04b_compare_cutoffs.py looks for exactly this name.
        tck="$tdir/${TRACT}_pilot_${c}.tck"
        # Same tckgen options as Step 5 apart from the budget, the cutoff and -quiet.
        # Comments cannot go between the backslash-continued lines, so they are explained here:
        # -algorithm iFOD2      probabilistic tracking on the FOD; the tckgen default,
        #                       spelled out so nobody has to guess
        # -seed_image           start every streamline inside the warped seed region
        # -seed_unidirectional  track away from the seed in one direction only, not both
        # -include              keep a streamline only if it reaches the target region
        # -exclude              discard a streamline the moment it enters this image, which
        #                       is the inverted corridor, i.e. the moment it leaves the
        #                       corridor. This single exclusion region is what makes a
        #                       0.01 cutoff usable at all.
        # -select / -seeds      stop at 1000 accepted streamlines or 5 million seeds,
        #                       whichever comes first

```

```

# -cutoff                the value under test (MRtrix default 0.05)
# -minlength/-maxlength length bounds in mm (35 and 65 for VTA -> hippocampus)
# -stop                  end the streamline on entering the target instead of
#                        letting it run on past it
# -nthreads              threads for this one command (NTHREADS in 00_config.sh)
# -force                 overwrite a .tck left by an earlier sweep; there is no
#                        skip-if-exists here, a rerun simply redoes everything
# -quiet                 suppress the progress output, so the log holds the table
#                        and not twenty progress bars
tckgen "$PROJECT/dwi/$s/wm_fod_norm.mif" "$tck" \
  -algorithm iFOD2 \
  -seed_image "$rois/${TRACT}_seed_diff.nii.gz" -seed_unidirectional \
  -include "$rois/${TRACT}_target_diff.nii.gz" \
  -exclude "$rois/${TRACT}_exclusion_mask.nii.gz" \
  -select "$PILOT_SELECT" -seeds "$PILOT_SEEDS" -cutoff "$c" \
  -minlength "$MINLEN" -maxlength "$MAXLEN" -stop \
  -nthreads "$NTHREADS" -force -quiet
# tckinfo prints the .tck header as "key: value" lines. awk keeps the line whose
# first field ($1) is the key we want and prints the second field, the number.
# count = streamlines written (selected); total_count = streamlines generated,
# including the ones rejected by the include, exclude and length rules.
count=$(tckinfo "$tck" | awk ' $1 == "count:" {print $2} ')
generated=$(tckinfo "$tck" | awk ' $1 == "total_count:" {print $2} ')
# Mean streamline length in mm. tckstats -output mean prints just that number, and
# -quiet keeps progress messages out of the log; awk '{print $1}' takes the first
# field in case anything trails it. ${count:-0} substitutes 0 when count is empty
# (tckinfo failed) so the -gt test does not choke on an empty string. A run that
# selected nothing has no lengths to average, hence NA.
if [ "${count:-0}" -gt 0 ]; then
  meanlen=$(tckstats "$tck" -output mean -quiet | awk '{print $1} ')
else
  meanlen=NA
fi
# One table row per participant and cutoff, same widths as the header.
printf "%-12s %-8s %-12s %-12s %-12s\n" "$s" "$c" "$count" "$generated" "$meanlen"
done
done
# Reaching 1000 everywhere rules cutoffs out but does not pick one: a permissive cutoff
# can also fill the corridor instead of following the tract. 04b renders the density
# images side by side and reports Dice against the most conservative cutoff; the pick is
# the most permissive value that both reaches the target in everyone and matches that
# picture. In the example dataset that was 0.01.
echo "Reaching the streamline target in every pilot participant is necessary but not"
echo "sufficient: compare the reconstructions with 04b_compare_cutoffs.py before choosing."

```

04b_compare_cutoffs.py

```

#!/usr/bin/env python3
"""Step 4b. Comparison of the pilot cutoffs.

```

Companion to 04_tune_cutoff.sh. That script ran reduced-budget tckgen at several FOD amplitude cutoffs in a few pilot participants; this one turns those tractograms into images and a table, so the cutoff for the production run (CUTOFF in 00_config.sh, read by 05_tractography.sh) is chosen by looking at the reconstructions and not by streamline counts alone. Reaching the target at a cutoff is necessary but not sufficient: the permissive reconstruction also has to follow the same path as the conservative one.

Run in a shell where 00_config.sh has been sourced. The script reads PROJECT, OUT, TRACT and SUBJECTS_FILE from the environment; it does not source the file itself:

```
source 00_config.sh
```

```
python 04b_compare_cutoffs.py "sub-01 sub-02 sub-03" "0.1 0.08 0.06 0.01"
```

Both arguments are optional and take the same form as for 04_tune_cutoff.sh: one quoted, space-separated list of participant IDs (default: the first five in \$SUBJECTS_FILE) and one quoted list of cutoffs (default: the four above). Put the most conservative (highest) cutoff first; it is the reference that the Dice overlap is measured against.

Needs, per pilot participant and cutoff:

```
$OUT/<subj>/tckgen/<TRACT>/<TRACT>_pilot_<cutoff>.tck    written by 04_tune_cutoff.sh
$PROJECT/dwi/<subj>/nodif_brain_mask.nii.gz              grid for the density images
$PROJECT/dwi/<subj>/mean_b0.nii.gz                      image background (optional)
```

and tckinfo, tckstats and tckmap (MRtrix3) on the PATH.

Writes, all under \$OUT/qc/<TRACT>_cutoff_pilot/:

```
<subj>_cutoff_compare.png    one column per cutoff; axial row above, coronal row below
cutoff_summary.csv          one row per participant and cutoff: streamlines selected,
                             streamlines generated, occupied voxels, Dice overlap with
                             the reference cutoff, mean and SD of streamline length
cutoff_summary.png          selected, generated, voxels and SD of length averaged
                             over participants, one bar per cutoff (Dice and mean
                             length are in the CSV only)
```

Runs in minutes, not hours: each tractogram gets one tckinfo, two tckstats and one tckmap call on at most 1000 streamlines, plus one small figure per participant (not timed; the page only times the whole sweep). Nothing downstream reads these files. Look first: keep the most permissive cutoff that reached the target in every participant and whose tract-density image still traces the conservative one instead of filling the corridor. Then check the CSV: Dice near 1 means the same voxels, and a jump in "voxels" or in "sd_len_mm" at a permissive cutoff means stray streamlines. Step 6 cleaning removes those outliers; in the example data the cleaned 0.01 bundle came out more compact than the uncleaned 0.06 one. Put the value you choose in 00_config.sh as CUTOFF before Step 5.

```
"""
```

```
import os
import subprocess
import sys
from pathlib import Path
```

```
# Select the file-only backend before pyplot is imported. This usually runs on a cluster
# node with no display, where the default interactive backend fails.
```

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import nibabel as nib
import numpy as np
import pandas as pd
```

```
# Read one variable exported by 00_config.sh and give a clear message when the reader
# forgot to source it. sys.exit() with a string prints it to stderr and exits with 1.
def env(name):
```

```
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]
```

```
# Same PROJECT, OUT and TRACT as the shell scripts, so this finds what 04_tune_cutoff.sh wrote.
PROJECT = Path(env("PROJECT"))
OUT = Path(env("OUT"))
TRACT = env("TRACT")
```

```
# Participants come in as one string ("sub-01 sub-02") that we split on whitespace, so the
# call looks like the call to 04_tune_cutoff.sh. With no argument, take the first five
# IDs in $SUBJECTS_FILE, which is that script's default too.
```

```

if len(sys.argv) > 1:
    PILOT = sys.argv[1].split()
else:
    PILOT = Path(env("SUBJECTS_FILE")).read_text().split()[0:5]
# Cutoffs stay as strings on purpose. They only build file names and label plots, and
# "0.1" has to match the name 04_tune_cutoff.sh used exactly; "0.10" through float() would
# come back as "0.1" and miss the file.
CUTOFFS = sys.argv[2].split() if len(sys.argv) > 2 else ["0.1", "0.08", "0.06", "0.01"]
# The first cutoff given is assumed to be the most conservative (highest). Every other
# cutoff's density mask is compared with it by Dice. Nothing checks the order: list 0.01
# first and the Dice column is measured against 0.01 instead.
REFERENCE = CUTOFFS[0] # most conservative cutoff: reference for Dice overlap

# One QC folder per tract. parents/exist_ok give it "mkdir -p" behaviour.
QC = OUT / "qc" / f"{TRACT}_cutoff_pilot"
QC.mkdir(parents=True, exist_ok=True)

# Run a command given as a list of arguments and hand back its stdout as text.
# check=True raises CalledProcessError on a non-zero exit, so a damaged .tck stops the
# script instead of quietly becoming a zero in the table.
def run(cmd):
    return subprocess.run(cmd, capture_output=True, text=True, check=True).stdout

# Count and length statistics for one tractogram. Argument: path to a .tck file.
# Returns (selected, generated, mean length in mm, SD of length in mm). The two lengths
# are NaN when the file holds no streamlines, since tckstats has nothing to summarise.
def tck_summary(tck):
    """Streamlines selected, streamlines generated (selected plus rejected), and the
    mean and SD of length (mm)."""
    # tckinfo prints the .tck header as "key: value" lines. Collect them in a dict.
    # partition() splits at the first colon only, so a value with a colon in it survives.
    info = {}
    for line in run(["tckinfo", str(tck)]).splitlines():
        key, _, value = line.strip().partition(":")
        info[key] = value.strip()
    # "count" is what tckgen kept (passed -include, -exclude and the length limits);
    # "total_count" is everything it generated to get there. tckgen makes one streamline
    # per seed, so total_count is also the number of seeds spent. A missing key reads 0.
    count = int(info.get("count", 0))
    generated = int(info.get("total_count", 0))
    if count == 0:
        return count, generated, np.nan, np.nan
    # tckstats summarises streamline lengths. "-output mean" (or "std") prints just that
    # one number instead of the whole table, and -quiet keeps progress messages out of the
    # way, so the entire stdout can be cast to float.
    mean = float(run(["tckstats", str(tck), "-output", "mean", "-quiet"]))
    sd = float(run(["tckstats", str(tck), "-output", "std", "-quiet"]))
    return count, generated, mean, sd

# Binary tract-density mask for one tractogram. Arguments: the .tck, a diffusion-space
# image whose voxel grid the mask should use, and a scratch path for the temporary TDI.
# Returns a boolean 3-D array on that grid, True wherever a streamline passed through.
def density_mask(tck, template, scratch):
    """Binary mask of the voxels visited by at least one streamline."""
    # tckmap counts streamlines per voxel (a tract-density image). -template gives the
    # output the same voxel grid and transform as the named image, so it lines up with the
    # brain mask and the mean b0. -force overwrites a scratch file left by an earlier run.
    run(["tckmap", str(tck), str(scratch), "-template", str(template), "-force", "-quiet"])
    # Threshold at zero: we want where the tract is, not how dense it is. A sparse stray
    # branch then counts as much as the core, which is what makes Dice sensitive to strays.

```

```

mask = nib.load(str(scratch)).get_fdata() > 0
# The scratch TDI (named _<subj>_<cutoff>_tdi.nii.gz in the QC folder) is deleted at
# once; only the numbers and the PNGs are kept.
scratch.unlink()
return mask

# Dice overlap of two boolean masks: 2 * |A and B| / (|A| + |B|). 1 means the same voxel
# set, 0 means no shared voxel. NaN when both masks are empty (avoids dividing by zero).
def dice(a, b):
    total = a.sum() + b.sum()
    return 2 * np.logical_and(a, b).sum() / total if total else np.nan

# Draw one 2-D slice: grey background with the tract mask in a fixed colour on top.
# Arguments: a matplotlib axis, the background slice, the boolean mask slice, a title.
def overlay(ax, background, mask, title):
    # imshow runs the first array axis down the page, so a nibabel slice would come out on
    # its side; rot90 turns it the usual way up (second array axis pointing up the page).
    ax.imshow(np.rot90(background), cmap="gray")
    # Masked pixels are drawn transparent, so only the tract colours the background.
    # vmin/vmax fix the colour scale, so the mask (all ones) gets the same colour in every
    # panel rather than whatever autoscaling would pick.
    ax.imshow(np.rot90(np.ma.masked_where(~mask, mask.astype(float))),
              cmap="autumn", alpha=0.8, vmin=0, vmax=1)
    ax.set_title(title, fontsize=10)
    ax.axis("off")

# One dict per participant and cutoff; becomes cutoff_summary.csv at the end.
rows = []
for s in PILOT:
    # The brain mask sets the voxel grid for every tckmap call. The mean b0 is a nicer
    # background but is optional in the project layout, so fall back to the mask itself.
    template = PROJECT / "dwi" / s / "nodif_brain_mask.nii.gz"
    b0 = PROJECT / "dwi" / s / "mean_b0.nii.gz"
    background = nib.load(str(b0 if b0.exists() else template)).get_fdata()

    # Density masks for this participant, keyed by cutoff string, in command-line order.
    masks = {}
    for c in CUTOFFS:
        # Name written by 04_tune_cutoff.sh. A missing tractogram is reported and skipped
        # rather than fatal, so a partial sweep (say, one cutoff not yet run) still plots.
        tck = OUT / s / "tckgen" / TRACT / f"{TRACT}_pilot_{c}.tck"
        if not tck.exists():
            print(f"[{s}] cutoff {c}: no tractogram (run 04_tune_cutoff.sh)")
            continue
        count, generated, mean, sd = tck_summary(tck)
        masks[c] = density_mask(tck, template, QC / f"_ {s} _ {c} _tdi.nii.gz")
        # "voxels" is the TDI footprint. A permissive cutoff that fills the corridor
        # instead of following the tract shows up here as a jump over the other cutoffs.
        rows.append({
            "subject": s, "cutoff": c, "selected": count, "generated": generated,
            "voxels": int(masks[c].sum()), "mean_len_mm": mean, "sd_len_mm": sd,
        })
    # No tractogram at any cutoff: nothing to draw for this participant.
    if not masks:
        continue

# Dice overlap of every cutoff with the reference cutoff
# rows holds every participant so far, hence the filter on the participant ID. If the
# reference tractogram is missing for this participant the key is never set and pandas
# leaves the CSV cell empty. The reference against itself scores 1 by construction.

```

```

for row in rows:
    if row["subject"] == s and REFERENCE in masks:
        row["dice_vs_reference"] = round(dice(masks[REFERENCE], masks[row["cutoff"]]), 3)

# Slices through the densest part of the union of all cutoffs
# The same slice is used for every column, so the columns can be compared by eye.
# Summing over axes (0, 1) leaves one voxel count per axial slice (third axis); over
# (0, 2) one per coronal slice (second axis).  argmax picks the fullest of each.
union = np.logical_or.reduce(list(masks.values()))
k_axial = int(np.argmax(union.sum(axis=(0, 1))))
k_coronal = int(np.argmax(union.sum(axis=(0, 2))))

# Two rows (axial, coronal) by one column per cutoff that had a tractogram; the width
# grows with the number of columns.  squeeze=False keeps axes a 2-D array even when
# there is a single cutoff, so axes[0][j] below works in every case.
fig, axes = plt.subplots(2, len(masks), figsize=(3.2 * len(masks), 6.4), squeeze=False)
for j, (c, mask) in enumerate(masks.items()):
    overlay(axes[0][j], background[:, :, k_axial], mask[:, :, k_axial],
            f"Axial, cutoff {c}")
    overlay(axes[1][j], background[:, k_coronal, :], mask[:, k_coronal, :],
            f"Coronal, cutoff {c}")
# "uncleaned" in the title is deliberate: these are raw tckgen outputs.  Much of the
# extra thickness at a permissive cutoff is what Step 6 removes, so judge the path,
# not the width.  bbox_inches="tight" trims the white margin around the figure.
fig.suptitle(f"{s} {TRACT} pilot cutoffs (uncleaned)", fontsize=11)
fig.tight_layout()
fig.savefig(QC / f"{s}_cutoff_compare.png", dpi=110, bbox_inches="tight")
# Close each figure explicitly; otherwise matplotlib keeps them all in memory.
plt.close(fig)
print(f"{s} wrote {s}_cutoff_compare.png")

# Every pilot participant was skipped.  Usually 04_tune_cutoff.sh has not been run yet, or
# TRACT in 00_config.sh no longer names the tract it was run for.
if not rows:
    sys.exit("no pilot tractograms found")

# The per-participant table.  Cutoffs are written as typed ("0.1", not 0.1).
summary = pd.DataFrame(rows)
summary.to_csv(QC / "cutoff_summary.csv", index=False)

# Mean across pilot participants, one bar per cutoff
# Four panels as (CSV column, panel title).  Dice is not charted; read it from the CSV.
panels = [("selected", "Streamlines selected"), ("generated", "Streamlines generated"),
          ("voxels", "Occupied voxels"), ("sd_len_mm", "SD of length (mm)")]
# sort=False keeps the cutoffs in command-line order; sorted as strings, "0.1" would land
# after "0.08".  mean() skips the NaN lengths of any empty tractogram.
means = summary.groupby("cutoff", sort=False)[[k for k, _ in panels]].mean()
fig, axes = plt.subplots(1, len(panels), figsize=(15, 3.6))
for ax, (key, title) in zip(axes, panels):
    ax.bar(means.index.astype(str), means[key], color="#3A6B8C")
    ax.set_title(title)
    ax.set_xlabel("FOD amplitude cutoff")
fig.tight_layout()
fig.savefig(QC / "cutoff_summary.png", dpi=110)
plt.close(fig)

# Echo the whole table so it lands in the terminal and in any log the run is tee'd into.
print(summary.to_string(index=False))
print(f"\nsummary -> {QC / 'cutoff_summary.csv'} and cutoff_summary.png")

```

05_tractography.sh

```
#!/bin/bash
# =====
# Step 5. Corridor-constrained tractography for every participant
# =====
# Seeds in the seed region, requires the target region, and discards any
# streamline that leaves the corridor (single exclusion mask from Step 3).
# Participants are processed serially; tckgen uses $NTHREADS threads.
# Writes <tract>_<cutoff>.tck per participant and a summary of streamlines
# selected and streamlines generated (selected plus rejected). The streamline
# count and length used as model covariates are taken from the cleaned bundles
# in Step 6, not from this step.
#
# This is the production version of the tckgen call piloted in Step 4: the full budget
# (SELECT streamlines out of at most SEEDS attempts) and the cutoff you settled on there.
# Nothing in this file is tract-specific; every value comes from 00_config.sh.
#
# Needs, per participant:
# $PROJECT/dwi/<s>/wm_fod_norm.mif           normalized WM FOD (Step 0b)
# $OUT/<s>/rois/<TRACT>_seed_diff.nii.gz      seed region in diffusion space (Step 2)
# $OUT/<s>/rois/<TRACT>_target_diff.nii.gz    target region in diffusion space (Step 2)
# $OUT/<s>/rois/<TRACT>_exclusion_mask.nii.gz inverted corridor (Step 3)
# Writes:
# $OUT/<s>/tckgen/<TRACT>/<TRACT>_<CUTOFF>.tck raw tractogram; Step 6 cleans it
# $OUT/qc/<TRACT>_tckgen_summary.csv          one row per participant, for you to
#                                             read; no later step uses it
# $OUT/logs/05_tractography.sh.log          everything printed, kept on disk
# Usage:
# bash 05_tractography.sh                   skips participants whose .tck already exists
# FORCE=1 bash 05_tractography.sh           redoes everyone, e.g. after rebuilding the corridor
# Changing CUTOFF changes the file name, so a new cutoff is never skipped as "exists" and
# the old tractogram stays on disk next to it.
# Runtime: about 2 min per participant at 3 T with eight threads, so roughly 2 hr for 57
# participants. The other hemisphere is a second run of this script with TRACT changed.
# What to look for. The last lines say how many participants reached SELECT. In the
# example dataset that was everyone, for both hemispheres and both tracts. A SHORT line
# means the participant stopped below SELECT; 1,800 is kept as is, something like 50
# gets a look in Step 7 and usually means the cutoff is too high for the data, the
# corridor is too narrow, or the seed or target sits outside it (Troubleshooting page).
# Generated at the SEEDS ceiling with Selected below SELECT means the budget ran out.
# MISSING lines are participants with no tractogram; the reason is earlier in the log.
# =====
# 00_config.sh holds every project-specific value. "source" runs it in this shell so
# its exports (PROJECT, OUT, TRACT, CUTOFF, SELECT...) are visible here. $(...) is
# command substitution; dirname "$0" is the directory this script sits in, so the config
# is found no matter which directory you run from.
source "$(dirname "$0")/00_config.sh"
# Two helpers from 00_config.sh. start_log copies everything printed below into
# $OUT/logs/05_tractography.sh.log (tee, with stderr folded in by 2>&1) while still
# showing it on screen; tckgen's own messages and the SHORT/MISSING lines end up
# there. read_subjects fills the array SUBJECTS with the IDs in $SUBJECTS_FILE.
start_log "$0"
read_subjects

# Main loop, one participant at a time. "${SUBJECTS[@]}" expands the array to one word
# per ID with the quotes intact. Serial: tckgen already uses NTHREADS cores for one
# participant, so this step peaks at NTHREADS (see the load note in 00_config.sh).
for s in "${SUBJECTS[@]}"; do
    rois="$OUT/$s/rois"
    tdir="$OUT/$s/tckgen/$TRACT"
    # The cutoff is part of the file name; Step 6 looks for exactly this name.
```

```

tck="$tdir/${TRACT}_${CUTOFF}.tck"
# Input checks. [ ! -f file ] is true when the file is absent; "continue" jumps to the
# next participant instead of letting tckgen die with a less helpful message. The
# exclusion mask is the last image Step 3 writes, so if it is there the corridor is
# complete, and Step 3 needed the seed and target to build it, so those are not
# checked separately.
if [ ! -f "$PROJECT/dwi/$s/wm_fod_norm.mif" ]; then
    echo "!! $s missing FOD image (run 00b)"
    continue
fi
if [ ! -f "$rois/${TRACT}_exclusion_mask.nii.gz" ]; then
    echo "!! $s missing corridor (run Steps 2 and 3)"
    continue
fi
# Skip-if-exists, so a run that was interrupted picks up where it left off. FORCE
# comes from 00_config.sh (default 0); FORCE=1 on the command line redoes everyone.
if [ "$FORCE" != 1 ] && [ -f "$tck" ]; then
    echo "== $s tractogram exists"
    continue
fi
# mkdir -p makes the parent directories too and is silent if they already exist.
mkdir -p "$tdir"
# tckgen creates its output file at start-up; writing to a temporary name keeps an
# interrupted run from being mistaken for a finished tractogram on restart.
partial="$tdir/${TRACT}_${CUTOFF}.partial.tck"
# "if command; then" branches on the command's exit status (0 = success), so a tckgen
# error lands in the else branch below. Comments cannot sit between the
# backslash-continued lines, so the options are explained here:
#   -algorithm iFOD2      probabilistic tracking with second-order integration over
#                           the FOD; the tckgen default, spelled out anyway
#   -seed_image           every streamline starts inside the warped seed region
#   -seed_unidirectional  track away from the seed in one direction only, not both
#   -include              keep a streamline only if it reaches the target region
#   -exclude              discard a streamline the moment it enters this image. The
#                           image is the inverted corridor, so this fires the moment a
#                           streamline leaves the corridor. It is the only exclusion
#                           region, and it is what makes a 0.01 cutoff usable at all.
#   -select 2500          stop once this many streamlines have passed every rule.
#                           Enough for stable profiles and endpoint maps; more just costs
#                           time. Not a connectivity measure: everyone who reaches the
#                           target gets the same count, which is why the covariate count
#                           comes from the cleaned bundle in Step 6 instead.
#   -seeds 25000000       ceiling on seeding attempts; tckgen gives up here even if
#                           SELECT was not reached. The default (1000 x select) was
#                           sometimes not enough to reach 2500.
#   -cutoff 0.01          minimum FOD amplitude at which tracking continues (MRtrix
#                           default 0.05). Chosen in Step 4; only safe this low because
#                           the corridor keeps streamlines from wandering. In the Step 4
#                           pilot, higher values burned many more seeds and failed to
#                           reach the target in some participants.
#   -minlength/-maxlength length bounds in mm. 35 and 65 were set for VTA ->
#                           hippocampus and held at 3 T (mean lengths 44 to 47 mm); for
#                           another tract family set them from the Step 4 pilot means.
#                           Streamlines outside the window are rejected outright, so a
#                           window that is too tight throws away real ones.
#   -stop                 end the streamline on entering the target instead of letting
#                           it run on past it, so the bundle ends where the target starts
#   -nthreads             threads for this one command (NTHREADS in 00_config.sh)
#   -force                overwrite the output if it exists, e.g. a .partial.tck left by
#                           a run that was killed before the rm -f below could run
# No -fslgrad: the gradient table is already inside the .mif from Step 0b. No -quiet
# either, so tckgen's own messages and warnings land in the log via start_log.
if tckgen "$PROJECT/dwi/$s/wm_fod_norm.mif" "$partial" \

```

```

-algorithm iFOD2 \
-seed_image "$rois/${TRACT}_seed_diff.nii.gz" -seed_unidirectional \
-include "$rois/${TRACT}_target_diff.nii.gz" \
-exclude "$rois/${TRACT}_exclusion_mask.nii.gz" \
-select "$SELECT" -seeds "$SEEDS" -cutoff "$CUTOFF" \
-minlength "$MINLEN" -maxlength "$MAXLEN" -stop \
-nthreads "$NTHREADS" -force; then
# Success: rename to the final name in one step. mv -f replaces an existing target
# without asking, which a FORCE=1 rerun needs.
mv -f "$partial" "$tck"
echo ">> $s finished"
else
# Failure: drop the partial so the next run redoes this participant instead of
# skipping it. rm -f does not complain if the file is already gone.
rm -f "$partial"
echo "!! $s tckgen failed or was interrupted"
fi
done

# Audit and summary table (rebuilt from the tractograms on every run)
# Loops over everyone again, including participants skipped above as "exists", so the
# CSV always describes what is on disk and not just what this run produced.
mkdir -p "$OUT/qc"
stats="$OUT/qc/${TRACT}_tckgen_summary.csv"
# ">" truncates the file and writes the header row; the rows below use ">>" to append.
# Columns: participant ID, streamlines selected, mean length in mm, streamlines
# generated (selected plus rejected, which is also the number of seeds consumed).
echo "Subject,Selected,MeanLength_mm,Generated" > "$stats"
# Tallies for the closing line. $((...)) is shell integer arithmetic.
reached=0
total=0
for s in "${SUBJECTS[@]"; do
tck="$OUT/$s/tckgen/${TRACT}/${TRACT}_${CUTOFF}.tck"
total=$((total + 1))
# No tractogram means an input was missing or tckgen failed above; the reason is
# earlier in the log. These participants get no CSV row.
if [ ! -f "$tck" ]; then
echo "MISSING: $s"
continue
fi
# tckinfo prints the .tck header as "key: value" lines. awk keeps the line whose
# first field ($1) is the key we want and prints the second field, the number.
# count = streamlines written (selected); total_count = streamlines generated,
# including every one that was rejected.
count=$(tckinfo "$tck" | awk ' $1 == "count:" {print $2}')
generated=$(tckinfo "$tck" | awk ' $1 == "total_count:" {print $2}')
# Mean streamline length in mm. tckstats -output mean prints only the mean; -quiet
# keeps its own messages out of the log. awk '{print $1}' takes the first field on the
# line in case anything trails it. ${count:-0} substitutes 0 when count is empty
# (tckinfo failed) so -gt does not choke on an empty string. An empty tractogram has
# nothing to average, hence NA.
if [ "${count:-0}" -gt 0 ]; then
meanlen=$(tckstats "$tck" -output mean -quiet | awk '{print $1}')
else
meanlen=NA
fi
echo "$s,$count,$meanlen,$generated" >> "$stats"
# Did this participant reach SELECT? -ge is integer >=. Anyone short is still kept
# (count is not connectivity); the SHORT line is so you know whom to look at in
# Step 7. Generated near SEEDS with Selected well below SELECT means the seed budget
# ran out before 2500 streamlines made it through.
if [ "${count:-0}" -ge "$SELECT" ]; then
reached=$((reached + 1))

```

```

else
    echo "SHORT: $s ($count selected of $generated generated)"
fi
done
# The closing tally. In the example dataset every participant reached 2,500 in both
# hemispheres, with Generated between 0.5 and 2 million (about 4% of SEEDS) for the
# posterior tract and 1.5 to 11 million for the anterior one, whose corridor is smaller.
# Next: python 06_clean_bundles.py reads the .tck files written above.
echo "reached $SELECT streamlines: $reached / $total"
echo "summary -> $stats"

```

06_clean_bundles.py

```

#!/usr/bin/env python3
"""Step 6. Bundle cleaning (pyAFQ) and the cleaned-bundle covariate table.

```

What it does

Takes the raw corridor tractogram that Step 5 wrote for each participant, drops the streamlines that stray from the bundle core or run unusually long, and saves what is left as the cleaned bundle. It then rebuilds one CSV holding the streamline count and mean length of every cleaned bundle; those two numbers are the reconstruction-quality covariates in the Step 9 models.

Run in a shell where 00_config.sh has been sourced:

```

source 00_config.sh
python 06_clean_bundles.py

```

Needs pyAFQ and DIPY importable in that Python (see the Step 6 page for the zipp hiccup on older systems). Participants run one after another. No timing was recorded, but there is no MRtrix call here, only pyAFQ on 2,500 streamlines, so expect minutes for a full sample, not the hours Step 5 takes. A participant whose cleaned bundle already exists is skipped; FORCE=1 in the shell redoes them. The thresholds live in this file, not 00_config.sh.

Inputs (must exist before running)

\$OUT/<subj>/tckgen/<TRACT>/<TRACT>_<CUTOFF>.tck	raw tractogram from Step 5
\$PROJECT/dwi/<subj>/nodif_brain_mask.nii.gz	reference grid for loading the .tck
\$SUBJECTS_FILE	one participant ID per line

Each tractogram from Step 5 is cleaned with AFQ.recognition.cleaning.clean_bundle: streamlines are resampled to 100 points, and those farther than DISTANCE_SD from the bundle core (Mahalanobis distance), or longer than the mean length by more than LENGTH_SD, are removed, for up to CLEAN_ROUNDS iterations.

Some tracts reconstruct as two distinct bundles in a subset of participants. With QB_SPLIT = True the tractogram is first divided with QuickBundles, the two largest clusters are cleaned separately, and the anatomically correct cluster is chosen by inspection and copied to <TRACT>_<CUTOFF>_cleaned.tck. Rerun the script afterwards to rebuild the covariate table.

Outputs

```

$OUT/<subj>/tckgen/<TRACT>/<TRACT>_<CUTOFF>_cleaned.tck
    Step 7 draws this as a tract-density overlay; Step 8 profiles FA/NODDI along it.
$OUT/nodewise/<TRACT>_tract_stats.csv
    Subject, Streamline_count, Mean_length_mm (cleaned bundle; model covariates),
    Count_uncleaned, Retention_pct
    Step 8b merges Streamline_count and Mean_length_mm into the analysis file that
    the Step 9 models read (they are listed in COVARIATES in 00_config.sh).

```

How to tell it worked

```

One "<raw> -> <clean> streamlines (xx% retained)" line per participant, then
"N of N participants have a cleaned bundle". On the example dataset retention was
26-58%; anything under 15% or over 80% is worth opening in a viewer. A SKIP line
is a participant Step 5 never finished (or whose tractogram is empty), and the
final count should equal the participant total minus the SKIP lines.
"""
import inspect # used once, to ask the installed clean_bundle whether it takes core_only
import os
import sys
from pathlib import Path

import pandas as pd
# clean_bundle moved between pyAFQ releases. Try the current location first and fall
# back to the old one, so the same script runs against either install.
try:
    from AFQ.recognition.cleaning import clean_bundle
except ImportError:
    # pyAFQ 1.3.2 and earlier
    from AFQ.segmentation import clean_bundle
# DIPY handles the file I/O and the geometry. A StatefulTractogram is a set of
# streamlines plus the reference image grid they sit in; that is what lets DIPY move
# between mm and voxel coordinates without guessing.
from dipy.io.stateful_tractogram import StatefulTractogram
from dipy.io.streamline import load_tractogram, save_tractogram
# These three are only used on the QB_SPLIT = True path.
from dipy.segment.clustering import QuickBundles
from dipy.segment.featurespeed import ResampleFeature
from dipy.segment.metricspeed import AveragePointwiseEuclideanMetric
from dipy.tracking.streamline import length # per-streamline length, for the table

# Fetch one variable exported by 00_config.sh, or stop with a message that says what
# to do. Python cannot source the config itself; it relies on the shell having
# exported everything, and a missing variable almost always means the source step
# was skipped or the config was edited without being sourced again.
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]

# Project settings from 00_config.sh. Paths become Path objects so the "/" joins
# below read like the shell scripts.
PROJECT = Path(env("PROJECT"))
OUT = Path(env("OUT"))
TRACT = env("TRACT")
CUTOFF = env("CUTOFF") # stays a string; it is only ever used in file names
# FORCE=1 in the shell means redo participants whose cleaned bundle already exists.
# Anything other than the exact string "1" counts as off.
FORCE = os.environ.get("FORCE", "0") == "1"
# str.split() with no argument splits on any run of whitespace, so this copes with
# CRLF line endings, blank lines and a missing final newline, the same as the
# read_subjects helper in 00_config.sh does for the shell scripts.
SUBJECTS = Path(env("SUBJECTS_FILE")).read_text().split()

# Cleaning settings (Table 1 on the Step 6 page). They are not in 00_config.sh; if you
# change one, change it here and note it in the manifest.
#
# N_POINTS: each streamline is resampled to this many points before the Mahalanobis
# distance is computed, so the bundle core is a mean position at each of 100 nodes.
# It matches the 100 nodes profiled in Step 8, which is convenient but not required.
N_POINTS = 100 # resampling for the Mahalanobis distance
# CLEAN_ROUNDS: cleaning is iterative because removing outliers moves the mean and
# tightens the SD, which can expose new outliers. pyAFQ stops early once a round

```

```

# removes nothing, so this is a ceiling, not a fixed number of passes.
CLEAN_ROUNDS = 5          # maximum cleaning iterations
# DISTANCE_SD: a streamline whose Mahalanobis distance from the core exceeds this many
# SD is dropped. Lower is stricter (fewer, tighter streamlines); higher keeps more of
# the raw tract, outliers included. The Table 1 set as a whole gave 26-58% retention.
DISTANCE_SD = 3          # Mahalanobis distance threshold, SD
# LENGTH_SD: drop streamlines longer than the mean by more than this many SD. It is
# one-sided (long only): the long ones are typically loops or detours on the way to
# the target. Short streamlines were already cut by -minlength in Step 5.
LENGTH_SD = 2            # length threshold, SD above the mean
# QB_SPLIT: leave False unless the raw tractograms clearly hold two separate bundles
# (hippocampus -> accumbens does this often; VTA -> hippocampus did not need it in
# the example data). Mahalanobis cleaning of a two-bundle mixture keeps the wrong one
# or a blend, so the split has to come first. See "Tracts with two bundles" on the page.
QB_SPLIT = False          # True: split into two bundles before cleaning
# QB_THRESHOLD_MM: a streamline joins an existing cluster when its distance to that
# cluster's centroid is under this; otherwise it starts a new one. Smaller means more,
# finer clusters. 5 mm with 100-point resampling is the setting used in atlas construction.
QB_THRESHOLD_MM = 5.0     # QuickBundles distance threshold

# Keyword arguments handed to clean_bundle on every call. stat="mean" is the centroid
# statistic: the core is the mean position at each node (Table 1 on the page).
CLEAN_ARGS = dict(n_points=N_POINTS, clean_rounds=CLEAN_ROUNDS, stat="mean",
                  distance_threshold=DISTANCE_SD, length_threshold=LENGTH_SD)
# pyAFQ 3 and later compute the distance over the middle 60% of each streamline by
# default; core_only=0 restores the full-length computation of earlier versions.
# The argument is only passed when the installed clean_bundle knows it; an older
# pyAFQ would otherwise choke on the unknown keyword. inspect.signature reads the real
# signature at run time, so this works whichever import succeeded above.
if "core_only" in inspect.signature(clean_bundle).parameters:
    CLEAN_ARGS["core_only"] = 0

# Run clean_bundle on one StatefulTractogram and hand back the cleaned bundle as a new
# StatefulTractogram in the same space as the input. return_idx=True makes pyAFQ
# return the indices of the streamlines it kept alongside its own cleaned tractogram;
# we discard the latter (the "_") and rebuild from the indices ourselves.
def clean(sft):
    # The cleaned bundle is rebuilt from the retained indices, so that its coordinate
    # space is that of the input under every pyAFQ version.
    # (We do not rely on which space the installed pyAFQ hands its own copy back in; a
    # .tck saved in the wrong space lands in the wrong place when Step 7 overlays it.)
    _, keep = clean_bundle(sft, return_idx=True, **CLEAN_ARGS)
    # from_sft copies the reference grid, space and origin of the input onto the subset.
    return StatefulTractogram.from_sft(sft.streamlines[keep], sft)

# Load a .tck with the participant's brain mask as the reference image. A .tck carries
# streamline coordinates but no image grid, so DIPY needs a NIfTI from the same
# diffusion space to attach an affine and dimensions. Any image on that grid would
# do; the mask is the one every participant is required to have (00_config.sh layout).
# bbox_valid_check=False stops DIPY refusing the file if any point maps to a voxel
# coordinate below 0 or past the last voxel. We would rather load a streamline that
# grazes the volume edge than have the run stop on it.
def load(tck, reference):
    return load_tractogram(str(tck), str(reference), bbox_valid_check=False)

# Pass 1: clean every participant that still needs it.
for s in SUBJECTS:
    # Same layout Step 5 used: one tckgen/<TRACT> folder per participant.
    tdir = OUT / s / "tckgen" / TRACT

```

```

raw = tdir / f"{TRACT}_{CUTOFF}.tck"
out = tdir / f"{TRACT}_{CUTOFF}_cleaned.tck"
reference = PROJECT / "dwi" / s / "nodif_brain_mask.nii.gz"

# Resume logic. No raw tractogram means Step 5 did not finish this participant
# (Step 5 writes to a temporary name and renames at the end, so a half-written
# file never sits at this path). An existing cleaned bundle is skipped unless
# FORCE=1, which is what makes an interrupted run restartable with the same command.
if not raw.exists():
    print(f"[{s}] SKIP: no tractogram from Step 5")
    continue
if out.exists() and not FORCE:
    print(f"[{s}] cleaned bundle exists")
    continue
sft = load(raw, reference)
n_raw = len(sft.streamlines)
# Zero streamlines means Step 5 ran but nothing passed every rule (its summary lists
# that participant with count 0 and length NA). Nothing to clean; skip, and the
# table at the end will simply have no row for them.
if n_raw == 0:
    print(f"[{s}] SKIP: 0 streamlines")
    continue

# Usual path: clean the whole tractogram as one bundle and write it out.
# bbox_valid_check=False on save for the same reason as on load.
if not QB_SPLIT:
    cleaned = clean(sft)
    save_tractogram(cleaned, str(out), bbox_valid_check=False)
    n_clean = len(cleaned.streamlines)
    # This retention line is what you scan after the run. 26-58% on the example
    # data; under 15% or over 80% means open this participant in a viewer.
    print(f"[{s}] {n_raw} -> {n_clean} streamlines ({100 * n_clean / n_raw:.0f}% retained)")
    continue

# Two-bundle path (QB_SPLIT = True). QuickBundles walks the streamlines once,
# assigning each to the nearest cluster within QB_THRESHOLD_MM of its centroid or
# starting a new cluster. The metric is the average distance between matching
# points after both streamlines are resampled to N_POINTS. The metric itself does
# not try the flipped order, but QuickBundles does before assigning, so a bundle
# whose streamlines are stored in mixed directions still lands in one cluster
# (checked on a synthetic bundle with half of it reversed; Step 8 relies on this too).
metric = AveragePointwiseEuclideanMetric(ResampleFeature(nb_points=N_POINTS))
clusters = QuickBundles(threshold=QB_THRESHOLD_MM, metric=metric).cluster(sft.streamlines)
# Keep the two clusters with the most streamlines (len(cluster) is that count);
# anything smaller is treated as stray and ignored.
largest = sorted(clusters, key=len, reverse=True)[:2]
# Clean each of the two on its own and save as _cluster1_ (the bigger one) and
# _cluster2_. cluster.indices are positions in the original tractogram, so from_sft
# carries the space over exactly as clean() does.
for i, cluster in enumerate(largest, start=1):
    part = StatefulTractogram.from_sft(sft.streamlines[cluster.indices], sft)
    cleaned = clean(part)
    name = f"{TRACT}_{CUTOFF}_cluster{i}_cleaned.tck"
    save_tractogram(cleaned, str(tdir / name), bbox_valid_check=False)
    print(f"[{s}] cluster {i}: {len(cluster)} -> {len(cleaned.streamlines)} streamlines")
# Nothing is written to the _cleaned.tck name in this mode. Look at both clusters
# in a viewer (the Step 7 page has the mrview -tractography.load and fsleyes lines),
# copy the anatomically right one to that name, note the choice in the manifest, and
# rerun this script for the table.
print(f"[{s}] inspect both clusters; copy the correct one to {out.name}")

# Covariate table from the cleaned bundles (rebuilt on every run)
# Pass 2 is a separate loop rather than rows collected in pass 1 so that the table

```

```

# also covers participants skipped above as already done, and picks up any
# _cleaned.tck copied in by hand after a QB split.
rows = []
for s in SUBJECTS:
    tdir = OUT / s / "tckgen" / TRACT
    raw = tdir / f"{TRACT}_{CUTOFF}.tck"
    out = tdir / f"{TRACT}_{CUTOFF}_cleaned.tck"
    # No cleaned bundle, no row. Step 8b merges on Subject, so that participant simply
    # drops out of the analysis file rather than appearing with blanks.
    if not out.exists():
        continue
    reference = PROJECT / "dwi" / s / "nodif_brain_mask.nii.gz"
    # length() returns one value per streamline, in mm because load_tractogram puts
    # streamlines in RAS mm space by default. The raw file is reloaded only to count
    # it; NaN if it has been deleted since.
    lengths = length(load(out, reference).streamlines)
    n_raw = len(load(raw, reference).streamlines) if raw.exists() else float("nan")
    rows.append({
        "Subject": s,
        # Streamline_count and Mean_length_mm are the two reconstruction-quality
        # covariates in the Step 9 models (COVARIATES in 00_config.sh). Both index how
        # well the tract reconstructed and vary across participants, so the models
        # adjust for them rather than analyse them.
        "Streamline_count": len(lengths),
        "Mean_length_mm": round(float(lengths.mean()), 3) if len(lengths) else float("nan"),
        # QC columns only; neither enters a model. Retention_pct is the ratio pass 1
        # printed, to one decimal, kept here so it survives in a file.
        "Count_uncleaned": n_raw,
        "Retention_pct": round(100 * len(lengths) / n_raw, 1) if n_raw else float("nan"),
    })

# Write the table next to the other node-wise inputs. mkdir(parents=True,
# exist_ok=True) is the Python spelling of mkdir -p. index=False keeps pandas from
# writing its row numbers as an extra unnamed first column.
table = OUT / "nodewise" / f"{TRACT}_tract_stats.csv"
table.parent.mkdir(parents=True, exist_ok=True)
pd.DataFrame(rows).to_csv(table, index=False)
# The last two lines are the summary to check: the count should equal the participant
# total minus the SKIP lines above, and the path is what Step 8b reads next.
print(f"\n{len(rows)} of {len(SUBJECTS)} participants have a cleaned bundle")
print(f"covariate table -> {table}")

```

07_visual_qc.py

```

#!/usr/bin/env python3
"""Step 7. Tract-density overlays of every cleaned bundle, with automatic flags.

```

Run in a shell where 00_config.sh has been sourced:

```

source 00_config.sh
python 07_visual_qc.py

```

For each participant the cleaned bundle is converted to a tract-density image (tckmap) and drawn over the mean $b = 0$ image in the axial and coronal slices that contain the most streamlines. Flags mark bundles that need inspection in a viewer; they do not replace review of the unflagged images.

Needs, per participant:

\$OUT/<subj>/tckgen/<TRACT>/<TRACT>_<CUTOFF>_cleaned.tck	written by Step 6
\$PROJECT/dwi/<subj>/nodif_brain_mask.nii.gz	grid for the TDI
\$PROJECT/dwi/<subj>/mean_b0.nii.gz	background (optional)

MRtrix3's tckmap must be on PATH. Everything else is Python: nibabel, numpy, pandas and matplotlib.

Outputs: \$OUT/qc/<TRACT>/<subj>_<TRACT>_qc.png and qc_flags.csv

Expect a few seconds per participant (one tckmap call on a small bundle plus one small figure), so a full sample takes minutes and needs no tmux. As it runs, one line per participant prints the voxel count, the peak density and any flag; the last lines give "N of M participants flagged" and the flagged rows. Zero flags is the expected result on clean data. Open the PNGs anyway: the flags only catch gross failures (empty, tiny or runaway bundles), not a bundle of normal size in the wrong place. What a good bundle looks like is described under "Criteria" on the Step 7 page.

Rerunning: the flags and qc_flags.csv are rebuilt on every run, but an existing PNG is left alone unless FORCE=1 (or you delete it), so a rerun after fixing one participant is cheap. Nothing later in the pipeline reads qc_flags.csv. A participant you decide to drop has to be taken out of \$SUBJECTS_FILE (or of \$COVARIATES_CSV, which Step 8b joins on) by hand: Step 8 profiles everyone in the list who has a cleaned bundle, and Step 9 models whoever survives that join.

```
"""
import os
import subprocess
import sys
from pathlib import Path

# Choose the file-only backend before pyplot is imported. With the default backend
# matplotlib tries to open a window, which fails on a cluster node or over ssh with
# no display.
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import nibabel as nib
import numpy as np
import pandas as pd

# Read one variable exported by 00_config.sh, or stop with a message that says what
# to do. sys.exit() given a string prints it to stderr and exits with status 1, so a
# forgotten `source 00_config.sh` fails loudly instead of producing paths like "/qc".
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]

# Settings from 00_config.sh. CUTOFF stays a string because it is only used to build
# the file name Steps 5 and 6 used (e.g. l_vta_l_hipp_0.01_cleaned.tck).
PROJECT = Path(env("PROJECT"))
OUT = Path(env("OUT"))
TRACT = env("TRACT")
CUTOFF = env("CUTOFF")
# FORCE is optional (default "0"); only the exact string "1" makes us redraw images.
FORCE = os.environ.get("FORCE", "0") == "1"
# split() with no argument breaks on any whitespace, so CRLF line endings, blank lines
# and a missing final newline in subjects.txt are all harmless (same tolerance as
# read_subjects in the config).
SUBJECTS = Path(env("SUBJECTS_FILE")).read_text().split()

# Flag thresholds (Table 1 on the Step 7 page; also listed under Reference:
# Parameters). They catch gross failures only. In the example dataset the whole
# corridor was under 2,000 voxels (Step 3 page), and tckgen -exclude keeps every
# streamline inside it, so a bundle occupying 5,000 voxels means the corridor or the
```

```

# exclusion mask went wrong; check that participant's corridor size in the Step 3 log
# and the warped regions from Step 2. Fewer than 50 means almost nothing survived
# tractography or cleaning. All of a participant's streamlines run through one narrow
# corridor, so the busiest voxel should hold far more than 5 of them; a peak below 5
# means they are spread thin or very few are left. All three counts depend on voxel
# size and on DILATE_VOX, so revisit them if your data or corridor differ much from the
# example's.
MIN_VOXELS = 50          # fewer occupied voxels: LOW_COVERAGE
MAX_VOXELS = 5000        # more occupied voxels: HIGH_COVERAGE (bundle leaves the pathway)
MIN_PEAK_DENSITY = 5     # peak streamlines per voxel below this: LOW_DENSITY

# One QC folder per tract. parents=True also creates $OUT/qc the first time;
# exist_ok=True keeps reruns quiet.
QC = OUT / "qc" / TRACT
QC.mkdir(parents=True, exist_ok=True)

# Turn the two numbers measured from the TDI into one flag string.
# n_voxels number of voxels with at least one streamline
# peak largest number of streamlines through any single voxel
# The checks run in order and the first hit wins, so an empty bundle reports
# LOW_COVERAGE rather than LOW_DENSITY. Returns "" for a bundle that passes; the CSV
# and the summary at the end both rely on that empty string.
def flag_for(n_voxels, peak):
    if n_voxels < MIN_VOXELS:
        return "LOW_COVERAGE"
    if n_voxels > MAX_VOXELS:
        return "HIGH_COVERAGE"
    if peak < MIN_PEAK_DENSITY:
        return "LOW_DENSITY"
    return ""

# Draw one panel: the b0 slice in grey with the density slice on top in "hot".
# ax matplotlib axes to draw into
# background 2-D slice of the b0 (or mask) image
# density 2-D slice of the TDI, same grid
# title panel label ("Axial" or "Coronal")
# imshow alone would run the first array axis down the page; np.rot90 turns the slice so
# its second axis runs up the page, which with the usual storage order (x left-right,
# y front-back, z bottom-top) puts anterior at the top of the axial panel and superior
# at the top of the coronal one. Left-right is whatever order the file is stored in,
# so do not read hemisphere off these panels; use a viewer for that. Voxels with no
# streamlines are masked rather than drawn as black so the anatomy shows through, and
# alpha=0.7 keeps the b0 visible under the bundle. axis("off") drops the ticks and
# frame; voxel indices mean nothing here.
def overlay(ax, background, density, title):
    ax.imshow(np.rot90(background), cmap="gray")
    ax.imshow(np.rot90(np.ma.masked_where(density <= 0, density)), cmap="hot", alpha=0.7)
    ax.set_title(title)
    ax.axis("off")

# Main loop. Every participant gets a row in rows whatever happens to them, so
# qc_flags.csv always lists the whole sample.
rows = []
for s in SUBJECTS:
    # Inputs for this participant. The brain mask doubles as the tckmap template so
    # the TDI lands on the diffusion grid; mean_b0 is only used for the picture.
    tck = OUT / s / "tckgen" / TRACT / f"{TRACT}_{CUTOFF}_cleaned.tck"
    template = PROJECT / "dwi" / s / "nodif_brain_mask.nii.gz"
    b0 = PROJECT / "dwi" / s / "mean_b0.nii.gz"
    png = QC / f"{s}_{TRACT}_qc.png"

```

```

# No cleaned bundle (Step 5 or 6 produced nothing for this participant): record it
# and move on rather than crash, so one bad participant does not block the rest.
if not tck.exists():
    rows.append({"Subject": s, "Voxels": 0, "Peak_density": 0, "Flag": "MISSING"})
    print(f"[{s}] MISSING cleaned bundle")
    continue

# Streamlines to voxels. tckmap's default output is the tract-density image (TDI):
# each voxel holds the number of streamlines that pass through it.
# -template put the output on this image's voxel grid, so the TDI overlays the
# b0 and mask voxel for voxel
# -force overwrite the scratch file if an interrupted run left one behind
# -quiet no progress bar or info messages in the output
# The scratch file is deleted as soon as the array is in memory; the leading "_"
# just marks it as temporary if a crash leaves it behind. check=True turns a
# non-zero exit from tckmap (a bad .tck, say) into an exception so the script stops
# instead of carrying on with no data. If tckmap is not on PATH at all, subprocess
# raises FileNotFoundError on its own.
tdi = QC / f"{s}_tdi.nii.gz"
subprocess.run(["tckmap", str(tck), str(tdi), "-template", str(template),
               "-force", "-quiet"], check=True)
density = nib.load(str(tdi)).get_fdata()
tdi.unlink()

# The two numbers the flags rest on: how many voxels the bundle touches at all, and
# how many streamlines share the busiest voxel. Printed per participant so an
# outlier that still clears the thresholds is visible as it scrolls by.
n_voxels = int((density > 0).sum())
peak = float(density.max())
flag = flag_for(n_voxels, peak)
rows.append({"Subject": s, "Voxels": n_voxels, "Peak_density": peak, "Flag": flag})
print(f"[{s}] {n_voxels} voxels, peak {peak:.0f} {flag}")

# Resume. The flag above was recomputed regardless; the picture is only redrawn
# when it is missing or FORCE=1. Delete one PNG to regenerate just that one.
if png.exists() and not FORCE:
    continue

# mean_b0 is optional in the project layout. Fall back to the brain mask, which
# gives a flat white silhouette but still shows whether the bundle sits inside it.
background = nib.load(str(b0 if b0.exists() else template)).get_fdata()
# Pick the slices with the most streamline hits. Summing over axes 0 and 1 leaves
# one total per z index (axial slices); summing over 0 and 2 leaves one per y index
# (coronal). This assumes the usual storage order (x left-right, y front-back,
# z bottom-top). If yours differs the panels are mislabelled; the flags are fine.
k_axial = int(np.argmax(density.sum(axis=(0, 1))))
k_coronal = int(np.argmax(density.sum(axis=(0, 2))))
# Two panels side by side, 10 x 5 inches; at the 100 dpi used below that is about
# 1000 x 500 px, enough to skim in a thumbnail browser.
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
overlay(axes[0], background[:, :, k_axial], density[:, :, k_axial], "Axial")
overlay(axes[1], background[:, k_coronal, :], density[:, k_coronal, :], "Coronal")
# The flag goes into the title so it is visible when skimming the images without
# the CSV open.
fig.suptitle(f"{s} {TRACT} cleaned" + (f" [{flag}]" if flag else ""), fontsize=11)
fig.tight_layout()
# bbox_inches="tight" trims the white margin. plt.close matters inside a loop:
# pyplot keeps every figure alive until it is closed, and a long sample would eat
# memory (matplotlib starts warning at 20 open figures).
fig.savefig(png, dpi=100, bbox_inches="tight")
plt.close(fig)

# Write the flag table and print a summary. index=False drops pandas' row numbers so

```

```
# the CSV has just Subject, Voxels, Peak_density and Flag. Anything with a non-empty
# Flag counts as flagged, so MISSING participants show up in the summary too.
flags = pd.DataFrame(rows)
flags.to_csv(QC / "qc_flags.csv", index=False)
flagged = flags[flags["Flag"] != ""]
print(f"\n{len(flagged)} of {len(flags)} participants flagged")
if len(flagged):
    print(flagged.to_string(index=False))
print(f"flags -> {QC / 'qc_flags.csv'}")
```

08_node_profiles.py

```
#!/usr/bin/env python3
"""Step 8. Along-tract profiles at 100 nodes (AFQ, Gaussian-weighted).
```

Run in a shell where 00_config.sh has been sourced:

```
source 00_config.sh
python 08_node_profiles.py
```

What this does. For every participant it loads the cleaned bundle from Step 6, lines the streamlines up so they all run seed -> target, resamples each one to 100 points (nodes) and, at every node, averages a scalar map (FA, NODDI) over the streamlines. The result is one curve of 100 values per metric per participant: the tract profile of Yeatman et al. (2012), computed with DIPY's afq_profile.

Streamlines are first oriented to the bundle centroid and the centroid is oriented so that it starts at the seed region; node 0 is therefore the seed end and node 99 the target end in every participant. At each node, every streamline's value is weighted by the inverse of its Mahalanobis distance from the bundle core (Yeatman et al., 2012), so a stray that survived Step 6 counts for less than the core.

METRICS maps a column name to a scalar image; add or remove entries freely. A metric whose image exists for no participant (for example NODDI when Step 8a was not run) is dropped with a notice; a participant missing any remaining image is skipped.

Needs, per participant:

\$OUT/<s>/tckgen/<TRACT>/<CUTOFF>_cleaned.tck	cleaned bundle (Step 6)
\$OUT/<s>/rois/<TRACT>_seed_diff.nii.gz	seed region in diffusion space (Step 2); says which end is 0
\$PROJECT/dwi/<s>/fa.nii.gz	FA map (preprocessing)
\$PROJECT/noddi/<s>/fit_*.nii.gz	NODDI maps (Step 8a, optional)

Output: \$OUT/nodewise/<TRACT>_nodewise_all_subjects.csv
(long format: Subject, Tract, Node, one column per metric)

Step 8b pivots this into one wide file per metric for the Step 9 models.

Runtime: the tutorial page quotes 20 to 40 min for 57 participants and four tracts, so roughly 5 to 10 min for one run of this script. There is no FORCE switch; every run rebuilds the CSV from scratch.

What to look for. The DONE line gives the participant count, which should equal the number of IDs in subjects.txt minus the SKIP lines above it; the CSV then has exactly 100 x that many rows (5,700 for 57 participants). If you run both hemispheres, check the orientation: the across-participant correlation of node i on the left with node i on the right is strongly positive when both sides run seed -> target and strongly negative when one side is reversed (Step 8 page, Verification). Nodes 0-4 and 95-99 sit in or next to gray matter and are partial-volume mixtures; whether Step 9 trims them (--trim in 09a) is a decision to make before modelling, not after.

```
"""
```

```
import os
```

```

import sys
from pathlib import Path

import nibabel as nib
import numpy as np
import pandas as pd
# DIPY does the real work. load_nifti and load_tractogram read the scalar maps and the
# .tck; QuickBundles with the feature and metric classes builds the bundle centroid;
# orient_by_streamline flips streamlines to match it; gaussian_weights and afq_profile
# are DIPY's implementation of the AFQ tract profile.
from dipy.io.image import load_nifti
from dipy.io.streamline import load_tractogram
from dipy.segment.clustering import QuickBundles
from dipy.segment.featurespeed import ResampleFeature
from dipy.segment.metricspeed import AveragePointwiseEuclideanMetric
from dipy.stats.analysis import afq_profile, gaussian_weights
from dipy.tracking.streamline import Streamlines, orient_by_streamline

# Read one exported variable, or stop with a hint. Argument: the variable name.
# Returns its value as a string. The Python steps cannot source 00_config.sh
# themselves; they only see what "source 00_config.sh" exported into the shell that
# launched them, so a missing name almost always means the config was not sourced (or
# was sourced in a different terminal). sys.exit with a string prints it and exits 1.
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]

# Values from 00_config.sh. PROJECT holds the raw data (dwi/, noddi/); OUT is where the
# workflow writes. TRACT and CUTOFF are spliced into file names, so they must be the
# values Steps 5 and 6 ran with or the cleaned .tck is not found and everyone is
# skipped. CUTOFF stays a string because it only ever appears in a file name.
PROJECT = Path(env("PROJECT"))
OUT = Path(env("OUT"))
TRACT = env("TRACT")
CUTOFF = env("CUTOFF")
# bash cannot export an array, so the ID list is re-read from the file. read_text()
# gives the whole file as one string and split() breaks it on any whitespace, so blank
# lines, CRLF endings and a missing final newline are all harmless (the same tolerance
# read_subjects has in 00_config.sh).
SUBJECTS = Path(env("SUBJECTS_FILE")).read_text().split()

# 100 nodes is the AFQ convention and the rest of the workflow is built on it: Step 9a's
# quartiles are nodes 0-24, 25-49, 50-74, 75-99 and Step 9c counts analyses in blocks
# of 100. Change this and those two need editing as well.
N_NODES = 100
# Below this many streamlines the participant is skipped rather than profiled. A real
# bundle has hundreds (the smallest cleaned bundle in the example dataset had 659), so a
# handful means tracking or cleaning failed, not a bundle worth a curve. The per-node
# covariance in gaussian_weights is also estimated across streamlines, so with this few
# the weights mean little. Why exactly 5 is not written down anywhere.
MIN_STREAMLINES = 5
# Column name -> image path under $PROJECT, with {s} standing in for the participant ID.
# Every image must be in diffusion space, where the streamlines are; the first one is
# also the reference grid when the .tck is loaded. NDI and ODI use the modulated
# (tissue-weighted) maps from Step 8a; FWF has no modulated form. Add a line for any
# other diffusion-space map you want profiled; the key becomes the CSV column and, via
# Step 8b, the <METRIC> in the Step 9 file names.
METRICS = {
    "FA": "dwi/{s}/fa.nii.gz",
    "NDI": "noddi/{s}/fit_NDI_modulated.nii.gz",

```

```

    "ODI": "noddi/{s}/fit_ODI_modulated.nii.gz",
    "FWF": "noddi/{s}/fit_FWF.nii.gz",
}

# Drop any metric that no participant has an image for, e.g. the three NODDI entries
# when Step 8a was never run. any(...) over the participants stops at the first image
# it finds, so this costs almost nothing. The list is built first because a dict cannot
# shrink while it is being iterated. A metric that exists for only some participants
# is kept; the participants without it are skipped in the main loop.
unavailable = [m for m, template in METRICS.items()
               if not any((PROJECT / template.format(s=s)).exists() for s in SUBJECTS)]
for m in unavailable:
    print(f"metric {m}: no image found for any participant; not profiled")
    del METRICS[m]
# Not even FA was found: PROJECT is probably wrong, or fa.nii.gz is not where the layout
# in 00_config.sh expects it.
if not METRICS:
    sys.exit("no scalar images found")

# Centre of mass of the participant's warped seed region. Argument: participant ID.
# Returns (x, y, z) in scanner mm, the same space the streamlines are loaded into
# below, so the two can be compared directly.
# get_fdata() pulls the voxel array; argwhere lists the (i, j, k) index of every
# non-zero voxel; mean(axis=0) is their centroid in voxel units; apply_affine pushes
# that through the image's voxel-to-world matrix into mm.
def seed_centre_mm(s):
    """Centre of mass of the warped seed region, in scanner (mm) coordinates."""
    img = nib.load(str(OUT / s / "rois" / f"{TRACT}_seed_diff.nii.gz"))
    voxels = np.argwhere(img.get_fdata() > 0)
    return nib.affines.apply_affine(img.affine, voxels.mean(axis=0))

# Flip streamlines so they all run seed -> target. Arguments: the loaded streamlines
# and the seed centre from seed_centre_mm. Returns a new Streamlines object; the input
# is left alone.
# Streamlines in a .tck may run in either direction, so without this node 0 would be
# the seed end for some streamlines and the target end for others, and the profile
# would blend the two directions.
def orient_seed_to_target(streamlines, seed_mm):
    """Orient all streamlines alike, with node 0 at the seed end."""
    # One reference streamline: the QuickBundles centroid. ResampleFeature puts every
    # streamline on N_NODES points, AveragePointwiseEuclideanMetric compares two
    # streamlines by the mean distance between matching points, and threshold=np.inf
    # means "everything is close enough", so there is exactly one cluster and
    # .centroids[0] is its mean streamline. QuickBundles tries each streamline both
    # ways before folding it into the running mean, so mixed directions in the file do
    # not smear the centroid (checked on a synthetic bundle with half of it reversed).
    metric = AveragePointwiseEuclideanMetric(ResampleFeature(nb_points=N_NODES))
    centroid = QuickBundles(threshold=np.inf, metric=metric).cluster(streamlines).centroids[0]
    # The centroid itself may run either way. Compare its two ends with the seed
    # centre (np.linalg.norm is Euclidean distance) and, if the first point is the far
    # one, reverse it (:::-1] is the array backwards). This is what makes node 0 the
    # seed end in every participant, not just "the same end within one participant".
    starts_at_target = (np.linalg.norm(centroid[0] - seed_mm)
                       > np.linalg.norm(centroid[-1] - seed_mm))
    if starts_at_target:
        centroid = centroid[::-1]
    # orient_by_streamline resamples each streamline and the centroid to 12 points,
    # sums the point-to-point distances as is and flipped, and reverses the streamline
    # when flipped is closer. It works on a copy. Streamlines() keeps the result as
    # the array-sequence type the DIPY calls below expect.

```

```

    return Streamlines(orient_by_streamline(streamlines, centroid))

# Sample one scalar map along the bundle. Arguments: image path, the oriented
# streamlines, and the (n_streamlines x N_NODES) weight matrix from gaussian_weights.
# Returns a 1-D array of N_NODES values.
# load_nifti returns the voxel array and its voxel-to-world affine.  afq_profile
# resamples each streamline to n_points, reads the map at every point (the affine maps
# mm back to voxels; values are trilinearly interpolated between voxels), then averages
# across streamlines at each node with np.average and the given weights.  DIPY checks
# that the weights sum to 1 per node and raises otherwise.
def profile(image, streamlines, weights):
    data, affine = load_nifti(str(image))
    return afq_profile(data, streamlines, affine, n_points=N_NODES, weights=weights)

# Main loop, one participant at a time.  rows collects one dict per node per
# participant and becomes the CSV at the end.
rows = []
for s in SUBJECTS:
    # The cleaned bundle from Step 6.  CUTOFF is part of the name, so editing it in the
    # config after Step 5 points this at a file that does not exist.
    tck = OUT / s / "tckgen" / TRACT / f"{TRACT}_{CUTOFF}_cleaned.tck"
    # Full path of every scalar map for this participant, keyed by column name.
    maps = {m: PROJECT / template.format(s=s) for m, template in METRICS.items()}
    # The .tck and every map must exist.  [tck, *maps.values()] is one list of all the
    # paths (the * unpacks the dict values into it).  Only the first missing path is
    # printed, to keep the log readable; fix it, rerun, and see if there is another.
    missing = [str(p) for p in [tck, *maps.values()] if not p.exists()]
    if missing:
        print(f"[{s}] SKIP: missing {missing[0]}")
        continue

    # load_tractogram needs a reference image for the grid and affine.  Any of the maps
    # will do, since they all sit on the diffusion grid, so the first one is used
    # (next(iter(...)) is "the first value in the dict").  Streamlines come back in
    # scanner mm (DIPY's default space), the same units as seed_centre_mm.
    # bbox_valid_check=False: DIPY otherwise refuses a tractogram with any point outside
    # the image's bounding box; Step 6 turned the check off for these same files, so we
    # do too.  .streamlines drops the tractogram wrapper and keeps the coordinates.
    reference = next(iter(maps.values()))
    streamlines = load_tractogram(str(tck), str(reference), bbox_valid_check=False).streamlines
    # Too small to weight or to trust; see MIN_STREAMLINES above.
    if len(streamlines) < MIN_STREAMLINES:
        print(f"[{s}] SKIP: {len(streamlines)} streamlines")
        continue

    # Orientation first, then the weights: the weights are computed per node and only
    # make sense once node i is the same place along the bundle in every streamline.
    # gaussian_weights resamples to N_NODES and, at each node, takes the mean and
    # covariance of the streamline positions, computes each streamline's Mahalanobis
    # distance from that mean, and returns 1/distance normalised to sum to 1 across
    # streamlines.  Streamlines near the bundle core dominate the average; strays that
    # survived Step 6's 3 SD cut count for little.  The weights depend on geometry
    # only, so the same matrix is reused for every metric.
    streamlines = orient_seed_to_target(streamlines, seed_centre_mm(s))
    weights = gaussian_weights(streamlines, n_points=N_NODES)
    # One 100-value profile per metric, keyed by column name.
    profiles = {m: profile(p, streamlines, weights) for m, p in maps.items()}
    # Long format: one row per node with Subject, Tract and Node as keys and every
    # metric as a column.  The ** splices the metric values into the row dict; float()
    # turns each numpy scalar into a plain number.  Tract is constant within a run but
    # lets CSVs from several tracts be stacked and still told apart.

```

```

for node in range(N_NODES):
    rows.append({"Subject": s, "Tract": TRACT, "Node": node,
                **{m: float(profiles[m][node]) for m in METRICS}})
    print(f"[{s}] {len(streamlines)} streamlines, {N_NODES} nodes, {len(METRICS)} metrics")

# Write the table. mkdir(parents=True, exist_ok=True) creates $OUT/nodewise if needed
# and is silent if it already exists. index=False keeps pandas' row numbers out of the
# file. len(rows) // N_NODES is the number of participants profiled; Step 8b reads
# this file next and pivots it to one wide row per participant.
out = OUT / "nodewise" / f"{TRACT}_nodewise_all_subjects.csv"
out.parent.mkdir(parents=True, exist_ok=True)
pd.DataFrame(rows).to_csv(out, index=False)
print(f"DONE -> {out} ({len(rows) // N_NODES} participants)")

```

08a_noddi_fit.py

```

#!/usr/bin/env python3
"""Step 8a. NODDI fit with AMICO (run before Step 8 when NODDI metrics are profiled).

```

Fits the NODDI model (Zhang et al., 2012) to each participant's preprocessed diffusion data with AMICO (Daducci et al., 2015). AMICO recasts the fit as a linear problem against a precomputed dictionary of response kernels, which is why a whole brain takes minutes here rather than hours. Step 8 then samples the maps along the cleaned bundle. If you only want FA profiles, skip this script; Step 8 notices the missing maps and drops the NODDI columns with a message.

Run in a shell where 00_config.sh has been sourced:

```

source 00_config.sh
python 08a_noddi_fit.py                # every participant in $SUBJECTS_FILE
python 08a_noddi_fit.py sub-01 sub-02  # selected participants

```

Needs, per participant (all from preprocessing; nothing from Steps 1 to 7):

```

$PROJECT/dwi/<subj>/data.nii.gz          eddy-corrected multi-shell DWI
$PROJECT/dwi/<subj>/bvals, bvecs          FSL-format b-values and directions
$PROJECT/dwi/<subj>/nodif_brain_mask.nii.gz brain mask; only voxels inside are fitted

```

NODDI needs at least two $b > 0$ shells; one shell cannot separate its compartments.

Writes fit_NDI, fit_ODI and fit_FWF, the tissue-weighted maps fit_NDI_modulated and fit_ODI_modulated (Parker et al., 2021), and fit_RMSE. Tract profiles use the modulated NDI and ODI maps and the unmodulated FWF map.

```

$PROJECT/noddi/<subj>/fit_NDI.nii.gz      neurite density index, 0 to 1
$PROJECT/noddi/<subj>/fit_ODI.nii.gz      orientation dispersion index, 0 to 1
$PROJECT/noddi/<subj>/fit_FWF.nii.gz      free-water (isotropic) fraction, 0 to 1
$PROJECT/noddi/<subj>/fit_NDI_modulated.nii.gz NDI x (1 - FWF)
$PROJECT/noddi/<subj>/fit_ODI_modulated.nii.gz ODI x (1 - FWF)
$PROJECT/noddi/<subj>/fit_RMSE.nii.gz     fit error per voxel, for QC only
$PROJECT/noddi/work/<subj>/              scheme file and kernels; safe to delete

```

AMICO may add a few housekeeping files of its own next to the maps; Step 8 ignores them.

Step 8 (08_node_profiles.py) reads the two modulated maps and fit_FWF through its METRICS dictionary. The modulated maps matter: a plain NDI mean over a stretch of tract that borders CSF is pulled around by how much free water each voxel holds; weighting each voxel by its tissue fraction (1 - FWF) down-weights the CSF-heavy ones (Parker et al., 2021). FWF has no modulated form, so the plain map is profiled.

The intrinsic parallel diffusivity defaults to the white-matter value ($1.7e-3$ mm²/s). For gray-matter regions set NODDI_DPAR=1.1e-3; results are then written to \$PROJECT/noddi_gm so that white-matter maps are not overwritten. Response kernels are generated per participant (work/<subj>/kernels), so several instances of the script can run concurrently on different participants. Step 8 reads noddi/ only; to profile the gray-matter fit, point its METRICS entries at noddi_gm/ yourself.

Runtime: a few minutes per participant at four threads (NODDI_NTHREADS, default 4; export it before running to change it); the per-participant kernels are part of that. What to look for: one "NODDI maps written" line per participant, then open a fit_NDI and fit_FWF pair in fsleyes. NDI should be highest in dense white matter, ODI lowest in the corpus callosum and FWF close to 1 in the ventricles. fit_RMSE should be low and fairly flat inside the brain; a region that lights up is where the model or the data went wrong and is worth a look before profiling.

Outputs: \$PROJECT/noddi/<subj>/ (or \$PROJECT/noddi_gm/<subj>/)

```
"""
import os
import sys
from pathlib import Path

# Thread count for this process. It is not in 00_config.sh; it comes straight from the
# environment, so "export NODDI_NTHREADS=8" before running changes it. Peak load on a
# shared machine is this number times the instances you start (see the note on NTHREADS
# and MAXJOBS in 00_config.sh).
N_THREADS = os.environ.get("NODDI_NTHREADS", "4")
# numpy's BLAS (OpenBLAS in the pip wheels) is loaded when numpy is imported and reads
# these variables then, and "import amico" pulls numpy in. Set them later and they do
# nothing.
os.environ["OPENBLAS_NUM_THREADS"] = N_THREADS      # must be set before amico is imported
os.environ["OMP_NUM_THREADS"] = N_THREADS

import amico

# Read one variable exported by 00_config.sh, or stop with a message that says which one
# is missing. Every Python script in the workflow starts with this same helper.
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]

PROJECT = Path(env("PROJECT"))
# FORCE=1 in the environment (or in 00_config.sh) redoes participants whose maps exist.
FORCE = os.environ.get("FORCE", "0") == "1"
# IDs on the command line win; with none given, "or" falls through to the list in
# $SUBJECTS_FILE. split() on whitespace tolerates CRLF endings and blank lines, the same
# things read_subjects in 00_config.sh guards against for the shell scripts.
SUBJECTS = sys.argv[1:] or Path(env("SUBJECTS_FILE")).read_text().split()

# Model constants. dPar is the diffusivity along the neurites inside the stick
# compartment and is fixed, not fitted, so it has to suit the tissue: 1.7e-3 for white
# matter (AMICO's default, used when NODDI_DPAR is unset) and about 1.1e-3 for gray
# matter. The white-matter value is not appropriate in gray matter, and the tutorial
# page notes the two fits can give different results.
D_PAR = os.environ.get("NODDI_DPAR")      # mm2/s; unset = AMICO default for white matter
# Free-water compartment, fixed at the diffusivity of water at body temperature; this is
# also AMICO's own default.
D_ISO = 3.0e-3                          # isotropic (free water) diffusivity, mm2/s
# Scanners often report small nonzero b (5, 10...) for the b = 0 volumes; anything under
# 100 is treated as b = 0 so those volumes normalise the signal instead of forming a shell.
B0_THRESHOLD = 100                      # volumes with b below this are treated as b = 0
# Jittered b-values (995, 1003...) are rounded to the nearest multiple of B_STEP so AMICO
# sees clean shells. Keep it smaller than the gap between your closest shells or two
# shells can round to the same value, and check the scheme file once: a shell that is
# not itself a multiple gets moved (3250 becomes 3200), so its kernels are built for 3200.
B_STEP = 200                            # b-values are rounded to the nearest multiple
```

```

# Output tree. The directory is chosen by whether NODDI_DPAR is set at all, not by its
# value, so a white-matter run with NODDI_DPAR=1.7e-3 exported would still land in
# noddigm. Unset it for the normal run.
STUDY = PROJECT / ("noddigm" if DPAR else "noddig")
STUDY.mkdir(parents=True, exist_ok=True)
# One-off AMICO housekeeping: precomputes the rotation matrices it uses to turn kernels
# towards each voxel's fibre direction and caches them as pickles in ~/.dipy. The first
# call takes a while; later calls find the cache and return at once.
amico.setup()

for s in SUBJECTS:
    # Inputs come from preprocessing, in the layout 00_config.sh documents.
    dwi_dir = PROJECT / "dwi" / s
    dwi = dwi_dir / "data.nii.gz"
    bvals = dwi_dir / "bvals"
    bvecs = dwi_dir / "bvecs"
    mask = dwi_dir / "nodif_brain_mask.nii.gz"
    out = STUDY / s

    # Done already? The modulated NDI map is the one Step 8 needs, so it is the test.
    if (out / "fit_NDI_modulated.nii.gz").exists() and not FORCE:
        print(f"[{s}] NODDI maps exist")
        continue

    # Report every missing input at once, then skip the participant.
    missing = [p.name for p in (dwi, bvals, bvecs, mask) if not p.exists()]
    if missing:
        print(f"[{s}] SKIP: missing {'', '.join(missing)}")
        continue

    # AMICO empties the output directory when it saves, so the scheme file and
    # the response kernels are kept in a separate working directory.
    # The working directory is also per participant on purpose: AMICO's default puts
    # every participant's kernels in one shared folder, and two instances running at once
    # then delete each other's files (the FileNotFoundError listed in Troubleshooting).
    work = STUDY / "work" / s
    work.mkdir(parents=True, exist_ok=True)
    out.mkdir(parents=True, exist_ok=True)

    # AMICO reads the acquisition as one scheme file (a row per volume with the gradient
    # direction and b-value together), not FSL's separate bvals and bvecs. fsl2scheme
    # converts them; bStep is the rounding described at B_STEP above.
    scheme = work / f"{s}.scheme"
    amico.util.fsl2scheme(str(bvals), str(bvecs), str(scheme), bStep=B_STEP)

    # Evaluation(study directory, participant ID, output_path) is AMICO's handle for one
    # fit. Left to itself it would write under <study>/<ID>/AMICO/NODDI/; giving
    # output_path explicitly puts the maps straight in noddig/<subj>/ where Step 8 looks.
    ae = amico.Evaluation(str(STUDY), s, str(out))
    # nthreads: voxels fitted in parallel. BLAS_nthreads: threads inside each linear
    # algebra call; 1 as on the tutorial page, so the two do not multiply and oversubscribe.
    ae.set_config("nthreads", int(N_THREADS))
    ae.set_config("BLAS_nthreads", 1)
    # Also write the tissue-weighted maps (NDI and ODI times 1 - FWF) that Step 8 profiles.
    ae.set_config("doSaveModulatedMaps", True)
    # Also write fit_RMSE, the per-voxel error between measured and fitted signal.
    ae.set_config("doComputeRMSE", True)
    # b0_thr is B0_THRESHOLD above. Only voxels inside the mask are fitted; the rest
    # come out as zero in every map.
    ae.load_data(str(dwi), str(scheme), str(mask), b0_thr=B0_THRESHOLD)

    # set_model picks NODDI and, as a side effect, resets ATOMS_path (where the kernels
    # live) to AMICO's shared default, which is why the per-participant path is set only
    # afterwards. Swap the order and the parallel-run problem above comes back.
    ae.set_model("NODDI")

```

```

ae.set_config("ATOMS_path", str(work / "kernels"))      # after set_model, which resets it
# model.set(dPar, dIso, IC_VFs, IC_ODs, isExvivo). Only dPar changes here; IC_VFs
# and IC_ODs are handed back unchanged so the kernel grid (the intra-cellular volume
# fractions and dispersions AMICO tabulates) stays at its default, and False keeps
# the in vivo model, i.e. no ex vivo "dot" compartment.
if D_PAR:
    ae.model.set(float(D_PAR), D_ISO, ae.model.IC_VFs, ae.model.IC_ODs, False)
# Build the kernel dictionary for this scheme and dPar in ATOMS_path. regenerate=True
# recomputes even if files are there, so a half-written set from a killed run is never
# picked up. load_kernels then projects them onto this participant's gradient scheme.
ae.generate_kernels(regenerate=True)
ae.load_kernels()
# The fit itself: per voxel, a non-negative linear fit of the signal against the kernel
# dictionary, then the kernel weights are turned into NDI, ODI and FWF. save_results
# writes the fit_*.nii.gz files into out (after clearing it; see the note at "work").
ae.fit()
ae.save_results()
# dPar is printed so a log shows which tissue setting produced the maps.
print(f"[{s}] NODDI maps written (dPar = {ae.model.dPar})")

print(f"DONE -> {STUDY}")

```

08b_build_analysis_csv.py

```

#!/usr/bin/env python3
"""Step 8b. Assemble the analysis files read by the Step 9 scripts.

```

Step 8 leaves the profiles in long format, one row per participant and node, so 100 rows per participant and one column per metric. The Step 9 models want the opposite shape, one row per participant with the 100 node values spread across columns, sitting next to the covariates, the outcomes and the two bundle statistics from Step 6. This script does that reshape and join once per metric. It is plain pandas, ignores FORCE (everything is recomputed on every run, which is cheap) and finishes in seconds on a full sample.

Run in a shell where 00_config.sh has been sourced:

```

source 00_config.sh
python 08b_build_analysis_csv.py

```

Inputs (all three must exist; a missing file is a pandas FileNotFoundError, not a skip)

\$OUT/nodewise/<TRACT>_nodewise_all_subjects.csv	node profiles (Step 8)
\$OUT/nodewise/<TRACT>_tract_stats.csv	cleaned-bundle count and length (Step 6)
\$COVARIATES_CSV	Subject, covariates, outcomes

Output, one file per metric, one row per participant:

```

$OUT/analysis/<TRACT>__<METRIC>__analysis.csv
Subject, covariates, outcomes, Streamline_count, Mean_length_mm, <METRIC>_0 ... <METRIC>_99

```

The double underscore in the name is what separates the fields, since TRACT itself has single underscores in it (l_vta_l_hipp). Step 9a rebuilds exactly this name from TRACT and --metric, and Step 9b takes the path on its command line, so do not rename the files.

What to look at when it finishes. One "<file>: N participants, 100 nodes" line per metric (FA alone if Step 8a was skipped; FA, NDI, ODI, FWF otherwise). N is the number of participants who have a covariate row AND a cleaned bundle AND a profile, so it can be smaller than subjects.txt. Anyone profiled in Step 8 but missing from the covariate file is named on a final line; add them to covariates.csv and rerun. Nodes must read 100.

Where the files go next: python 09a_tract_models.py --metric NDI ... and
Rscript 09b_nodewise_permutation.R "\$OUT/analysis/<TRACT>__NDI__analysis.csv" ...
both read them straight from \$OUT/analysis.

```

"""
import os
import sys
from pathlib import Path

# Standard library plus pandas; no imaging libraries are needed for this step.
import pandas as pd

# Read one exported variable or stop with a clear message. Argument: the variable name.
# Returns its value as a string. os.environ only holds what the shell exported, which is
# why the docstring says "source 00_config.sh": every export in that file lands here.
# "bash 00_config.sh" would run it in a throwaway shell and export nothing. sys.exit with
# a string prints it to stderr and exits with status 1, so a missing variable stops the
# run before any file is read or written.
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]

# Three config values are read: OUT and TRACT here, COVARIATES_CSV a few lines down.
# A stale TRACT (edited in 00_config.sh but not re-sourced) would silently build files for
# the wrong tract, so check the file names this script prints against what you meant.
OUT = Path(env("OUT"))
TRACT = env("TRACT")

# Load the three inputs. dtype={"Subject": str} is not decoration. Left to itself pandas
# guesses the ID type per file: an all-digit column comes back as integers (0123 turns
# into 123), and if one file also holds an ID like sub-01 it is read as text instead, and
# merge() then refuses to join the two ("trying to merge on int64 and object columns").
# Forcing str in all three keeps the IDs exactly as written in subjects.txt.
profiles = pd.read_csv(OUT / "nodewise" / f"{TRACT}_nodewise_all_subjects.csv",
                      dtype={"Subject": str})
tract_stats = pd.read_csv(OUT / "nodewise" / f"{TRACT}_tract_stats.csv", dtype={"Subject": str})
covariates = pd.read_csv(env("COVARIATES_CSV"), dtype={"Subject": str})
# Every column in the profile file that is not an identifier is a metric, so this list
# follows whatever Step 8 actually profiled and nothing here needs editing when METRICS
# changes there. Tract is constant within the file (it is just TRACT) and is dropped.
metrics = [c for c in profiles.columns if c not in ("Subject", "Tract", "Node")]

# parents=True also creates $OUT if it is missing; exist_ok=True means no error on reruns.
analysis_dir = OUT / "analysis"
analysis_dir.mkdir(parents=True, exist_ok=True)

# One wide file per metric rather than one giant table. Step 9 is run per metric anyway,
# and a 100-column file is easier to open and eyeball than a 400-column one.
for metric in metrics:
    # Long to wide. Participants become rows, the 100 Node values become 100 columns, and
    # each cell is this metric's value at that node. pivot() refuses to guess if a
    # participant/node pair appears twice ("Index contains duplicate entries"); the usual
    # cause is an ID listed twice in subjects.txt, since Step 8 then profiles it twice.
    # A participant with fewer than 100 rows would get NaN at the missing nodes. Step 8
    # always writes all 100, but if it happened 9b would drop them and 9a would average
    # over the nodes present.
    wide = profiles.pivot(index="Subject", columns="Node", values=metric)
    # pivot() leaves bare node numbers (0 ... 99) as the column labels. Rename them to
    # NDI_0 ... NDI_99 so each column says which metric it is. This naming is a contract
    # with both Step 9 scripts (9a rebuilds f"{METRIC}_{node}", 9b greps ^<METRIC>_(d+$),
    # so keep prefix, underscore, integer. int() strips a stray .0 if Node were ever read
    # back as float.
    wide.columns = [f"{metric}_{int(node)}" for node in wide.columns]
    # Only the two bundle statistics travel on; Count_uncleaned and Retention_pct stay in

```

```

# the Step 6 table. Both are in COVARIATES by default (00_config.sh), so they have to
# be in this file or Step 9a stops with "columns not found". 00_config.sh also has the
# note on why Streamline_count is the covariate to think twice about.
bundle = tract_stats[["Subject", "Streamline_count", "Mean_length_mm"]]
# Two inner joins on Subject (pandas' default, how="inner"). A participant survives
# only with a covariate row, a Step 6 row (a cleaned bundle) and a Step 8 profile.
# Dropping rather than padding with NaN is deliberate, since Step 9 drops incomplete
# rows anyway, but it is why the count printed below can be smaller than subjects.txt;
# the check at the bottom names the ones lost at the covariate join. wide carries
# Subject as its index after pivot(), so reset_index() turns it back into a column
# that merge() can match on.
table = covariates.merge(bundle, on="Subject").merge(wide.reset_index(), on="Subject")
# index=False stops pandas writing its own 0..N-1 row index as an unnamed first
# column, which would only be clutter in every downstream file.
path = analysis_dir / f"{TRACT}__{metric}__analysis.csv"
table.to_csv(path, index=False)
# The line to read. Participants is the number of IDs present in all three inputs,
# so never more than the smallest of them; nodes should be 100.
print(f"{path.name}: {len(table)} participants, {wide.shape[1]} nodes")

# Who fell out at the covariate join. "profiled - covariates" is set arithmetic, the IDs
# in the first set that are not in the second. Only this direction is reported because
# it is the one you fix here, by adding rows to covariates.csv and rerunning; a
# participant missing from tract_stats has no _cleaned.tck, and Step 6 prints a SKIP
# line for the usual reasons (no tractogram, 0 streamlines). The reverse case (in the
# covariate file but never profiled) is silent, so compare the counts above with
# wc -l subjects.txt if they look low. Nothing is printed when the sets match.
profiled = set(profiles["Subject"])
absent = sorted(profiled - set(covariates["Subject"]))
if absent:
    print(f"profiled but absent from the covariate file: {' '.join(absent)}")

```

09a_tract_models.py

```

#!/usr/bin/env python3
"""Step 9a. Whole-tract and quartile models.

```

Step 8b left one wide table per metric with a row per participant: the covariates and outcomes from covariates.csv, the cleaned bundle's streamline count and mean length, and the metric at each of the 100 nodes. This script collapses the nodes into segment means (the whole tract and four quartiles), regresses each outcome on each segment mean with the covariates, and writes one tidy table of estimates. Given two tracts it also fits a mixed model asking whether the association differs between them.

Run in a shell where 00_config.sh has been sourced:

```

source 00_config.sh
python 09a_tract_models.py --metric NDI --outcomes memory_accuracy,memory_bias
python 09a_tract_models.py --metric NDI --outcomes memory_accuracy \
    --tracts l_vta_l_hipp,anterior_l_vta_l_hipp

```

The metric is averaged over the whole tract and within quartiles (nodes 0-24, 25-49, 50-74, 75-99). All variables are standardized, so coefficients are standardized betas.

One tract. For each segment: $\text{outcome} \sim \text{metric} + \text{covariates}$ (ordinary least squares). Quartile p values are corrected across the four quartiles (Benjamini-Hochberg). Uniformity along the tract: $\text{outcome} \sim \text{mean}(Q1, Q4) + (Q4 - Q1) + \text{covariates}$. An effect carried by the mean and not by the difference is diffuse, not localized.

Two tracts that share most of their course (--tracts a,b). Both are stacked, two rows per participant, each row with the streamline count and length of its own tract:

metric ~ outcome + tract + covariates + (1 | participant), fitted by maximum likelihood. The outcome x tract interaction is tested by likelihood ratio (1 df); the main effect is reported from the model without the interaction.

Input: \$OUT/analysis/<TRACT>__<METRIC>__analysis.csv (Step 8b; --analysis-dir overrides)
Columns: Subject, covariates, outcomes, Streamline_count, Mean_length_mm and <METRIC>_0 ... <METRIC>_99. Node 0 is the seed end, 99 the target end (Step 8).
Output: \$OUT/analysis/<label>__<METRIC>__tract_models.csv
One row per model term; <label> is the tract, or a+b for two tracts. Columns: model (ols, uniformity, mixed), tract, segment, outcome, term, n, beta, statistic, p, p_fdr (quartile rows only), interaction_p (mixed rows only).
Nothing downstream reads this file; it is the table you report. Step 9b goes back to the analysis CSV for the node-wise test.

Takes seconds: a few dozen small OLS fits, plus ten mixed models per outcome when two tracts are given. The table is printed as well as written. Check that n is your sample size minus anyone with a missing value, that p_fdr is filled on the Q1-Q4 rows only and interaction_p on the mixed rows only, and that the last line names the file you expected.

```
"""
import argparse
import os
import sys
import warnings
from pathlib import Path

# statsmodels.formula.api gives R-style formulas ("y ~ x + z") for both OLS and the mixed
# model; multipletests does the Benjamini-Hochberg correction; chi2 turns the
# likelihood-ratio statistic of the two-tract test into a p value.
import pandas as pd
import statsmodels.formula.api as smf
from scipy.stats import chi2
from statsmodels.stats.multitest import multipletests

# Read one variable exported by 00_config.sh, or stop with a message that says what to
# do. Argument: the variable name. Returns its value as a string. Python only sees
# variables exported in the shell that launched it, which is why "source 00_config.sh"
# has to come first, and again after every edit to the config.
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]

# Command line. --metric is the node-column prefix Step 8b used (NDI gives NDI_0 ...
# NDI_99). --outcomes lists the outcome columns to model; every model below is fitted
# once per outcome. --tracts defaults to $TRACT from the config; two names, comma
# separated, add the mixed model. --trim drops that many nodes at each end before
# averaging (see SEGMENTS). --analysis-dir is for analysis files kept somewhere other
# than $OUT/analysis, for instance a copy you are re-analysing on a laptop; the output
# CSV is written there as well.
parser = argparse.ArgumentParser(description="Whole-tract and quartile models")
parser.add_argument("--metric", required=True, help="metric prefix, e.g. NDI")
parser.add_argument("--outcomes", required=True, help="comma-separated outcome columns")
parser.add_argument("--tracts", default=None, help="one tract, or two separated by a comma")
parser.add_argument("--trim", type=int, default=0, help="nodes excluded at each end")
parser.add_argument("--analysis-dir", default=None)
args = parser.parse_args()

# Settings. "a or b" takes a when it is set and b otherwise, so --tracts wins over the
# config's TRACT. COVARIATES is the same comma-separated list Step 9b uses. Nothing is
# stripped of whitespace: COVARIATES="ICV, age" would look for a column called " age" and
# stop in load() with "columns not found"; a space after the comma in --tracts makes load()
```

```

# look for a file whose name starts with a space, and read_csv stops with FileNotFoundError.
METRIC = args.metric
OUTCOMES = args.outcomes.split(",")
TRACTS = (args.tracts or env("TRACT")).split(",")
COVARIATES = env("COVARIATES").split(",")
ANALYSIS = Path(args.analysis_dir) if args.analysis_dir else Path(env("OUT")) / "analysis"
# Two guards. The mixed model is written for exactly two tracts, and trimming 25 or more
# nodes would leave Q1 or Q4 with nothing to average.
if len(TRACTS) > 2:
    sys.exit("--tracts takes one tract or two")
if not 0 <= args.trim < 25:
    sys.exit("--trim must be between 0 and 24 (nodes excluded at each end)")

# Node ranges as half-open (lo, hi) pairs, so range(lo, hi) is the node list. Whole is
# nodes 0-99 and each quartile is 25 nodes. Node 0 is the seed end and 99 the target end
# (Step 8 orients every bundle that way), so Q1 sits at the seed and Q4 at the target.
# --trim shortens only Whole, Q1 and Q4. The end nodes (roughly 0-4 and 95-99) lie in or
# next to grey matter and carry partial-volume signal from the seed and target regions,
# which is the reason to consider trimming. Decide before looking at any result, and say
# so when reporting; Step 9b has no trim option and always tests all 100 nodes.
SEGMENTS = {"Whole": (args.trim, 100 - args.trim), "Q1": (args.trim, 25), "Q2": (25, 50),
            "Q3": (50, 75), "Q4": (75, 100 - args.trim)}

def q(column):
    """Quote a column name so that any name is valid in a model formula.

    patsy reads the formula as Python, so a column called memory-bias or 2back would
    break "y ~ memory-bias". Q("memory-bias") looks the column up by string instead.
    The fitted term is then named Q("memory-bias") as well, which is why two_tracts()
    uses q(outcome) again when it pulls the estimate out of the result.
    """
    return f'Q("{column}")'

# The covariate part of every formula, e.g. Q("ICV") + Q("Mean_length_mm") + ... The
# same string goes into every model so the whole-tract, quartile and mixed fits are
# adjusted identically and can be compared with each other.
RHS = " + ".join(q(c) for c in COVARIATES)

# Standardize every column of a frame: subtract the mean, divide by the SD.
# Argument: a numeric DataFrame. Returns a frame of the same shape. ddof=1 is the
# sample SD (N - 1), the same as R's scale(), so a beta from here matches lm() on scaled
# data. The outcome is standardized along with everything else, which is what makes the
# coefficients standardized betas; the intercept comes out at essentially zero.
def zscore(frame):
    return (frame - frame.mean()) / frame.std(ddof=1)

# Read one tract's analysis CSV from Step 8b and check that every column we will model
# is there. Argument: the tract label. Returns the DataFrame. Subject is read as str
# so an ID like 0123 keeps its leading zero; the mixed model groups rows by this column.
# A missing covariate or outcome is nearly always a typo in COVARIATES or --outcomes, so
# we stop and name it rather than let the column selection below fail with a KeyError.
def load(tract):
    table = pd.read_csv(ANALYSIS / f"{tract}__({METRIC})__analysis.csv", dtype={"Subject": str})
    absent = [c for c in COVARIATES + OUTCOMES if c not in table.columns]
    if absent:
        sys.exit(f"{tract}: columns not found: {'', ' '.join(absent)}")
    return table

```

```

# Average the metric over one segment's nodes, per participant.
# Arguments: the analysis table and a SEGMENTS key. Returns a Series, one value per row.
# The columns are <METRIC>_lo ... <METRIC>_(hi-1) and mean(axis=1) averages across them
# within each row. pandas skips NaN, so a participant with a few missing nodes is
# averaged over the nodes present rather than dropped; only an all-NaN segment gives
# NaN, and dropna() in the callers removes that participant.
def segment_mean(table, segment):
    lo, hi = SEGMENTS[segment]
    return table[[f"{METRIC}_{node}" for node in range(lo, hi)]].mean(axis=1)

def single_tract(tract):
    """OLS per segment, FDR across quartiles, and the uniformity contrast.

    Argument: the tract label. Returns a list of result rows (dicts): for every outcome,
    five "ols" rows (Whole, Q1-Q4) and two "uniformity" rows.
    """
    table = load(tract)
    rows = []
    for outcome in OUTCOMES:
        quartile_rows = []
        for segment in SEGMENTS:
            # A small frame with just what this model needs. .assign adds the segment
            # mean as a column called "metric"; dropna removes any participant with a
            # missing outcome, covariate or segment mean, so n can differ between
            # outcomes. Standardizing after dropna keeps the z-scores on the people who
            # are actually in the model.
            d = table[COVARIATES + [outcome]].assign(metric=segment_mean(table, segment))
            d = d.dropna()
            # outcome ~ metric + covariates by ordinary least squares. patsy adds the
            # intercept itself. .fit() with no arguments gives the usual (non-robust)
            # standard errors and t tests.
            fit = smf.ols(f"{q(outcome)} ~ metric + {RHS}", zscore(d)).fit()
            # One row per fit. nobs is the number of rows the model saw; params,
            # tvalues and pvalues are indexed by term name and "metric" is the one we
            # report. The covariate terms are fitted but not written out.
            row = {"model": "ols", "tract": tract, "segment": segment, "outcome": outcome,
                  "term": "metric", "n": int(fit.nobs), "beta": fit.params["metric"],
                  "statistic": fit.tvalues["metric"], "p": fit.pvalues["metric"]}
            rows.append(row)
            # Whole stands alone; the four quartiles are corrected together below.
            if segment != "Whole":
                quartile_rows.append(row)
        # Benjamini-Hochberg across this outcome's four quartile p values. multipletests
        # returns (reject, corrected p, alpha_sidak, alpha_bonferroni); [1] is the
        # corrected p vector, in the same order as the input. Adjacent quartiles are
        # correlated, which is why the Step 9 page asks for FDR or a max-statistic
        # permutation here; this script does FDR. The dicts in quartile_rows are the
        # same objects that sit in rows, so adding p_fdr here lands in the output table.
        adjusted = multipletests([r["p"] for r in quartile_rows], method="fdr_bh")[1]
        for row, p_fdr in zip(quartile_rows, adjusted):
            row["p_fdr"] = p_fdr

    # Uniformity contrast, seed-end quartile against target-end quartile. Entering
    # the mean of Q1 and Q4 and their difference together is just a re-expression of
    # "Q1 + Q4", but the two terms answer different questions: the mean picks up an
    # effect shared by both ends, the difference an effect that is stronger at one
    # end. Mean significant and difference near zero reads as a diffuse effect along
    # the tract; a significant difference says the two ends behave differently and
    # the quartile rows are where to look. Nothing is corrected for these two.
    q1, q4 = segment_mean(table, "Q1"), segment_mean(table, "Q4")
    d = table[COVARIATES + [outcome]].assign(mean_q1q4=(q1 + q4) / 2, diff_q4q1=q4 - q1)
    d = d.dropna()

```

```

fit = smf.ols(f"{q(outcome)} ~ mean_q1q4 + diff_q4q1 + {RHS}", zscore(d)).fit()
# Both terms are written out, labelled with segment "Q1,Q4" so they sort together.
for term in ("mean_q1q4", "diff_q4q1"):
    rows.append({"model": "uniformity", "tract": tract, "segment": "Q1,Q4",
                 "outcome": outcome, "term": term, "n": int(fit.nobs),
                 "beta": fit.params[term], "statistic": fit.tvalues[term],
                 "p": fit.pvalues[term]})

return rows

def two_tracts(tract_a, tract_b):
    """Mixed model across two tracts; likelihood-ratio test of outcome x tract.

    Arguments: the two tract labels; both analysis CSVs must exist for METRIC. Returns
    one "mixed" row per outcome and segment, carrying the outcome main effect from the
    additive model and interaction_p from the likelihood-ratio test.
    """
    tables = {tract: load(tract) for tract in (tract_a, tract_b)}
    rows = []
    for outcome in OUTCOMES:
        for segment in SEGMENTS:
            # Stack the two tracts long: two rows per participant, each with its own
            # segment mean and, through COVARIATES, its own Streamline_count and
            # Mean_length_mm (Step 6 measured those per tract, so they differ between a
            # participant's two rows). The outcome and the other covariates repeat.
            # ignore_index rennumbers the rows; dropna then removes incomplete rows, so a
            # participant can survive for one tract and not the other.
            stacked = pd.concat([
                t[["Subject", outcome] + COVARIATES].assign(metric=segment_mean(t, segment),
                                                            tract=name)
                for name, t in tables.items()], ignore_index=True).dropna()
            # Standardize on the stacked table, both tracts pooled, so a metric offset
            # between the tracts survives and is absorbed by the tract term instead of
            # being scaled away. "tract" is text, so patsy treats it as a factor and
            # codes it as one dummy against the first level in sorted order.
            numeric = ["metric", outcome] + COVARIATES
            stacked[numeric] = zscore(stacked[numeric])
            # The metric is the dependent variable here: each participant has one
            # outcome value but two metric values, one per tract, so the model has to
            # explain the metric. groups= gives every participant a random intercept
            # (statsmodels' default random-effects structure), the "(1 | participant)"
            # of the docstring. reml=False fits by maximum likelihood, which the
            # likelihood-ratio test needs: REML likelihoods are not comparable between
            # models with different fixed effects. "main" is the additive model and
            # "full" adds the outcome x tract interaction (patsy's "*" is a + b + a:b).
            # mixedlm is chatty with small samples. The usual ConvergenceWarning is "The
            # MLE may be on the boundary of the parameter space", which means the
            # participant variance came out near zero (statsmodels warns below 0.01).
            # Warnings are muted for these two fits only. If an estimate looks odd,
            # refit it by hand outside the with-block and read what it was complaining
            # about.
            with warnings.catch_warnings():
                warnings.simplefilter("ignore")
                main = smf.mixedlm(f"metric ~ {q(outcome)} + tract + {RHS}", stacked,
                                  groups=stacked["Subject"]).fit(reml=False)
                full = smf.mixedlm(f"metric ~ {q(outcome)} * tract + {RHS}", stacked,
                                  groups=stacked["Subject"]).fit(reml=False)
            # Likelihood-ratio statistic, 2 * (logL_full - logL_main), on 1 df because
            # the interaction adds a single coefficient. The smaller model cannot fit
            # better in theory, but the optimiser can stop a hair short of the maximum
            # and give a tiny negative; max(..., 0) clamps that. chi2.sf is the upper
            # tail of the chi-square distribution, so it is the p value directly.
            lr = max(2 * (full.llf - main.llf), 0)

```

```

# The main effect is read from the additive model, as the docstring says.
# Its term is named Q("<outcome>"), e.g. Q("memory_accuracy"), hence the
# q(outcome) lookup. n counts participants, not rows, so it lines up with
# the OLS rows.
rows.append({"model": "mixed", "tract": f"{tract_a}+{tract_b}", "segment": segment,
            "outcome": outcome, "term": outcome,
            "n": stacked["Subject"].nunique(), "beta": main.params[q(outcome)],
            "statistic": main.tvalues[q(outcome)], "p": main.pvalues[q(outcome)],
            "interaction_p": chi2.sf(lr, 1)})

return rows

# Run. One single-tract block per tract, so a two-tract call gives both sets of OLS and
# uniformity rows first and then the mixed rows. *TRACTS unpacks the two-element list
# into the two arguments two_tracts() takes.
results = []
for tract in TRACTS:
    results += single_tract(tract)
if len(TRACTS) == 2:
    results += two_tracts(*TRACTS)

# Write and show. The row dicts become one DataFrame; columns that only some rows carry
# (p_fdr, interaction_p) are empty elsewhere, and the model column says which kind of row
# each one is. The file name joins the tract labels with "+", so a two-tract run sits
# next to the single-tract files instead of overwriting one. option_context widens the
# printout and rounds to three decimals for the terminal only; the CSV keeps full
# precision, and index=False leaves out pandas' row numbers.
results = pd.DataFrame(results)
label = "+".join(TRACTS)
path = ANALYSIS / f"{label}__{METRIC}__tract_models.csv"
results.to_csv(path, index=False)
with pd.option_context("display.width", 200, "display.float_format", "{:.3f}".format):
    print(results.to_string(index=False))
print(f"\n-> {path}")

```

09b_nodewise_permutation.R

```

# =====
# Step 9b. Node-wise cluster-extent permutation test (Freedman-Lane)
# =====
# At each node: outcome ~ metric_node + covariates. The residuals of the
# reduced model (covariates only) are permuted, all nodes are refitted, and the
# maximum cluster extent of each permutation forms the null distribution.
# Adapted from a reference implementation used in atlas construction.
#
# What it does. Step 9a asked whether the metric relates to the outcome over the whole
# tract or within quartiles; this script asks WHERE along the tract. One linear model
# per node (100 of them) gives 100 t values; adjacent nodes with p < .05 are joined into
# clusters; the question is then whether a cluster that long could arise by chance.
# Permutation answers it: shuffle the part of the outcome the covariates do not explain,
# refit all 100 nodes, keep the longest cluster, repeat N_PERMUTATIONS times. A real
# cluster passes when no more than 5% of the shuffles produced one at least as long.
# Nothing is standardized here, so Estimate is in outcome units per unit of the metric;
# the 9a betas are standardized, so do not compare the two directly.
#
# Usage (in a shell where 00_config.sh has been sourced):
# Rscript 09b_nodewise_permutation.R <analysis.csv> <outcome> <METRIC>_ <out_dir> <label>
# 1. analysis.csv wide analysis file from Step 8b
# 2. outcome outcome column, e.g. memory_accuracy
# 3. METRIC_ node-column prefix including the underscore, e.g. NDI_
# 4. out_dir directory for the results

```

```

# 5. label <tract>__<metric>__<outcome>, used in the output file names
# Environment: COVARIATES (comma-separated columns), N_PERMUTATIONS (default 5000),
# R_PERM_CORES (default: all cores but one)
# Outputs: <label>_nodewise.csv, <label>_clusters.csv, <label>_summary.csv
#
# One call does one outcome for one tract and one metric, so the Step 9 page loops in
# the shell:
# for outcome in memory_accuracy memory_bias; do
#   Rscript 09b_nodewise_permutation.R "$OUT/analysis/${TRACT}__NDI__analysis.csv" \
#     "$outcome" NDI_ "$OUT/permutation" "${TRACT}__NDI__${outcome}"
# done
# The label is not parsed here, but Step 9c splits it on the double underscore to get
# tract, metric and outcome back, so keep that exact shape.
#
# Needs:
# $OUT/analysis/<TRACT>__<METRIC>__analysis.csv Step 8b: one row per participant,
# the covariate and outcome columns, Streamline_count, Mean_length_mm and the
# node columns <METRIC>_0 ... <METRIC>_99
# R with readr, tibble, foreach, doParallel (dplyr and stringr are attached below
# but nothing calls them)
# Writes, into <out_dir> (normally $OUT/permutation):
# <label>_nodewise.csv one row per node: Node, Estimate, t_value, p_value, df, n
# <label>_clusters.csv one row per observed cluster, with its permutation p value
# <label>_summary.csv one row: sample size, settings, counts, extent threshold
# Step 9c stacks these three into results_long.csv for the Explorer.
#
# Runtime: a few minutes per call on one core at 5,000 permutations; more cores cut that
# (the default is every core but one). The seed is fixed and the shuffles are drawn
# before the workers start, so the same N gives the same numbers whatever the core count.
# What to look for. The last line printed is "Done: <label> - clusters=K sig nodes=M
# ext_thr=T": K clusters were formed, M nodes had p < .05, and a cluster needs at least
# T nodes to pass. Open _clusters.csv and look at PassExtentThreshold; TRUE rows are
# the ones that survive. In the example dataset the clusters sat at nodes 27 to 55 and
# moved when the covariate list changed, which is why the page treats node-wise results
# as description and lets the whole-tract model carry the inference. "Dropping N
# incomplete participants" means N rows had an NA in the outcome, a covariate or a node;
# if N is large, look at the analysis file before trusting anything else.
# =====
# suppressPackageStartupMessages hides the "Attaching package" chatter so the terminal
# shows only this script's own messages. readr reads and writes the CSVs, tibble builds
# the output tables, foreach + doParallel + parallel run the permutations on several
# cores. dplyr and stringr are attached but nothing below calls them; on an R install
# without them, those two library() calls are the only place this script would fail.
suppressPackageStartupMessages({
  library(readr); library(dplyr); library(stringr)
  library(foreach); library(doParallel); library(parallel); library(tibble)
})

# Positional arguments. commandArgs(trailingOnly = TRUE) returns only what came after
# the script name, so args[1] is the analysis CSV. stopifnot aborts with a message if
# fewer than five were given; extra ones are ignored.
args <- commandArgs(trailingOnly = TRUE)
stopifnot(length(args) >= 5)
data_csv <- args[1]
response_col <- args[2]
metric_prefix <- args[3] # "NDI_", underscore included; "NDI" alone matches nothing
out_dir <- args[4]
base <- args[5] # <tract>__<metric>__<outcome>; every output name starts with it

# Covariates: comma-separated column names, exported by 00_config.sh
# Sys.getenv returns "" when the variable is missing; stopping here with a plain message
# beats a cryptic "object not found" from lm later on. strsplit returns a list with one
# element per input string, and [[1]] takes the vector for our single string. Nothing

```

```

# is trimmed, so "ICV, age" (with a space) would look for a column named " age".
# The same list is used by Step 9a, so the three resolutions are comparable.
# Streamline_count is in the default list and correlates strongly with NDI (see
# 00_config.sh); decide before running whether it stays in, and report both.
covariate_env <- Sys.getenv("COVARIATES", unset = "")
if (covariate_env == "") stop("COVARIATES is not set: run `source 00_config.sh` first")
covariate_cols <- strsplit(covariate_env, ",")[1]

# Settings. alpha_node is the per-node threshold that decides which nodes join a
# cluster; alpha_familywise is the cluster-level threshold applied to the permutation
# p values. Both are .05 as on the page. Lowering alpha_node makes clusters shorter
# and rarer in the real data and in the null alike, so the extent threshold drops too;
# it is a cluster-forming choice, not a correction. N_PERMUTATIONS comes from
# 00_config.sh (5,000); the cluster p values are proportions of that many draws, so
# their resolution is 1/N. rng_seed fixes the shuffles, so a rerun reproduces the
# output exactly. use_parallel is not read anywhere below; the core count further down
# is what decides between the parallel and serial paths.
alpha_node <- 0.05
alpha_familywise <- 0.05
num_permutations <- as.integer(Sys.getenv("N_PERMUTATIONS", unset = "5000"))
rng_seed <- 123
use_parallel <- TRUE

# recursive = TRUE creates missing parents (like mkdir -p); showWarnings = FALSE keeps
# quiet when the directory already exists, which it will from the second outcome on.
dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

# show_col_types = FALSE silences readr's column-type printout (over 100 columns here).
dat <- readr::read_csv(data_csv, show_col_types = FALSE)

# Collect node columns
# The pattern anchors the prefix at the start and allows only digits to the end (\\d+$),
# so NDI_0 ... NDI_99 match and something like NDI_mean would not. value = TRUE returns
# the matching names rather than their positions.
node_cols <- grep(paste0("^", metric_prefix, "\\d+$"), names(dat), value = TRUE)
if (length(node_cols) == 0) {
  stop("No columns matching ", metric_prefix, "0.. in: ", data_csv)
}
# Strip the prefix to get the node number, then sort numerically. read_csv hands the
# columns back in file order, which is already 0..99 from Step 8b, but ordering by the
# integer guards against a file where NDI_10 sits before NDI_2. The cluster code below
# relies on node_idx increasing along the vector.
node_idx <- as.integer(sub(paste0("^", metric_prefix), "", node_cols))
ord <- order(node_idx)
node_cols <- node_cols[ord]; node_idx <- node_idx[ord]
num_nodes <- length(node_cols)

# Numeric casts
# A stray text value ("missing", ".", a note) makes readr read the whole column as text;
# as.numeric turns the numbers back and the text into NA, which the complete-case
# filter below then drops. Expect a "NAs introduced by coercion" warning when that
# happens; it is telling you which column to clean.
dat[[response_col]] <- as.numeric(dat[[response_col]])
for (cc in covariate_cols) dat[[cc]] <- as.numeric(dat[[cc]])

# Drop incomplete
# complete.cases is TRUE for rows with no NA in any of the listed columns: the outcome,
# every covariate and every node. A participant missing even one node is dropped
# entirely, so that all 100 fits use the same people and the permutation, which
# shuffles one residual vector for all nodes at once, stays valid. "if (n_dropped)"
# treats 0 as FALSE. message() writes to stderr, so add 2>&1 if you redirect to a log.
all_model_cols <- c(response_col, covariate_cols, node_cols)
mask <- complete.cases(dat[, all_model_cols])

```

```

n_dropped <- sum(!mask)
if (n_dropped) message("Dropping ", n_dropped, " incomplete participants")
dat <- dat[mask, , drop = FALSE]
n_subj <- nrow(dat)
# Five is a sanity floor, not a power check: with the default five covariates the full
# model has seven coefficients (intercept, node, covariates) and needs more rows than
# that to have any residual df at all. quit(status = 0) exits cleanly, so a shell loop
# running under set -e is not aborted; the missing output files are the sign that this
# outcome was skipped.
if (n_subj < 5) {
  message("Fewer than 5 complete participants (", n_subj, ") - skipping ", base)
  quit(status = 0)
}

y <- dat[[response_col]]

# Formulas
# Built as text so the covariate list can be anything. "node" is the column the fit
# functions below create for the current node's metric; y is the outcome. The full
# model tests the node; the reduced model, without it, is what gets permuted.
full_formula <- as.formula(
  paste("y ~ node +", paste(covariate_cols, collapse = " + "))
)
red_formula <- as.formula(
  paste("y ~", paste(covariate_cols, collapse = " + "))
)

# Nodewise fit
# One ordinary least squares fit of the full model for one node.
# y outcome vector, all complete participants
# node that node's metric, one value per participant
# covariates_df data frame holding the covariate columns
# Returns a named vector: Estimate (slope of y on the node metric), t, two-sided p,
# residual df and n. The statistics come back NA when the node cannot be tested: fewer
# than three rows, a metric that is constant across participants (no variance, no
# slope), or a "node" term that lm dropped because it is perfectly collinear with a
# covariate (summary() leaves aliased terms out of its coefficient table, hence the
# rownames check). Such nodes count as not significant and break any cluster.
fit_node_full <- function(y, node, covariates_df) {
  df0 <- data.frame(y = y, node = node, covariates_df)
  df0 <- df0[complete.cases(df0), ]
  if (nrow(df0) < 3 || sd(df0$node) == 0)
    return(c(Estimate=NA, t=NA, p=NA, df=NA, n=nrow(df0)))
  fit <- lm(full_formula, data = df0)
  sm <- summary(fit)$coefficients
  if (!("node" %in% rownames(sm)))
    return(c(Estimate=NA, t=NA, p=NA, df=fit$df.residual, n=nrow(df0)))
  # summary() already reports Pr(>|t|); recomputing it as 2*pt(-|t|, df) gives the same
  # two-sided p and is the exact formula the permutation loop uses, so the observed and
  # null p values are on the same footing. df.residual is n minus the coefficient count.
  tval <- unname(sm["node", "t value"])
  est <- unname(sm["node", "Estimate"])
  pval <- 2*pt(-abs(tval), df=fit$df.residual)
  c(Estimate=est, t=tval, p=pval, df=fit$df.residual, n=nrow(df0))
}

# Observed statistics, one fit per node. lm is deterministic, so this set.seed changes
# nothing; the seed that matters is reset just before the shuffles are drawn. lapply
# returns a list of named vectors and do.call(rbind, ...) stacks them into a matrix
# with one row per node and the names as columns.
set.seed(rng_seed)
node_stats <- lapply(seq_along(node_cols), function(i) {
  fit_node_full(y, dat[[node_cols[i]]], dat[, covariate_cols, drop=FALSE])
})

```

```

})
node_stats <- do.call(rbind, node_stats)
# First output. Step 9c reads Node, t_value and p_value from this file, and the
# Explorer draws t_value along the tract with the p < .05 nodes highlighted.
node_stats_df <- tibble(
  Node=node_idx, Estimate=node_stats[, "Estimate"],
  t_value=node_stats[, "t"], p_value=node_stats[, "p"],
  df=node_stats[, "df"], n=node_stats[, "n"]
)
write_csv(node_stats_df, file.path(out_dir, paste0(base, "_nodewise.csv")))

# Cluster helpers
# Group significant nodes into runs of consecutive node numbers.
# sig          logical, one entry per node, TRUE where p < alpha_node
# nodes_numeric the node numbers in the same order (node_idx)
# Returns a list of integer vectors, one per cluster, each holding the node numbers in
# that run; an empty list when nothing is significant. Adjacency is judged on the node
# NUMBER (k follows k-1), not on position in the vector, so a gap in the numbering ends
# a cluster. A lone significant node is a cluster of size 1. The same function serves
# the observed data and every permutation, so the cluster rule cannot drift apart.
clusters_from_sig <- function(sig, nodes_numeric) {
  if (!any(sig)) return(list())
  idx <- which(sig); cls <- list(); run <- c(nodes_numeric[idx[1]])
  if (length(idx) > 1) for (k in 2:length(idx)) {
    if (nodes_numeric[idx[k]] == nodes_numeric[idx[k-1]] + 1)
      run <- c(run, nodes_numeric[idx[k]])
    else { cls[[length(cls)+1]] <- run; run <- c(nodes_numeric[idx[k]]) }
  }
  cls[[length(cls)+1]] <- run; cls
}
# Length of the longest cluster, or 0 when there is none. This one number per
# permutation is the whole null distribution. vapply(..., 1L) promises one integer per
# cluster, so max() gets an integer vector rather than a list.
max_cluster_size_from_sig <- function(sig, nodes_numeric) {
  cls <- clusters_from_sig(sig, nodes_numeric)
  if (length(cls)==0) 0L else max(vapply(cls, length, 1L))
}

# Observed clusters. Nodes with an NA p (untestable) are treated as not significant,
# so they end a cluster rather than being skipped over.
sig_mask <- !is.na(node_stats_df$p_value) & node_stats_df$p_value < alpha_node
obs_clusters <- clusters_from_sig(sig_mask, node_stats_df$Node)
obs_sizes <- vapply(obs_clusters, length, 1L)
obs_max_size <- if (length(obs_sizes)) max(obs_sizes) else 0L
num_sig_nodes <- sum(sig_mask)
num_clusters <- length(obs_clusters)

# Freedman-Lane
# Fit the reduced model (covariates only) once on the real outcome and keep its fitted
# values and residuals. Each permuted outcome is fitted + shuffled residuals: the part
# of y the covariates explain stays with its owner, only the leftover is reassigned
# across participants. That is what makes the null hold for the node term with the
# covariates still in the model (Freedman & Lane, 1983; Winkler et al., 2014).
# Shuffling y outright would also scramble its relation to the covariates, so the null
# would no longer describe data with those covariates in place.
df_red_global <- data.frame(y=y, dat[, covariate_cols, drop=FALSE])
fit_red <- lm(red_formula, data=df_red_global)
yhat_red <- fitted(fit_red); resid_red <- resid(fit_red)

# Nodes whose metric is constant across participants have no slope to test; they keep p = 1
# in every permutation. Checked once here rather than 5,000 x 100 times in the loop.
analysable <- vapply(seq_along(node_cols), function(i)
  sd(dat[[node_cols[i]]]) > 0, logical(1))

```

```

# All permutations are drawn once, here, so that the null distribution is identical
# for any number of cores and any scheduling of the parallel workers.
# sample.int(n) is a random ordering of 1..n; vapply stacks the draws into an n_subj by
# num_permutations integer matrix, one shuffle per column. Drawing inside the workers
# instead would give each worker its own RNG stream and the result would change with
# the core count; the validation on the Scripts page checked 1, 3 and 4 cores agree.
set.seed(rng_seed)
perm_mat <- vapply(seq_len(num_permutations), function(k) sample.int(n_subj),
                  integer(n_subj))

# One permutation. Takes the permutation number, builds the permuted outcome, refits
# the full model at every analysable node, and returns the length of the longest run of
# nodes with p < alpha_node (an integer, 0 if none). Same model, same p formula and
# same cluster rule as the observed pass, which is what makes the two comparable. Only
# the p values are kept; estimates from permuted data mean nothing. The leading dot in
# .perm is only a naming convention, so the argument and the foreach loop variable perm
# further down do not share a name.
perm_fun <- function(.perm) {
  perm_idx <- perm_mat[, .perm]
  y_perm <- yhat_red + resid_red[perm_idx]
  p_perm <- rep(1, num_nodes)
  for (i in which(analysable)) {
    df_perm <- data.frame(y=y_perm, node=dat[[node_cols[i]]],
                        dat[, covariate_cols, drop=FALSE])
    fit <- lm(full_formula, data=df_perm)
    sm <- summary(fit)$coefficients
    if ("node" %in% rownames(sm)) {
      tval <- sm["node", "t value"]
      p_perm[i] <- 2*pt(-abs(tval), df=fit$df.residual)
    }
  }
  max_cluster_size_from_sig(p_perm < alpha_node, node_idx)
}

# Core count. R_PERM_CORES in the shell overrides. Anything that is not a number
# (unset, "", "all") comes out of as.integer as NA and suppressWarnings hides the
# coercion warning; NA or anything below 1 falls back to every core but one.
# detectCores counts logical CPUs, so on a shared machine set R_PERM_CORES to something
# polite.
env_cores <- suppressWarnings(as.integer(Sys.getenv("R_PERM_CORES", unset = NA)))
if (!is.na(env_cores) && env_cores > 0L) {
  cores <- env_cores
} else {
  cores <- max(1L, parallel::detectCores() - 1L)
}

# Run the permutations. makeCluster starts that many separate R processes (PSOCK
# workers, the default type) and registerDoParallel points foreach's %dopar% at them.
# foreach(perm = 1:N) %dopar% expr evaluates expr once per value of perm, spread over
# the workers. foreach finds the variables the body needs by walking perm_fun and the
# functions it calls, and ships them (dat, perm_mat, node_idx, ...) to each worker, so
# no .export is needed. .packages = "stats" loads the package holding lm and pt on the
# workers; they attach it at startup anyway, so this is belt and braces. .combine = c
# joins the returned integers into one vector instead of a list. stopCluster shuts the
# workers down; without it they linger until R exits. With one core the same function
# runs through a plain vapply. If something inside perm_fun fails, all that comes back
# is 'task N failed - "<message>"' with no hint of where; rerun with R_PERM_CORES=1 and
# R names the failing call.
if (cores > 1L) {
  cl <- parallel::makeCluster(cores); registerDoParallel(cl)
  perm_max_sizes <- foreach(perm = 1:num_permutations, .combine = c,
                          .packages = "stats") %dopar% perm_fun(perm)
}

```

```

parallel::stopCluster(cl)
} else {
  perm_max_sizes <- vapply(1:num_permutations, perm_fun, integer(1))
}

# Cluster p value: proportion of permutations whose largest cluster is at least as long.
# The extent threshold is the smallest extent with p <= alpha. Extents are whole
# numbers, so this is one node above the (1 - alpha) quantile; a cluster exactly at the
# quantile has p > alpha and does not pass.
# quantile(type = 1) is the inverse of the empirical distribution function: it returns
# an actual value from perm_max_sizes, never an interpolated fraction, so the + 1 lands
# on a whole node. The p value uses >=, so a null cluster exactly as long as the
# observed one counts against it. When no permutation reached the observed length the
# proportion comes out 0, which only means below 1/N_PERMUTATIONS, the smallest p these
# draws can resolve. The threshold is one number per analysis, written to the summary
# and copied into every cluster row.
cluster_p_from_size <- function(size) mean(perm_max_sizes >= size)
extent_threshold <- as.integer(quantile(perm_max_sizes, probs = 1 - alpha_familywise,
                                     type = 1)) + 1L

# Second output, one row per observed cluster. match() finds each cluster node's row in
# the node table so its t value and estimate can be pulled out. Direction is the sign
# of the mean t; MaxAbsTNode is where |t| peaks, handy when reporting a location.
# PassExtentThreshold is computed from the p value, not from Size >= threshold, so the
# two can never disagree. Step 9c keeps only the TRUE rows and takes the smallest
# ClusterPValue among them for the Explorer.
if (num_clusters > 0) {
  all_clusters_df <- do.call(rbind, lapply(seq_along(obs_clusters), function(k) {
    nodes <- obs_clusters[[k]]; idx_in_table <- match(nodes, node_stats_df$Node)
    t_vals <- node_stats_df$t_value[idx_in_table]
    ests <- node_stats_df$Estimate[idx_in_table]
    tibble(
      ClusterID=k, Size=length(nodes), StartNode=min(nodes), EndNode=max(nodes),
      Nodes=paste(nodes, collapse=","),
      MeanTValue=mean(t_vals, na.rm=TRUE),
      Direction=ifelse(mean(t_vals, na.rm=TRUE)>0,"Positive","Negative"),
      MaxAbsTValue=t_vals[which.max(abs(t_vals))],
      MaxAbsTNode=nodes[which.max(abs(t_vals))],
      MeanEstimate=mean(ests, na.rm=TRUE),
      ClusterPValue=cluster_p_from_size(length(nodes)),
      ExtentThresholdNodes=extent_threshold,
      PassExtentThreshold=cluster_p_from_size(length(nodes)) <= alpha_familywise
    )
  })))
} else {
  # No clusters at all: write a header-only file with the same columns and types, so
  # Step 9c and anything else reading the directory finds the file it expects.
  all_clusters_df <- tibble(
    ClusterID=integer(), Size=integer(), StartNode=integer(), EndNode=integer(),
    Nodes=character(), MeanTValue=double(), Direction=character(),
    MaxAbsTValue=double(), MaxAbsTNode=integer(),
    MeanEstimate=double(),
    ClusterPValue=double(), ExtentThresholdNodes=integer(), PassExtentThreshold=logical()
  )
}
write_csv(all_clusters_df, file.path(out_dir, paste0(base, "_clusters.csv")))

# Third output, one row that records what was modelled, on how many participants, with
# which settings, and what came out. Step 9c reads N_subjects, Covariates and
# ExtentThresholdNodes from here (the column names are fixed; it looks them up by
# name). N_subjects is the count after the incomplete rows went; N_dropped says how
# many that was. NodesTested should be 100.
summary_df <- tibble(

```

```

Outcome=response_col,
MetricPrefix=metric_prefix,
Covariates=paste(covariate_cols, collapse=", "),
N_subjects=n_subj, N_dropped=n_dropped,
NodesTested=num_nodes,
NodewiseAlpha=alpha_node, FamilywiseAlpha=alpha_familywise,
NumPermutations=num_permutations,
NumNodewiseSignificant=num_sig_nodes,
NumClustersFormed=num_clusters,
ObservedMaxClusterSize=obs_max_size,
ExtentThresholdNodes=extent_threshold,
NumClustersPassingExtent=sum(all_clusters_df$PassExtentThreshold, na.rm=TRUE)
)
write_csv(summary_df, file.path(out_dir, paste0(base, "_summary.csv")))
# Last line on the terminal; the numbers repeat NumClustersFormed, NumNodewiseSignificant
# and ExtentThresholdNodes from the summary, so a quick scan of the shell loop's output
# tells you which outcomes produced anything.
message("Done: ", base, " - clusters=", num_clusters,
        " sig nodes=", num_sig_nodes, " ext_thr=", extent_threshold)

```

09c_stack_for_explorer.py

```

#!/usr/bin/env python3
"""Step 9c. Stack the node-wise results into the file read by the Explorer.

```

Step 9b writes three small CSVs per analysis (one outcome x one tract x one metric) and nothing ties them together. The Explorer wants one long table with a row per node and the analysis-level facts (N, covariates, extent threshold, whether a cluster passed) repeated on every row of that analysis, so this script walks the permutation directory, joins the three files of each analysis and stacks the lot into results_long.csv. Plain pandas, reads no environment variables, ignores FORCE; it recomputes on every run and takes well under a second for a few dozen analyses.

```

python 09c_stack_for_explorer.py /path/to/permutation/results
python 09c_stack_for_explorer.py "$OUT/permutation"      # the usual call

```

The only argument is the directory Step 9b wrote to; with no argument the current directory is used. Every analysis in that directory is picked up, whatever its tract, metric or outcome, so run 9b for everything you want to see in the Explorer first and then run this once.

```

Expects the three outputs of 09b_nodewise_permutation.R for every analysis, named
<tract>_<metric>_<outcome>_nodewise.csv, _clusters.csv and _summary.csv.
    $OUT/permutation/<label>_summary.csv      one row; we read N_subjects, Covariates and
                                                ExtentThresholdNodes
    $OUT/permutation/<label>_nodewise.csv      one row per node; we read Node, t_value, p_value
    $OUT/permutation/<label>_clusters.csv      one row per cluster (header only if none); we
                                                read ClusterPValue and PassExtentThreshold

```

The _summary.csv files drive the loop. A summary whose _nodewise or _clusters partner is missing stops the run with a FileNotFoundError rather than being skipped; rerun 9b for that analysis and try again.

Output: results_long.csv in the same directory, one row per node, with columns
 outcome, tract, metric, node, t, p, hemisphere, N, covariates,
 extent_threshold, cluster_p, passed
 hemisphere is L or R when the tract name contains the token l or r (l_vta_l_hipp,
 anterior_r_vta_r_hipp) and is otherwise empty; cluster_p and passed
 describe the largest cluster of the analysis that met the extent threshold.
 An existing results_long.csv is overwritten without asking.

What to look at when it finishes. One line, "wrote ../results_long.csv (N rows,

K analyses)". K should equal the number of 9b calls you made and N should be 100 x K. A "skipped" line above it names a summary file whose name did not split into three double-underscore fields: a 9b label with fewer than two "__" in it, or some other *_summary.csv sitting in the directory. Extra "__" are not caught (see the loop).

Where the file goes next: open the Explorer (the Explorer page of the tutorial) and load results_long.csv there; it is read in the browser and not uploaded anywhere. The Explorer derives clusters and significant-node counts from the t and p columns itself, marks clusters at least extent_threshold nodes long as passing and takes its FWE verdict from that; N, covariates and cluster_p are shown as given. passed is only read when extent_threshold is absent, but 9b defines both by the same rule, so they agree.

```
"""
import sys
from pathlib import Path

# Standard library plus pandas; nothing from the imaging stack is needed for this step.
import pandas as pd

# sys.argv[1] is the first word after the script name on the command line. Falling back
# to "." lets you cd into $OUT/permutation and run the script bare. Path() gives us the
# / operator and .glob() below instead of string concatenation.
results_dir = Path(sys.argv[1] if len(sys.argv) > 1 else ".")
# One dict per node row, collected here and turned into a DataFrame once at the end;
# appending to a DataFrame inside the loop would copy the whole table every time.
rows = []

# One analysis per _summary.csv. sorted() fixes the row order of results_long.csv run to
# run; glob's own order is whatever the filesystem hands back.
for summary_file in sorted(results_dir.glob("*_summary.csv")):
    # Strip the suffix to recover the label 9b was called with, then split it on the
    # double underscore into tract, metric and outcome. The 2 is maxsplit: at most two
    # splits, so an outcome containing "__" survives intact, but a tract or metric with
    # "__" in it is mis-split without complaint (a__b__NDI__out gives tract a, metric b,
    # outcome NDI__out), so keep "__" out of TRACT. Fewer than two "__" gives fewer
    # than three parts, which is what the check catches. Skipping rather than stopping
    # keeps one odd file from blocking the rest.
    label = summary_file.name[:-len("_summary.csv")]
    parts = label.split("__", 2)
    if len(parts) != 3:
        print(f"skipped {summary_file.name}: label is not <tract>__<metric>__<outcome>")
        continue
    tract, metric, outcome = parts

    # The summary file has exactly one row, so .iloc[0] turns it into a Series and the
    # summary[...] lookups below return scalars rather than one-element columns.
    summary = pd.read_csv(summary_file).iloc[0]
    nodewise = pd.read_csv(results_dir / f"{label}_nodewise.csv")
    clusters = pd.read_csv(results_dir / f"{label}_clusters.csv")
    # Clusters whose extent survived the permutation test (cluster p <= .05 in 9b).
    # readr writes R logicals as TRUE/FALSE; pandas reads those back as a boolean column
    # when there are rows and as an empty object column when 9b formed no
    # clusters. Casting to str and upper-casing makes the comparison work either way.
    passing = clusters[clusters["PassExtentThreshold"].astype(str).str.upper() == "TRUE"]

    # Hemisphere from the tract name: the first underscore-separated token that is
    # exactly l or r (l_vta_l_hipp -> L, anterior_r_vta_r_hipp -> R). A tract without
    # such a token gets "" and the Explorer shows it with no hemisphere label. The
    # left-right panel pairs an L analysis with an R one whose tract name matches once
    # the l/r tokens are stripped, same outcome and metric, so name the two sides alike.
    # next() with a second argument returns the first match or that default if none.
    tokens = tract.lower().split("_")
    hemisphere = next((t.upper() for t in tokens if t in ("l", "r")), "")
```

```

# One output row per node. itertuples() yields each row as a namedtuple, so the 9b
# columns are attributes (node.Node, node.t_value). The analysis-level values are
# repeated on every row on purpose: the Explorer reads them off the node rows. The
# Explorer would accept 9b's Node/t_value/p_value names too; the short names are
# the documented format.
for node in nodewise.itertuples():
    rows.append({
        "outcome": outcome, "tract": tract, "metric": metric,
        # t and p are rounded to keep the file readable. A node 9b could not fit
        # (metric constant across participants, or collinear with the covariates) is
        # NA in R, NaN here, and lands in the CSV as an empty cell.
        "node": int(node.Node), "t": round(node.t_value, 4), "p": round(node.p_value, 5),
        # N is the number of participants 9b actually modelled, those with a complete
        # outcome, covariates and profile, not the length of $SUBJECTS_FILE.
        "hemisphere": hemisphere, "N": int(summary["N_subjects"]),
        "covariates": summary["Covariates"],
        # Smallest cluster length (adjacent nodes with p < .05) whose permutation p is
        # <= .05, so any cluster at least this long passes; same number on every row.
        "extent_threshold": int(summary["ExtentThresholdNodes"]),
        # cluster_p is the smallest p among the passing clusters, which is the p of
        # the longest one since 9b's cluster p only falls as extent grows; "" when
        # nothing passed. passed is 1 if at least one cluster passed, else 0. Both
        # describe the analysis, not the node, so a node outside every cluster
        # carries them too.
        "cluster_p": passing["ClusterPValue"].min() if len(passing) else "",
        "passed": int(len(passing) > 0),
    })

# No rows means no *_summary.csv here (wrong directory, or 9b has not run yet) or every
# one was skipped. sys.exit with a string prints it to stderr and exits with status 1,
# so nothing is written.
if not rows:
    sys.exit(f"no *_summary.csv files found in {results_dir}")
# index=False stops pandas from writing its 0..N-1 row numbers as an unnamed first
# column. That matters here because the Explorer treats any column it does not know as a
# grouping variable: a filter, and part of what tells one analysis from the next, so a
# column of row numbers would split every node into its own analysis.
out = results_dir / "results_long.csv"
pd.DataFrame(rows).to_csv(out, index=False)
# The analysis count assumes 100 nodes per analysis (integer division). If the row count
# is not a whole multiple of 100, some analysis was profiled at a different node count.
print(f"wrote {out} ({len(rows)} rows, {len(rows) // 100} analyses)")

```

00b_fod_estimation.sh

```

#!/bin/bash
# =====
# Step 0b. Fibre orientation distributions (multi-shell multi-tissue CSD)
# =====
# Run once, before Step 5, when preprocessing ended at the tensor.
# Phase 1 (per participant) response functions: dwi2response dholander
# Phase 2 (group) average response functions: responsemean
# Phase 3 (per participant) dwi2fod msmt_csd, then mtnormalise
# Single-shell data support two tissues only: remove the GM response and GM FOD
# arguments from the dwi2fod and mtnormalise calls.
#
# What this does. Turns each participant's preprocessed diffusion series into the
# normalized white-matter FOD image that tckgen tracks on in Steps 4 and 5. Each
# participant gets their own WM/GM/CSF response functions, those get averaged across
# the sample, and the average is what every participant's deconvolution uses. That is
# the point of the three phases: one shared set of response functions keeps FOD

```

```

# amplitudes comparable across people, and Steps 4 and 5 apply one CUTOFF (0.01 in the
# shipped config) to everyone. If your preprocessing already produced wm_fod_norm.mif,
# skip this script entirely.
#
# Needs, per participant, in $PROJECT/dwi/<ID>/:
#   data.nii.gz   bvals   bvecs   nodif_brain_mask.nii.gz
# plus the ID list in $SUBJECTS_FILE and MRtrix3 on the PATH.
#
# Writes, per participant, in $PROJECT/dwi/<ID>/:
#   dwi.mif, mask.mif           MRtrix copies of the inputs
#   wm_response.txt gm_response.txt csf_response.txt
#   wm_fod.mif gm_fod.mif csf_fod.mif           raw FODs
#   wm_fod_norm.mif gm_fod_norm.mif csf_fod_norm.mif   after mtnormalise
# and at the group level $PROJECT/dwi/group_{wm,gm,csf}_response.txt.
# Only wm_fod_norm.mif is read downstream (04_tune_cutoff.sh, 05_tractography.sh).
# Everything else stays on disk for inspection and reruns. The log goes to
# $OUT/logs/00b_fod_estimation.sh.log.
#
# How to run: edit 00_config.sh, then
#   bash 00b_fod_estimation.sh
#   FORCE=1 bash 00b_fod_estimation.sh           # recompute everything
# Expect hours on a full sample; the three-participant validation run got from here
# through Step 8b in under an hour on four threads. Use tmux or a job scheduler.
# Safe to rerun after an interruption: participants that already finished are skipped.
#
# How to tell it worked: the audit table at the very end should say "ok" on every row,
# and there should be no "!!" lines in $OUT/logs/00b_fod_estimation.sh.log below the
# "==" date line that starts this run (the log appends, so older runs sit above it).
# Then load one or two wm_fod_norm.mif files in mrview (ODF display tool) and check
# that the FOD lobes follow the obvious white-matter bundles.
# =====
# $0 is this script's path; dirname strips the file name, so the config is found next
# to the script no matter where you run it from. $(...) drops a command's output in.
source "$(dirname "$0")/00_config.sh"
# Both from 00_config.sh: copy everything printed from here on into the log (it appends,
# so earlier runs stay in the file), and load the participant IDs into SUBJECTS.
start_log "$0"
read_subjects

# Phase 1 worker. Converts one participant's data to MRtrix format and estimates the
# three tissue response functions. Argument: participant ID. Writes dwi.mif, mask.mif
# and {wm,gm,csf}_response.txt in $PROJECT/dwi/<ID>/. It runs as a background job
# (see the Phase 1 loop), so a "return" only ends that participant's job.
estimate_response() {
    # "local" keeps these names inside the function instead of leaking into the script's
    # own variables. $1 is the first argument (the ID).
    local s=$1
    local d="$PROJECT/dwi/$s"
    local f
    # Input check. [ ! -f path ] is true when the file is absent. A missing file prints a
    # "!!" line and skips the participant instead of letting mrconvert fail mid-batch.
    for f in data.nii.gz bvals bvecs nodif_brain_mask.nii.gz; do
        if [ ! -f "$d/$f" ]; then
            echo "!! $s missing $d/$f"
            return
        fi
    done
    # Restart guard. wm_response.txt is what this script treats as "done"; skip unless the
    # run was started with FORCE=1 (00_config.sh picks it up from the environment).
    if [ "$FORCE" != 1 ] && [ -f "$d/wm_response.txt" ]; then
        echo "== $s response functions exist"
        return
    fi
}

```

```

# Convert to MRtrix's .mif format with the gradient table stored in the header.
# dwi2response and dwi2fod below read it from there, so neither needs -fslgrad.
# -fslgrad bvecs bvals embed the FSL-style table (note the order: bvecs, then bvals)
# -force overwrite an existing output; MRtrix refuses to otherwise
# -quiet no progress bars, which would garble a log shared by 8 jobs
mrconvert "$d/data.nii.gz" "$d/dwi.mif" -fslgrad "$d/bvecs" "$d/bvals" -force -quiet
mrconvert "$d/nodif_brain_mask.nii.gz" "$d/mask.mif" -force -quiet
# Unsupervised WM, GM and CSF response estimation (Dhollander et al., 2016); the three
# outputs are in that order. -mask keeps the search inside the brain. A trailing
# backslash continues the command on the next line.
dwi2response dhollander "$d/dwi.mif" \
    "$d/wm_response.txt" "$d/gm_response.txt" "$d/csf_response.txt" \
    -mask "$d/mask.mif" -force -quiet
echo ">> $s response functions"
}

# Phase 3 worker. Runs multi-shell multi-tissue CSD (Jeurissen et al., 2014) with the
# group-average response functions, then intensity normalisation. Argument: participant
# ID. Writes {wm,gm,csf}_fod.mif and {wm,gm,csf}_fod_norm.mif in $PROJECT/dwi/<ID>/.
estimate_fod() {
    local s=$1
    local d="$PROJECT/dwi/$s"
    # $g is where Phase 2 put the group_*_response.txt files.
    local g="$PROJECT/dwi"
    # Restart guard, same idea as Phase 1. wm_fod_norm.mif only exists once mtnormalise
    # finished cleanly (see the rename below), so its presence really does mean "done".
    if [ "$FORCE" != 1 ] && [ -f "$d/wm_fod_norm.mif" ]; then
        echo "== $s FOD exists"
        return
    fi
    # msmt_csd takes (response, output) pairs, one per tissue; here WM, GM, CSF. The group
    # files are used, not the participant's own, so every FOD is deconvolved the same way.
    # -mask only fit voxels inside the brain mask (faster, nothing fitted to air)
    # -nthreads threads for this one command; NTHREADS x FOD_JOBS is the peak load
    dwi2fod msmt_csd "$d/dwi.mif" \
        "$g/group_wm_response.txt" "$d/wm_fod.mif" \
        "$g/group_gm_response.txt" "$d/gm_fod.mif" \
        "$g/group_csf_response.txt" "$d/csf_fod.mif" \
        -mask "$d/mask.mif" -nthreads "$NTHREADS" -force -quiet
    # The normalized WM FOD marks a finished participant, so it is written under a
    # temporary name and renamed only when mtnormalise succeeds.
    # mtnormalise (Raffelt et al., 2017) takes (input, output) pairs per tissue and scales
    # them jointly so the tissue sum comes out roughly constant across the brain; that is
    # also its bias-field correction. It puts FOD amplitudes on a common scale across
    # participants, and CUTOFF (Step 4) is an amplitude threshold. "if command; then"
    # branches on the exit status, so the mv below only runs when mtnormalise returned 0.
    if mtnormalise \
        "$d/wm_fod.mif" "$d/wm_fod_norm.partial.mif" \
        "$d/gm_fod.mif" "$d/gm_fod_norm.mif" \
        "$d/csf_fod.mif" "$d/csf_fod_norm.mif" \
        -mask "$d/mask.mif" -force -quiet; then
        # Success: rename to the real name. mv -f overwrites without asking.
        mv -f "$d/wm_fod_norm.partial.mif" "$d/wm_fod_norm.mif"
        echo ">> $s FOD"
    else
        # Failure: drop the partial file. wm_fod_norm.mif was never written, so the next run
        # redoes this participant.
        rm -f "$d/wm_fod_norm.partial.mif"
        echo "!! $s FOD estimation failed or was interrupted"
    fi
}

# Phase 1

```

```

# "${SUBJECTS[@]}" expands the array from read_subjects to one word per ID. The trailing
# & starts the function as a background job, and throttle (00_config.sh) sleeps until
# fewer than $MAXJOBS jobs are running before the loop starts the next one. MAXJOBS is 8
# in the shipped config; this phase is light enough for that.
for s in "${SUBJECTS[@]}"; do
    estimate_response "$s" &
    throttle "$MAXJOBS"
done
# Block until the last background job finishes. Do not swap in a bare "wait": the
# logging process from start_log never exits, and bash 5.0 to 5.2 would wait on it too.
wait_for_jobs

# Phase 2
# One group-average response function per tissue; responsemean just averages the text
# files it is given. Cheap, so it reruns every time regardless of FORCE.
# The * sits outside the quotes on purpose so the shell expands it; ${tissue} needs the
# braces because $tissue_response would otherwise be read as one variable name.
# Two things worth knowing. The glob picks up every folder under $PROJECT/dwi with a
# response file, not just the IDs in $SUBJECTS_FILE, so an excluded participant still
# contributes unless the folder is moved. And adding participants later and rerunning
# changes this average, but Phase 3 skips everyone who already has wm_fod_norm.mif;
# set FORCE=1 if you want the whole sample deconvolved with the same response functions.
for tissue in wm gm csf; do
    responsemean "$PROJECT"/dwi/*/${tissue}_response.txt \
        "$PROJECT/dwi/group_${tissue}_response.txt" -force -quiet
done
echo "== group response functions written"

# Phase 3 (memory-intensive: $FOD_JOBS participants at a time)
# dwi2fod on a multi-shell series uses a lot of memory, so the concurrency is lower here
# (FOD_JOBS=2 in the shipped config, against MAXJOBS=8 above). Peak load is roughly
# NTHREADS x FOD_JOBS threads; lower one of them if the machine swaps or is shared.
for s in "${SUBJECTS[@]}"; do
    estimate_fod "$s" &
    throttle "$FOD_JOBS"
done
wait_for_jobs

# Audit
# One row per participant, "ok" or "MISSING" for the file Steps 4 and 5 need. present()
# comes from 00_config.sh; $(...) substitutes what it prints into the printf. The header
# says "Subject" because that is the column name the CSVs downstream use. A MISSING
# row here should have a matching "!!" line further up in the log.
printf "\nSubject\twm_fod_norm\n"
for s in "${SUBJECTS[@]}"; do
    printf "%s\t%s\n" "$s" "$(present "$PROJECT/dwi/$s/wm_fod_norm.mif")"
done

```